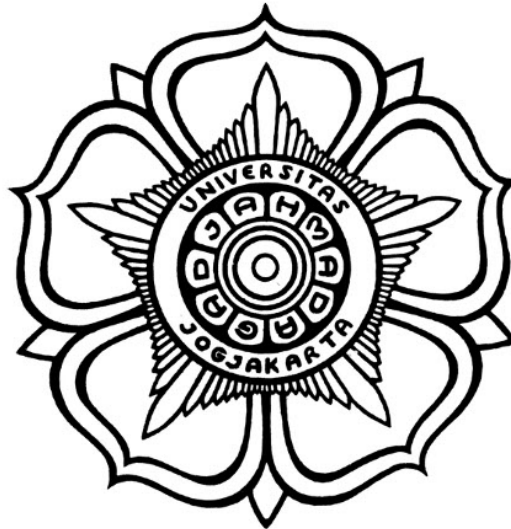


RANCANG BANGUN RUPIAH DIGITAL RITEL BERBASIS HYPERLEDGER FABRIC

SKRIPSI



***THE SUSTAINABLE DEVELOPMENT GOALS
Decent Work and Economic Growth
Industry, Innovation and Infrastructure***

Disusun oleh:

**Muhammad Fachrizal Giffari
22/504570/TK/55192**

**PROGRAM SARJANA PROGRAM STUDI TEKNOLOGI INFORMASI
DEPARTEMEN TEKNIK ELEKTRO DAN TEKNOLOGI INFORMASI
FAKULTAS TEKNIK UNIVERSITAS GADJAH MADA
YOGYAKARTA
2026**

HALAMAN PENGESAHAN

RANCANG BANGUN RUPIAH DIGITAL RITEL BERBASIS HYPERLEDGER FABRIC

SKRIPSI

Diajukan Sebagai Salah Satu Syarat untuk Memperoleh
Gelar Sarjana Teknik
pada Departemen Teknik Elektro dan Teknologi Informasi
Fakultas Teknik
Universitas Gadjah Mada

Disusun oleh:

Muhammad Fachrizal Giffari
22/504570/TK/55192

Telah disetujui dan disahkan

Pada tanggal

Dosen Pembimbing I

Dosen Pembimbing II

Teguh Bharata Adji, S.T., M.T., M.Eng., Ph.D.
NIP. 196909201995121001

Dr. Ir. Guntur Dharma Putra, S.T., M.Sc.
NIP. 111199104201802102

PERNYATAAN BEBAS PLAGIASI

Saya yang bertanda tangan di bawah ini :

Nama : Muhammad Fachrizal Giffari
NIM : 22/504570/TK/55192
Tahun terdaftar : 2022
Program : Sarjana
Program Studi : Teknologi Infomrasi
Fakultas : Teknik Universitas Gadjah Mada

Menyatakan bahwa dalam dokumen ilmiah Skripsi ini tidak terdapat bagian dari karya ilmiah lain yang telah diajukan untuk memperoleh gelar akademik di suatu lembaga Pendidikan Tinggi, dan juga tidak terdapat karya atau pendapat yang pernah ditulis atau diterbitkan oleh orang/lembaga lain, kecuali yang secara tertulis disitasi dalam dokumen ini dan disebutkan sumbernya secara lengkap dalam daftar pustaka.

Dengan demikian saya menyatakan bahwa dokumen ilmiah ini bebas dari unsur-unsur plagiasi dan apabila dokumen ilmiah Skripsi ini di kemudian hari terbukti merupakan plagiasi dari hasil karya penulis lain dan/atau dengan sengaja mengajukan karya atau pendapat yang merupakan hasil karya penulis lain, maka penulis bersedia menerima sanksi akademik dan/atau sanksi hukum yang berlaku.

Yogyakarta, 01 Juli 2026

Materai Rp10.000

(Tanda tangan)

Muhammad Fachrizal Giffari
22/504570/TK/55192

HALAMAN PERSEMBAHAN

Bismillaahirrahmaanirrahiim.

Dengan nama Allah Yang Maha Pengasih, Maha Penyayang.

Skripsi ini kupersembahkan kepada kedua orang tuaku, saudaraku, dan juga teman-temanku.

Skripsi ini juga kupersembahkan kepada almarhum Bagas Amar Hakiki.

Biarkan halaman ini menjadi saksi bahwa kamu adalah manusia yang baik.

KATA PENGANTAR

Puja dan puji syukur ke hadirat Allah Subhaanahu wa ta'aala atas limpahan rahmat, karunia, serta petunjuk-Nya sehingga tugas akhir berupa penyusunan skripsi dengan judul "Rancang Bangun Rupiah Digital Ritel Berbasis Hyperledger Fabric" ini telah terselesaikan dengan baik. Dalam hal penyusunan tugas akhir ini, penulis telah banyak mendapatkan arahan, bantuan, serta dukungan dari berbagai pihak. Oleh karena itu, pada kesempatan ini penulis mengucapkan terima kasih kepada:

1. Bapak Prof. Ir. Hanung Adi Nugroho, Ph.D., IPM, SMIEEE., selaku Ketua Departemen Teknik Elektro dan Teknologi Informasi Fakultas Teknik Universitas Gadjah Mada.
2. Bapak Dr. Ir. Ahmad Nasikun, S.T., M.Sc., IPM., selaku Ketua Program Sarjana Program Studi Teknologi Informasi Fakultas Teknik Universitas Gadjah Mada.
3. Bapak Teguh Bharata Adji, S.T., M.T., M.Eng., Ph.D., selaku Dosen Pembimbing I yang senantiasa memberikan bimbingan dan masukan yang terbaik dalam pelaksanaan serta penulisan laporan tugas akhir.
4. Bapak Dr. Ir. Guntur Dharma Putra, S.T., M.Sc., selaku Dosen Pembimbing II yang telah memberikan arahan dan masukan teknis yang terbaik dalam pelaksanaan tugas akhir.
5. Ibu Dr. Eng. Silmi Fauziati, S.T., M.T., selaku Dosen Pembimbing Akademik yang senantiasa membantu penulis perihal dinamika perkuliahan.
6. Seluruh dosen dan tenaga pendidik Departemen Teknik Elektro dan Teknologi Informasi Fakultas Teknik Universitas Gadjah Mada yang telah memberikan bekal ilmu kepada penulis selama menjalani masa perkuliahan.
7. Bapak Sujono, S.E., Ibu Mulyati, S.E., dr. Puji Arifianti Ramadhany, dan Muhammad Feizal Syafaat, S.T., selaku keluarga penulis yang senantiasa memanjatkan doa kepada penulis serta memberikan dukungan, semangat, dan motivasi kepada penulis dalam pengerjaan tugas akhir.
8. UKM UGM eSports 2024 yang telah memberikan penulis pandangan baru terkait dinamika eSports, dinamika UGM, hingga dinamika negara.
9. Cornelius Arden Satwika Hermawan, S.T., dan Mahsa Quereda Bahjah, S.T., selaku rekan kelompok mata kuliah Pemrograman Berorientasi Obyek yang terus berlanjut hingga Teknisiade, PPK Ormawa, Bengkel Inovasi, Kerja Praktik. Semoga kita dapat menjadi rekan kerja lagi di kesempatan lain.
10. WS yang telah menemani selama 24 jam dan membuat hidup penulis menjadi warna warni. Terimakasih sudah membuat kehidupan kuliah menjadi seperti tujuh belas lagi seperti yang dideskripsikan oleh Tulus, "Muda jiwa selamanya muda, Kisah kita abadi selamanya".
11. Abid, Ale, Caca, Ervina, Lintang, Nabila, Niko, Sanie, Vivian, Yajna, Abang Efrem, Bapa Poli, Abang Rinto, Abang Rino, Mama Fani, dan semuanya yang tidak bisa penulis sebutkan. Terimakasih sudah menjadi keluarga untuk penulis di Desa Iso, Kecamatan Kei Kecil Timur. Terimakasih juga sudah merayakan ulang tahun penulis dengan cara yang sangat berkesan.

12. Tim Jelajah Kei Kecil 2025, khususnya untuk Almarhum Bagas Amar Hakiki. Terimakasih sudah menjadi sahabat yang selalu menemani penulis bahkan saat berada di fase terendah.
13. Teman-teman Program Studi Teknologi Informasi Departemen Teknik Elektro dan Teknologi Informasi Universitas Gadjah Mada Angkatan 2022 yang telah menemani dan membantu penulis selama menjalani masa perkuliahan.

Tiada gading yang tak retak. Penulis menyadari bahwa skripsi ini masih jauh dari kata sempurna. Oleh karena itu, penulis sangat mengharapkan segala bentuk masukan, saran, dan kritik yang membangun. Akhir kata, penulis berharap semoga tulisan ini dapat memberikan manfaat dan wawasan tambahan bagi kita semua, aamiin

Yogyakarta, Juni 2026

Muhammad Fachrizal Giffari

DAFTAR ISI

HALAMAN PENGESAHAN	ii
PERNYATAAN BEBAS PLAGIASI	iii
HALAMAN PERSEMBAHAN	iv
KATA PENGANTAR	vi
DAFTAR ISI	vii
DAFTAR TABEL	x
DAFTAR GAMBAR	xi
DAFTAR SINGKATAN.....	xiii
INTISARI.....	xiv
ABSTRACT	xv
BAB I Pendahuluan	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	3
1.3 Tujuan Penelitian	4
1.4 Batasan Penelitian	4
1.5 Manfaat Penelitian	5
1.6 Sistematika Penulisan.....	6
BAB II Tinjauan Pustaka dan Dasar Teori	7
2.1 Tinjauan Pustaka	7
2.1.1 Penelitian Terdahulu tentang Desain dan Keamanan CBDC.....	7
2.1.2 Penelitian Terdahulu Berbasis Hyperledger Fabric.....	8
2.1.3 Posisi dan Celah Penelitian	9
2.2 Dasar Teori	10
2.2.1 Central Bank Digital Currency	10
2.2.1.1 Model Pembayaran Berbasis Akun dan Token	11
2.2.1.2 Arsitektur Penerbitan CBDC	14
2.2.2 Blockchain dan Distributed Ledger Technology	14
2.2.2.1 Struktur Blockchain	14
2.2.2.2 Algoritme Konsensus	15
2.2.2.3 Tipe Blockchain	16
2.2.3 Hyperledger Fabric.....	16
2.2.3.1 Arsitektur Hyperledger Fabric	16
2.2.3.2 Hyperledger Fabric Ledger.....	17
2.2.3.3 Alur Transaksi Hyperledger Fabric	18
2.2.4 Hyperledger Caliper	19
2.2.5 Kepatuhan: KYC, Tingkatan Dompot, dan Limit Transaksi	19

2.2.6	Transfer Ritel Langsung	20
2.2.7	Application Programming Interface	20
2.2.8	Bahasa Pemrograman Go	21
2.3	Analisis Perbandingan Metode	21
2.3.1	Perbandingan Hyperledger Fabric dengan Kerangka Kerja Blockchain Lain	21
2.3.2	Pemilihan Algoritme Konsensus Hyperledger Fabric	24
2.3.3	Pemilihan Alat Pengujian Blockchain	25
2.3.4	Pemilihan Arsitektur CBDC Dua Jenjang	26
2.3.5	Pemetaan Purwarupa terhadap Dimensi Desain CBDC Optimal ...	27
BAB III Metode Penelitian.....		30
3.1	Metode yang Digunakan.....	30
3.1.1	Metode Perancangan	30
3.1.1.1	Analisis Kebutuhan Sistem.....	30
3.1.1.2	Perancangan Activity Diagram.....	31
3.1.1.3	Perancangan Arsitektur Sistem Blockchain	31
3.1.1.4	Perancangan Basis Data Luring	44
3.1.2	Metode Implementasi	45
3.1.2.1	Software Development Life Cycle	45
3.1.2.2	Kerangka Kerja Blockchain	45
3.1.2.3	Mekanisme Konsensus	46
3.1.2.4	State Database	46
3.1.2.5	Basis Data Luring.....	46
3.1.2.6	Bahasa Pemrograman.....	48
3.1.3	Metode Pengujian	48
3.2	Alur Tugas Akhir	51
3.2.1	Mempelajari Proses Bisnis & Sistem Blockchain	52
3.2.2	Melakukan Studi Literatur.....	52
3.2.3	Menentukan Metode Penelitian	52
3.2.4	Pengembangan Sistem Blockchain bi-coin	52
3.2.4.1	Instalasi Kerangka Kerja Hyperledger Fabric	52
3.2.4.2	Konfigurasi Jaringan Blockchain	53
3.2.4.3	Deploy Chaincode	53
3.2.4.4	Pengembangan Fabric Gateway	56
3.2.4.5	Pengembangan Backend.....	56
3.2.5	Pengujian Sistem Blockchain bi-coin	57
3.2.5.1	Konfigurasi Hyperledger Caliper	58
3.2.5.2	Menjalankan Pengujian Hyperledger Caliper	58
BAB IV Hasil dan Pembahasan.....		59

4.1	Hasil Implementasi Sistem	59
4.1.1	Jaringan Hyperledger Fabric	59
4.1.2	Chaincode Digital Rupiah	59
4.1.3	Backend API	61
4.1.4	Hasil Pengujian Fungsional	62
4.2	Hasil Evaluasi Kinerja	63
4.3	Perbandingan Hasil Penelitian dengan Hasil Terdahulu	65
BAB V	Kesimpulan dan Saran	69
5.1	Kesimpulan	69
5.2	Saran	69
DAFTAR PUSTAKA		71
LAMPIRAN		L-1
L.1	Source Code Tipe Data dan Struktur <i>Chaincode</i>	L-2
L.2	Source Code Inisialisasi <i>Ledger</i>	L-6
L.3	Source Code Penerbitan dan Distribusi Likuiditas	L-7
L.4	Source Code Transfer dan Penegakan Limit	L-9
L.5	Source Code Kepatuhan KYC	L-11
L.6	Source Code Pengawasan dan Rekonsiliasi	L-12
L.7	Source Code Konfigurasi Jaringan Caliper	L-14
L.8	Source Code Konfigurasi <i>Benchmark</i> Caliper	L-16
L.9	Source Code Skrip Pembangkit Aset Jaringan Caliper	L-17
L.10	Source Code <i>Workload</i> Transfer Ritel	L-18
L.11	Source Code <i>Workload</i> Beban Dasar Ritel	L-20

DAFTAR TABEL

Tabel 2.1	Perbandingan penelitian terdahulu dengan penelitian ini.....	9
Tabel 2.2	Perbandingan sistem terpusat, <i>permissionless blockchain</i> , dan <i>permissioned blockchain</i> untuk CBDC.....	22
Tabel 2.3	Perbandingan platform DLT <i>permissioned</i> untuk sistem CBDC.....	23
Tabel 2.4	Perbandingan algoritme konsensus untuk <i>blockchain permissioned</i> ..	24
Tabel 2.5	Perbandingan Hyperledger Caliper dan BlockBench. Diadaptasi dari Hanif [1] dan sumber primer [2,3].....	25
Tabel 2.6	Perbandingan arsitektur CBDC satu jenjang dan dua jenjang.....	26
Tabel 2.7	Pemetaan purwarupa terhadap dimensi desain CBDC optimal [4] ...	27
Tabel 3.1	Organisasi pada jaringan Rupiah Digital	41
Tabel 3.2	Objek status utama pada ledger.....	43
Tabel 3.3	Perbandingan basis data status Hyperledger Fabric. Diadaptasi dari Hanif [1] dan dokumentasi resmi Hyperledger Fabric [5].	46
Tabel 3.4	Perbandingan basis data kandidat untuk penyimpanan luring KYC. .	48
Tabel 3.5	Skenario beban pengujian kinerja	49
Tabel 4.1	Kelompok fungsi chaincode digital-rupiah.....	59
Tabel 4.2	Batas tingkatan dompet ritel (dalam Rupiah)	61
Tabel 4.3	Peran akses pada backend API	62
Tabel 4.4	Konfigurasi evaluasi transfer ritel.....	64
Tabel 4.5	Hasil evaluasi transfer ritel Hyperledger Caliper	64
Tabel 4.6	Hasil evaluasi fungsi lain Hyperledger Caliper	65
Tabel 4.7	Perbandingan purwarupa MeetCoin dan purwarupa Rupiah Digital ritel. Sumber penelitian terdahulu: Hanif [1].....	66
Tabel 4.8	Perbandingan penelitian ini dengan tinjauan literatur Aryani (2023) dan PoC Proyek Garuda (2024).	68

DAFTAR GAMBAR

Gambar 2.1	Arsitektur sistem <i>blockchain</i> MeetCoin sebagai penelitian terdahulu. Sumber: Hanif [1].	8
Gambar 2.2	Taksonomi CBDC berdasarkan segmen pengguna dan model distribusi; model dua jenjang yang diadopsi pada penelitian ini ditampilkan dengan latar abu-abu sesuai rancangan Rupiah Digital Bank Indonesia	11
Gambar 2.4	Perbandingan model pembayaran CBDC berbasis akun dan berbasis token: model akun memverifikasi identitas pemegang, sedangkan model token memverifikasi keaslian objek nilai. Diadaptasi dari Auer dan Böhme [6].	12
Gambar 2.3	Peta jalan pengembangan Rupiah Digital dalam Proyek Garuda. Sumber: Bank Indonesia [7].	13
Gambar 2.5	Konfigurasi desain Rupiah Digital menurut White Paper CBDC Bank Indonesia. Sumber: Bank Indonesia [7].	13
Gambar 2.6	Perbandingan arsitektur penerbitan CBDC: tidak langsung (<i>indirect</i> /dua jenjang), langsung (<i>direct</i> /satu jenjang), dan hibrida (<i>hybrid</i>). Diadaptasi dari Auer dan Böhme [6]	14
Gambar 2.7	Struktur rantai blok: setiap blok memuat <i>hash</i> kriptografis dari blok sebelumnya	15
Gambar 2.8	Pohon Merkle: <i>hash</i> transaksi digabungkan secara berjenjang membentuk Merkle Root yang disimpan dalam kepala blok	15
Gambar 2.9	Diagram transisi <i>state</i> algoritme konsensus Raft: setiap simpul berstatus <i>Follower</i> , <i>Candidate</i> , atau <i>Leader</i>	16
Gambar 2.10	Arsitektur komponen Hyperledger Fabric: klien berinteraksi dengan <i>peer endorser</i> , transaksi diurutkan oleh <i>orderer</i> berbasis Raft, lalu didistribusikan ke semua <i>peer</i> dalam <i>channel</i>	17
Gambar 2.11	Dualitas komponen <i>ledger</i> Hyperledger Fabric: <i>blockchain</i> sebagai catatan transaksi historis yang tidak dapat diubah, dan <i>world state</i> sebagai representasi status terkini pada CouchDB	18
Gambar 2.12	Siklus transaksi Hyperledger Fabric dengan mekanisme konsensus Raft	19
Gambar 2.13	Perbandingan <i>throughput</i> transfer aset pada Ethereum, Hyperledger Fabric, dan Besu. Sumber: [8].	24
Gambar 2.14	Arsitektur Rupiah Digital yang diusulkan Aryani [9]	28
Gambar 3.15	Siklus SDLC dengan metode <i>prototyping</i> : empat fase berulang hingga purwarupa memenuhi kebutuhan fungsional dan kinerja	30
Gambar 3.23	Arsitektur dua jenjang sistem Rupiah Digital	31
Gambar 3.16	Activity diagram pendaftaran peserta	32
Gambar 3.17	Activity diagram penerbitan Digital Rupiah	33
Gambar 3.18	Activity diagram distribusi likuiditas ke PJP	34
Gambar 3.19	Activity diagram onboarding KYC pelanggan	35
Gambar 3.20	Activity diagram transfer saldo ritel	36
Gambar 3.21	Activity diagram pembekuan dan pencairan peserta	37
Gambar 3.22	Activity diagram laporan dan pengawasan supervisi	39

Gambar 3.24	Arsitektur teknologi Rupiah Digital menurut White Paper CBDC Bank Indonesia. Sumber: Bank Indonesia [7].	39
Gambar 3.25	Arsitektur sistem Rupiah Digital: lapisan <i>client</i> , <i>backend API</i> (Go REST + JWT/RBAC), basis data PostgreSQL luring, <i>Fabric Gateway SDK</i> , serta jaringan Hyperledger Fabric 2.5 dengan lima organisasi <i>peer</i> , CouchDB, CA per organisasi, dan <i>orderer Raft</i>	40
Gambar 3.26	Arsitektur jaringan Hyperledger Fabric sistem Rupiah Digital (5 organisasi)	41
Gambar 3.27	Diagram use case keseluruhan sistem Rupiah Digital	42
Gambar 3.28	Diagram use case operasi wholesale	42
Gambar 3.29	Diagram use case operasi retail	43
Gambar 3.30	Diagram <i>Entity-Relationship</i> basis data luring (<i>off-chain</i>) pada PostgreSQL. Tabel <i>auth_users</i> berdiri sendiri untuk autentikasi portal, sedangkan <i>kyc_profiles</i> menjadi pusat relasi profil KYC, riwayat pemeriksaan penyedia, dan jejak audit.	45
Gambar 3.31	Alur pelaksanaan tugas akhir: dari studi literatur hingga penarikan kesimpulan, dengan umpan balik iteratif antara implementasi dan pengujian	52
Gambar 3.32	Implementasi fungsi <i>Mint</i> pada <i>chaincode</i> : memverifikasi status beku dan batas saldo tingkatan sebelum menambah saldo dompet	54
Gambar 3.33	Penegakan kebijakan transfer bersama untuk batas keluar, batas masuk, saldo maksimum, dan kecukupan saldo	55
Gambar 3.34	Penurunan tingkatan dompet ritel dari jenis subjek, risiko, dan tingkat uji tuntas KYC	56
Gambar 3.35	Pengelompokan rute REST <i>backend</i> berdasarkan peran akses: setiap subrouter menerapkan <i>middleware RequireRole</i> yang berbeda sesuai hierarki wewenang	57
Gambar 4.36	Siklus hidup peserta pada <i>chaincode</i> : dari <i>pending</i> (menunggu persetujuan) menjadi <i>active</i> , kemudian dapat dibekukan (<i>frozen</i>) dan dicairkan kembali. Penolakan (<i>RejectParticipant</i>) maupun pelepasan keanggotaan (<i>OffboardParticipant</i>) menghapus rekaman peserta dari <i>ledger</i> (<i>DelState</i>), bukan menyimpannya sebagai status	60

DAFTAR SINGKATAN

API	=	<i>Application Programming Interface</i>
AML	=	<i>Anti-Money Laundering</i>
BI	=	Bank Indonesia
BSPI	=	Blueprint Sistem Pembayaran Indonesia
CBDC	=	<i>Central Bank Digital Currency</i>
CA	=	<i>Certificate Authority</i>
CFT	=	<i>Combating the Financing of Terrorism</i>
DLT	=	<i>Distributed Ledger Technology</i>
KYC	=	<i>Know Your Customer</i>
MSP	=	<i>Membership Service Provider</i>
OJK	=	Otoritas Jasa Keuangan
PJP	=	Penyedia Jasa Pembayaran
REST	=	<i>Representational State Transfer</i>
RTGS	=	<i>Real-Time Gross Settlement</i>
TEE	=	<i>Trusted Execution Environment</i>
TLS	=	<i>Transport Layer Security</i>
TPS	=	<i>Transactions Per Second</i>
UTXO	=	<i>Unspent Transaction Output</i>

INTISARI

Mata uang digital bank sentral (*Central Bank Digital Currency*, CBDC) merupakan inisiatif strategis Bank Indonesia melalui Proyek Garuda. Eksperimen Proyek Garuda yang terdokumentasi publik berfokus pada CBDC grosir, sehingga masih terdapat celah teknis untuk purwarupa ritel yang dapat diuji dan menggabungkan distribusi dua jenjang, uji tuntas nasabah berbasis risiko, limit dompet deterministik, dan pengawasan institusional. Penelitian ini merancang dan mengimplementasikan purwarupa Rupiah Digital ritel pada jaringan *permissioned* Hyperledger Fabric.

Metode *prototyping* mencakup analisis kebutuhan, perancangan arsitektur, implementasi, serta pengujian fungsional dan kinerja. *Chaincode* Go hanya menyimpan status KYC, risiko, tingkat uji tuntas, persetujuan pejabat senior, masa berlaku, dan hash dokumen; data identitas mentah tetap berada di PostgreSQL. Tingkatan dompet ritel diturunkan dari profil KYC yang disetujui dan tidak dipilih oleh pengguna. Kebijakan BASIC, STANDARD, dan MERCHANT menegakkan batas per transaksi, pengeluaran harian, pengeluaran bulanan, penerimaan bulanan, dan saldo maksimum menggunakan stempel waktu transaksi Fabric. Dompet institusional Bank Indonesia, bank, dan penyedia jasa pembayaran berada di luar tingkatan ritel serta menggunakan batas tingkat sistem. Hyperledger Caliper dikonfigurasi dengan beban khusus transfer yang terdiri atas 70% transfer antarpelanggan, 25% transfer pelanggan-ke-pedagang, dan 5% pembacaan.

Pengujian otomatis memverifikasi penurunan tingkatan, kewajiban uji tuntas lanjutan untuk risiko tinggi, penegakan batas masuk dan keluar, pemisahan data privat, serta batas peran. Purwarupa memisahkan tanggung jawab secara eksplisit: pelanggan memberikan data identitas, bank atau PJP melakukan KYC, sedangkan Bank Indonesia mengawasi institusi dan peristiwa buku besar. Klaim kinerja dibatasi pada pengukuran yang dihasilkan oleh protokol Caliper yang dapat direproduksi. Purwarupa ini menjadi dasar teknis yang dapat diaudit untuk mengkaji kebijakan Rupiah Digital ritel tanpa mengklaim kesiapan produksi atau kesetaraan limit dengan ketentuan hukum.

Kata kunci : Rupiah Digital, CBDC ritel, Hyperledger Fabric, KYC berbasis risiko, limit dompet

ABSTRACT

Central Bank Digital Currency (CBDC) is a strategic Bank Indonesia initiative under Project Garuda. Publicly documented Project Garuda experiments focus on wholesale CBDC, leaving a technical gap for a testable retail prototype that combines two-tier distribution, risk-based customer due diligence, deterministic wallet limits, and institutional supervision. This study designs and implements a retail Digital Rupiah prototype on a permissioned Hyperledger Fabric network.

The prototyping method covers requirements analysis, architecture design, implementation, and functional and performance testing. Go chaincode stores only KYC status, risk, due-diligence level, senior-approval status, expiry, and document hashes; raw identity data remains in PostgreSQL. Retail wallet tiers are derived from the approved KYC profile rather than selected by a user. BASIC, STANDARD, and MERCHANT policies enforce per-transaction, daily outgoing, monthly outgoing, monthly incoming, and maximum-balance limits using the Fabric transaction timestamp. Institutional Bank Indonesia, bank, and payment-service-provider wallets remain outside retail tiering and use system-level limits. Hyperledger Caliper is configured for a transfer-only workload consisting of 70% customer-to- customer transfers, 25% customer-to-merchant transfers, and 5% reads.

Automated tests verify tier derivation, high-risk enhanced-due-diligence rules, incoming and outgoing limit enforcement, privacy separation, and role boundaries. The resulting prototype demonstrates a coherent separation of responsibilities: customers provide identity data, banks or payment service providers perform KYC, and Bank Indonesia supervises institutions and ledger events. Performance claims are limited to reproducible measurements produced by the specified Caliper protocol. The prototype therefore provides an auditable technical basis for studying retail Digital Rupiah policy, without claiming production readiness or legal limit equivalence.

Keywords : Digital Rupiah, retail CBDC, Hyperledger Fabric, risk-based KYC, wallet limits

BAB I

PENDAHULUAN

1.1 Latar Belakang

Transformasi digital pada sektor sistem pembayaran Indonesia berlangsung sangat cepat dalam satu dekade terakhir. Penetrasi telepon pintar dan pertumbuhan ekonomi digital mendorong masyarakat beralih dari uang tunai menuju instrumen pembayaran nontunai. Bank Indonesia melalui Blueprint Sistem Pembayaran Indonesia (BSPI) menempatkan digitalisasi sebagai agenda utama untuk menjaga kedaulatan moneter, mendorong inklusi keuangan, serta memperkuat integrasi ekonomi dan keuangan digital nasional [10]. Dalam konteks tersebut, kehadiran mata uang digital bank sentral atau *Central Bank Digital Currency* (CBDC) menjadi salah satu inisiatif strategis untuk melengkapi uang kartal dan uang elektronik yang telah ada.

Pertumbuhan ekosistem digital tersebut, bagaimanapun, juga membawa risiko baru bagi stabilitas sistem keuangan. Pada tahun 2022, penetrasi internet Indonesia mencapai 74% dengan 204,7 juta pengguna, sementara pelanggan seluler mencapai 370,1 juta atau setara 133% populasi; pandemi COVID-19 menambah 21 juta konsumen digital baru yang sebelumnya belum terlayani secara daring sebagaimana dirangkum dalam Tabel 1 White Paper CBDC Bank Indonesia [7]. Di sisi lain, kapitalisasi pasar aset kripto global melonjak hingga hampir 3 triliun dolar AS, tumbuh 1.393% antara 2019 dan 2021, sementara investor aset kripto di Indonesia mencapai 16,3 juta per September 2022 dengan laju pertumbuhan 81,6% *year-over-year* [7]. Fenomena ini memunculkan *cryptoization*, yaitu kondisi di mana adopsi aset kripto oleh penduduk menyebar lebih cepat dibandingkan mata uang resmi [7]. Lebih jauh, disrupsi digital tidak lagi terbatas pada isu *shadow banking*, melainkan telah merambah pada fenomena *shadow currency* dan *shadow central banking* yang mengikis efektivitas kebijakan bank sentral [7]. *Stablecoin* dan aset kripto tanpa aset dasar memperbesar risiko pencucian uang (*money laundering*), pendanaan terorisme, serta risiko makrofinansial berupa melemahnya kendali otoritas atas stabilitas moneter dan sistem keuangan [7]. Dalam menghadapi tantangan tersebut, kehadiran mata uang digital bank sentral menjadi instrumen strategis untuk menjaga kepercayaan publik terhadap uang, mempertahankan kedaulatan moneter, dan memastikan bank sentral tetap relevan sebagai jangkar stabilitas di era ekonomi digital [7, 11].

CBDC merupakan representasi digital dari uang yang diterbitkan dan menjadi kewajiban langsung bank sentral, sehingga memiliki kualitas bebas risiko yang setara dengan uang tunai [12]. Berbeda dengan uang elektronik yang diterbitkan oleh penyelenggara swasta dan merupakan kewajiban penerbitnya, CBDC menempatkan

bank sentral sebagai penjamin nilai akhir. Bagi Indonesia, Bank Indonesia menamai inisiatif CBDC sebagai Rupiah Digital melalui Proyek Garuda, yang menempuh pendekatan bertahap dimulai dari Rupiah Digital *wholesale* (w-Digital Rupiah) untuk transaksi antarlembaga, kemudian diperluas menuju Rupiah Digital *retail* (r-Digital Rupiah) bagi masyarakat luas [7, 13]. Desain Rupiah Digital mengadopsi model dua jenjang (*two-tier*), yaitu Bank Indonesia menerbitkan dan mengelola Rupiah Digital pada tingkat *wholesale*, sementara distribusi kepada masyarakat dilakukan melalui pihak perantara seperti perbankan dan penyedia jasa pembayaran [14].

Salah satu pilihan teknologi yang banyak dikaji untuk menopang CBDC adalah *Distributed Ledger Technology* (DLT) berbasis *blockchain*. Teknologi ini menawarkan keunggulan berupa integritas data yang terjaga melalui catatan yang tidak dapat diubah (*immutable*), jejak audit yang transparan, serta kemampuan otomasi aturan bisnis melalui *smart contract* [15, 16]. Untuk kebutuhan bank sentral, jenis *blockchain* yang sesuai adalah *permissioned blockchain*, yaitu jaringan dengan keanggotaan terbatas dan teridentifikasi, sehingga bank sentral tetap memegang kendali atas tata kelola jaringan [17]. Hyperledger Fabric merupakan salah satu kerangka kerja *permissioned blockchain* paling matang yang banyak digunakan pada ranah keuangan. Fabric mendukung arsitektur modular, identitas berbasis *Membership Service Provider* (MSP), pemisahan transaksi melalui kanal (*channel*), serta model eksekusi *execute-order-validate* yang memungkinkan throughput tinggi tanpa bergantung pada mekanisme *proof-of-work* yang boros energi [18].

Meskipun kajian mengenai CBDC berkembang pesat, fokus literturnya masih timpang. Sebagian besar penelitian menelaah CBDC dari sisi makroekonomi, kebijakan moneter, dan stabilitas keuangan [11, 12, 19], atau dari sisi taksonomi desain tingkat tinggi seperti pilihan *token-based* versus *account-based* dan model satu jenjang versus dua jenjang [6, 20–22]. Sebaliknya, artefak teknis yang utuh, terbuka, dan dapat diuji secara empiris untuk konteks CBDC ritel masih jarang tersedia.

Pada ranah teknis pun, penelitian terdahulu cenderung menyoroti satu aspek secara terpisah. Sejumlah studi berfokus pada privasi dan auditabilitas [23–25], keamanan dan pemodelan ancaman [26], atau arsitektur jaringan [17, 27]. Padahal kelayakan sebuah sistem CBDC ritel justru ditentukan oleh keterpaduan aspek-aspek tersebut. Belum banyak penelitian yang mengintegrasikan penerbitan dan distribusi likuiditas berjenjang, kepatuhan KYC dan limit berbasis tingkatan dompet, transfer ritel, serta pengawasan ke dalam satu sistem yang utuh sekaligus terukur kinerjanya.

Untuk konteks Indonesia, Bank Indonesia telah menetapkan arah desain Rupiah Digital berupa model dua jenjang dan interoperabilitas sistem pembayaran [7, 14], dan sejumlah kajian merumuskan desain CBDC yang sesuai bagi perekonomian Indonesia [9, 28, 29]. Namun, realisasi teknis Proyek Garuda yang terdokumentasi

secara publik baru mencakup lapisan *wholesale* [13], sedangkan purwarupa lapisan *retail* yang selaras dengan spesifikasi tersebut belum tersedia secara terbuka. Dari sisi evaluasi kinerja, CBDC ritel yang telah diluncurkan maupun yang masih berupa pilot (seperti eNaira di Nigeria [30] dan pilot ritel di Thailand [31]) belum menyertakan tolok ukur kinerja terstandar yang dapat direproduksi, sementara sistem berkinerja tinggi seperti Hamilton/OpenCBDC justru meninggalkan model *blockchain* dan memakai arsitektur *account-based* terpusat [32]. Akibatnya, data *throughput* dan latensi yang terukur untuk skenario CBDC ritel di atas *permissioned blockchain*, khususnya untuk Rupiah Digital, masih langka.

Penelitian sebelumnya di lingkungan Departemen Teknik Elektro dan Teknologi Informasi telah menunjukkan kelayakan pembangunan sistem *blockchain* berbasis Hyperledger Fabric untuk aplikasi token sederhana [1], namun belum mencakup karakteristik khusus CBDC ritel. Dengan demikian, celah penelitian ini terletak pada irisan keempat hal di atas, yaitu belum tersedianya purwarupa CBDC ritel yang sekaligus (i) mengintegrasikan penerbitan berjenjang, kepatuhan, transfer ritel, serta pengawasan dalam satu sistem; (ii) selaras dengan rancangan dua jenjang Bank Indonesia; dan (iii) dievaluasi kinerjanya secara dapat direproduksi menggunakan Hyperledger Caliper. Berdasarkan celah tersebut, penelitian ini melakukan rancang bangun sistem Rupiah Digital ritel berbasis Hyperledger Fabric yang merepresentasikan ekosistem multipihak (Bank Indonesia, bank validator, penyedia jasa pembayaran (PJP), pedagang, pelanggan ritel, pengawas, serta pengamat dari Otoritas Jasa Keuangan (OJK)) dilengkapi mekanisme pengawasan terdesentralisasi. Sistem yang dibangun selanjutnya dievaluasi kinerjanya menggunakan Hyperledger Caliper untuk mengukur *throughput* dan latensi pada berbagai skenario beban, sehingga dapat diketahui kelayakannya sebagai purwarupa infrastruktur Rupiah Digital.

1.2 Rumusan Masalah

Berdasarkan latar belakang di atas, rumusan masalah pada penelitian ini adalah sebagai berikut.

1. Bagaimana merancang dan membangun sistem Rupiah Digital ritel berbasis Hyperledger Fabric yang mendukung model penerbitan dan distribusi likuiditas berjenjang dari Bank Indonesia menuju bank, penyedia jasa pembayaran, dan pelanggan ritel?
2. Bagaimana menerapkan kontrol kepatuhan pada lapisan *chaincode* berupa KYC berbasis risiko, penurunan tingkatan dompet, limit transaksi, dan pengawasan transaksi?
3. Bagaimana kinerja sistem yang dibangun dari sisi *throughput* dan latensi pada berbagai skenario beban transaksi yang diukur menggunakan Hyperledger Caliper?

1.3 Tujuan Penelitian

Tujuan yang ingin dicapai pada penelitian ini adalah sebagai berikut.

1. Merancang dan membangun purwarupa sistem Rupiah Digital ritel berbasis Hyperledger Fabric dengan arsitektur multipihak yang mendukung penerbitan dan distribusi likuiditas berjenjang.
2. Mengimplementasikan kontrol kepatuhan pada *chaincode* berupa KYC berbasis risiko, penurunan tingkatan dompet, limit transaksi masuk dan keluar, serta pengawasan transaksi.
3. Mengevaluasi kinerja sistem dari sisi *throughput* dan latensi pada berbagai skenario beban menggunakan Hyperledger Caliper.

1.4 Batasan Penelitian

1. Sistem yang dibangun merupakan purwarupa (*prototype*) yang dijalankan pada lingkungan jaringan tersimulasi secara lokal menggunakan *container*, bukan sistem produksi.
2. Jaringan *blockchain* menggunakan Hyperledger Fabric versi 2.5 dengan lima organisasi peserta dan mekanisme konsensus *ordering* berbasis Raft (etcdraft).
3. Penelitian berfokus pada perancangan, implementasi, dan evaluasi kinerja fungsional; aspek hukum, kebijakan moneter, dan dampak makroekonomi dari penerapan Rupiah Digital berada di luar cakupan. Analisis strategi kebijakan moneter dan dampak makroekonomi CBDC—termasuk peran *CBDC rate*—didelegasikan pada studi terpisah seperti [4].
4. Evaluasi kinerja dilakukan menggunakan Hyperledger Caliper dengan beban yang dihasilkan oleh sejumlah *worker* pada lingkungan pengujian lokal.
5. Pengujian dijalankan pada satu inang (*single host*) dengan dua *worker* Caliper dan satu *peer* per organisasi, tanpa latensi jaringan dan persebaran geografis yang nyata. Karena itu, angka *throughput* dan latensi yang diperoleh tidak dapat langsung digeneralisasi ke perangkat keras maupun topologi produksi [32, 33].
6. Komposisi beban transaksi dan distribusi nilai yang digunakan merupakan asumsi desain yang menyerupai pola pembayaran ritel di Indonesia, bukan jejak transaksi empiris. Pada setiap *worker*, ronde pemanasan memakai 100 pelanggan dan 20 pedagang, sedangkan ronde stabil dan puncak masing-masing memakai 600 pelanggan dan 80 pedagang.
7. Evaluasi kinerja menggunakan skenario tanpa perebutan dompet (*contention-free*), yaitu setiap kunci dompet yang dimutasi dipilih secara deterministik dan digunakan paling banyak satu kali dalam satu ronde terukur. Skenario *hot-wallet contention*,

seperti banyak pelanggan yang membayar pedagang yang sama secara bersamaan, tidak diuji. Pembatasan ini dipilih untuk memisahkan kapasitas dasar pemrosesan jaringan dari konflik MVCC akibat konsentrasi akses. Penelitian juga tidak memiliki jejak transaksi empiris untuk menetapkan tingkat konsentrasi maupun kebijakan *retry/backoff* yang representatif. Oleh karena itu, hasil kinerja menggambarkan kondisi terbaik tanpa perebutan kunci dan tidak mewakili beban pedagang populer atau dompet bersama.

8. Purwarupa ini tidak mengimplementasikan pembayaran CBDC tanpa koneksi jaringan. Kemampuan tersebut memerlukan desain perangkat aman, verifikasi kriptografis, pencegahan *double-spending*, dan rekonsiliasi setelah konektivitas kembali [34, 35]. Dengan demikian, hasil penelitian hanya merepresentasikan transaksi yang diselesaikan melalui koneksi ke *ledger*.
9. Proses KYC disimulasikan secara terprogram tanpa integrasi penyedia identitas nyata, dan sistem belum menyediakan privasi transaksi (tanpa *private data collection* maupun teknik kriptografi penjaga privasi) sehingga seluruh *ledger* bersifat transparan bagi peserta berwenang. Dengan demikian, kompromi (*trade-off*) antara privasi dan keterlacakan belum ditangani [24, 36, 37].
10. Proyek Garuda menetapkan tiga prinsip 3i sebagai prasyarat desain Rupiah Digital: integrasi dengan infrastruktur pasar keuangan (IPK) domestik, interoperabilitas teknis dan semantik dengan sistem yang ada maupun yang akan datang, serta interkoneksi lintas batas untuk jaringan CBDC antarnegara [7]. Purwarupa ini memenuhi prinsip tersebut hanya pada tataran internal: alur penerbitan–distribusi–pembayaran–pengawasan berjalan secara terpadu dalam satu jaringan Hyperledger Fabric. Namun, integrasi eksternal dengan BI-RTGS dan BI-FAST hanya dimodelkan sebagai titik masuk (*stub*) tanpa koneksi nyata, Penyimpan Digital Rupiah (KDR) belum diimplementasikan, konformitas terhadap standar SNAP API nasional tidak diterapkan, dan tidak ada gerbang (*gateway*) menuju IPK domestik maupun jaringan CBDC lintas batas. Dengan demikian, pemenuhan prinsip 3i secara penuh merupakan pekerjaan lanjutan yang berada di luar cakupan purwarupa ini.

1.5 Manfaat Penelitian

Penelitian ini diharapkan memberikan manfaat sebagai berikut.

1. **Manfaat teoretis**, yaitu memberikan referensi rancang bangun sistem CBDC ritel berbasis *permissioned blockchain* yang utuh, meliputi model penerbitan berjenjang, kontrol kepatuhan, transfer ritel, dan pengawasan, sehingga dapat menjadi acuan bagi penelitian lanjutan.

2. **Manfaat praktis**, yaitu menyediakan purwarupa yang dapat digunakan sebagai bahan eksperimen dan diskusi teknis bagi pemangku kepentingan dalam pengembangan Rupiah Digital, khususnya pada aspek arsitektur teknis dan evaluasi kinerja.

1.6 Sistematika Penulisan

Penulisan skripsi ini disusun ke dalam lima bab dengan sistematika sebagai berikut.

BAB I PENDAHULUAN berisi latar belakang, rumusan masalah, tujuan penelitian, batasan penelitian, manfaat penelitian, dan sistematika penulisan.

BAB II TINJAUAN PUSTAKA DAN DASAR TEORI memaparkan tinjauan terhadap penelitian terdahulu yang relevan serta dasar teori yang mendasari penelitian, meliputi CBDC, *blockchain* dan DLT, Hyperledger Fabric, Hyperledger Caliper, KYC berbasis risiko, dan limit transaksi.

BAB III METODE PENELITIAN menguraikan alat dan bahan, metode perancangan, metode implementasi, metode pengujian, serta alur penelitian yang ditempuh.

BAB IV HASIL DAN PEMBAHASAN menyajikan hasil implementasi sistem, hasil evaluasi kinerja menggunakan Hyperledger Caliper, serta perbandingan dengan penelitian terdahulu.

BAB V KESIMPULAN DAN SARAN memuat kesimpulan yang menjawab rumusan masalah serta saran untuk pengembangan penelitian selanjutnya.

BAB II

TINJAUAN PUSTAKA DAN DASAR TEORI

2.1 Tinjauan Pustaka

Kajian mengenai penerapan *blockchain* untuk mata uang digital bank sentral telah berkembang dengan beragam fokus. Bagian ini meninjau sejumlah penelitian terdahulu yang relevan dengan rancang bangun sistem Rupiah Digital ritel berbasis Hyperledger Fabric, baik dari sisi arsitektur, kepatuhan, transfer ritel, pengawasan, maupun evaluasi kinerja.

2.1.1 Penelitian Terdahulu tentang Desain dan Keamanan CBDC

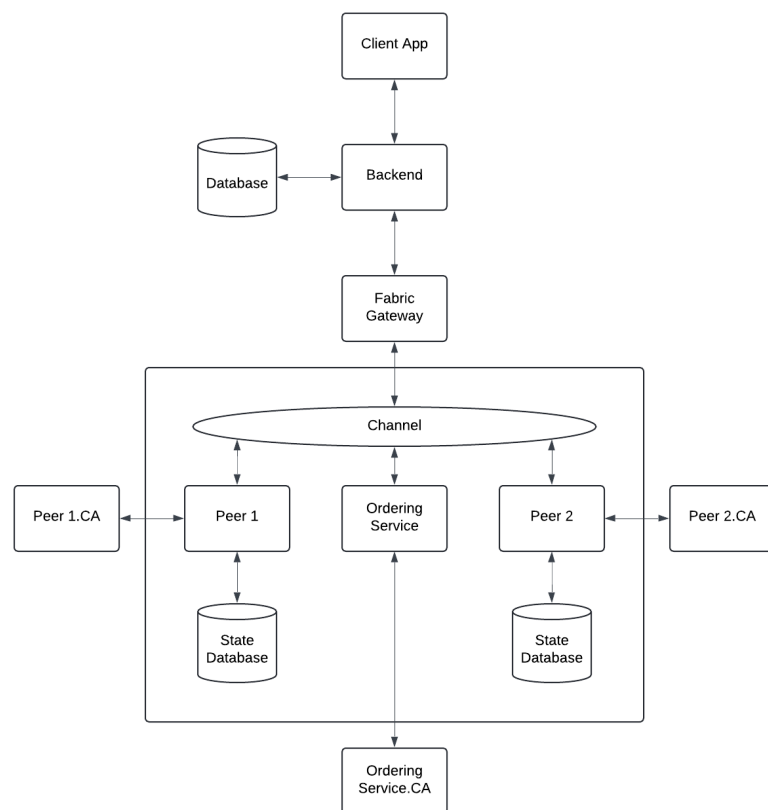
Sethaput dan Innet [16] menyusun *systematization of knowledge* (SoK) mengenai aplikasi *blockchain* pada CBDC dan mengidentifikasi bahwa pilihan antara *token-based* dan *account-based*, model satu jenjang versus dua jenjang, serta tingkat privasi merupakan dimensi desain yang paling menentukan. Bhawana dan Kumar [17] membangun jaringan CBDC berbasis *permissioned blockchain* dan menegaskan bahwa keanggotaan terbatas memberikan kendali tata kelola yang dibutuhkan bank sentral tanpa mengorbankan transparansi. Sun dkk. [27] mengusulkan model *multi-blockchain* untuk memisahkan beban transaksi antarwilayah, sedangkan Islam dan In [23] menggunakan *consortium blockchain* dengan model *unspent transaction output* (UTXO) untuk menjaga privasi sekaligus auditabilitas.

Pada ranah kepatuhan, sejumlah studi menyoroti pentingnya penanaman aturan *Anti-Money Laundering* dan *Combating the Financing of Terrorism* (AML/CFT) langsung pada desain CBDC ritel [38], serta model audit CBDC berbasis *blockchain* yang belajar dari pengalaman *Digital Currency Electronic Payment* (DCEP) Tiongkok [39]. Hans dkk. [26] menerapkan pemodelan ancaman STRIDE untuk mengidentifikasi risiko keamanan pada sistem CBDC berbasis *blockchain*, menegaskan perlunya analisis keamanan sejak tahap perancangan. Block-PAD [40] menawarkan kerangka transaksi CBDC yang tangguh dan fleksibel dengan memanfaatkan identitas digital.

Pada sisi ketahanan layanan, Project Polaris dari *BIS Innovation Hub* menunjukkan bahwa dukungan pembayaran tanpa konektivitas memerlukan desain perangkat aman dan kontrol kriptografis khusus [34, 35]; kemampuan ini diposisikan sebagai keterbatasan dan arah pengembangan lanjutan pada penelitian ini.

2.1.2 Penelitian Terdahulu Berbasis Hyperledger Fabric

Dari sisi platform dan kinerja, Androulaki dkk. [18] memperkenalkan Hyperledger Fabric sebagai *distributed operating system* untuk *permissioned blockchain* dengan arsitektur *execute-order-validate*. Sejumlah penelitian mengevaluasi kinerja Fabric pada beragam beban, termasuk pemodelan latensi [41], evaluasi kerangka berbasis Fabric [42], serta perbandingan *throughput* antara *blockchain permissioned* dan *permissionless* [43]. Di lingkungan Departemen Teknik Elektro dan Teknologi Informasi, Hanif [1] membangun sistem *blockchain* aplikasi MeetCoin menggunakan Hyperledger Fabric dan mengevaluasi kinerjanya dengan Hyperledger Caliper, namun terbatas pada token sederhana tanpa karakteristik CBDC ritel.



Gambar 2.1. Arsitektur sistem *blockchain* MeetCoin sebagai penelitian terdahulu. Sumber: Hanif [1].

Gambar 2.1 menunjukkan arsitektur pendahulu yang menghubungkan aplikasi klien, *backend*, Fabric Gateway, dua *peer*, dan satu *ordering service*. Penelitian ini menggunakan komponen dasar serupa, tetapi memperluas jaringan menjadi organisasi BI, bank, PJP, dan pengawas serta menambahkan kebijakan khusus CBDC ritel.

2.1.3 Posisi dan Celah Penelitian

Sintesis atas tinjauan tersebut memperlihatkan bahwa penelitian terdahulu terkelompok menurut fokusnya: arsitektur dan tata kelola jaringan [17, 27], privasi dan auditabilitas [23–25], keamanan dan kepatuhan [26, 38, 39], evaluasi kinerja Hyperledger Fabric [18, 33, 41–43], serta perumusan desain CBDC bagi konteks Indonesia [9, 28, 29]. Masing-masing kelompok menjawab sebagian persoalan, tetapi belum ada yang memadukannya menjadi satu sistem ritel yang utuh sekaligus terukur kinerjanya.

Celah penelitian. Dari sintesis tersebut teridentifikasi empat hal yang belum dipenuhi secara bersamaan oleh satu penelitian: (1) integrasi penerbitan dan distribusi likuiditas berjenjang, kepatuhan KYC dan limit berbasis tingkatan dompet, transfer antarpelanggan dan pelanggan-ke-pedagang, serta pengawasan dalam satu artefak; (2) keselarasan dengan rancangan dua jenjang Rupiah Digital Bank Indonesia, mengingat Proyek Garuda yang terdokumentasi publik baru menjangkau lapisan *wholesale* [13]; (3) ketersediaan tolok ukur kinerja yang dapat direproduksi untuk CBDC ritel di atas *permissioned blockchain*, yang masih langka baik pada sistem terdeploy maupun pilot [30–32]; dan (4) demonstrasi yang dapat diuji secara empiris, bukan sekadar rancangan konseptual. Penelitian ini menempati irisan keempat hal tersebut. Tabel 2.1 merangkum posisi penelitian ini terhadap penelitian terdahulu.

Tabel 2.1. Perbandingan penelitian terdahulu dengan penelitian ini

Penelitian	Platform	Fokus utama	Integrasi fitur ritel	Tolok ukur kinerja
Bhawana & Kumar [17]	Permissioned	Arsitektur CBDC	Tidak	Terbatas
Islam & In [23]	Consortium	Privasi (UTXO)	Tidak	Tidak
Hans dkk. [26]	Umum	Keamanan (STRIDE)	Tidak	Tidak
Auer & Böhme; Tsareva & Komarov [6, 22]	Agnostik	Taksonomi desain	Tidak (konsep.)	Tidak
PayOff [44]	Blockchain	Pembayaran tanpa konektivitas	Sebagian	Tidak
Block-PAD [40]	Blockchain	Identitas digital	Sebagian	Terbatas

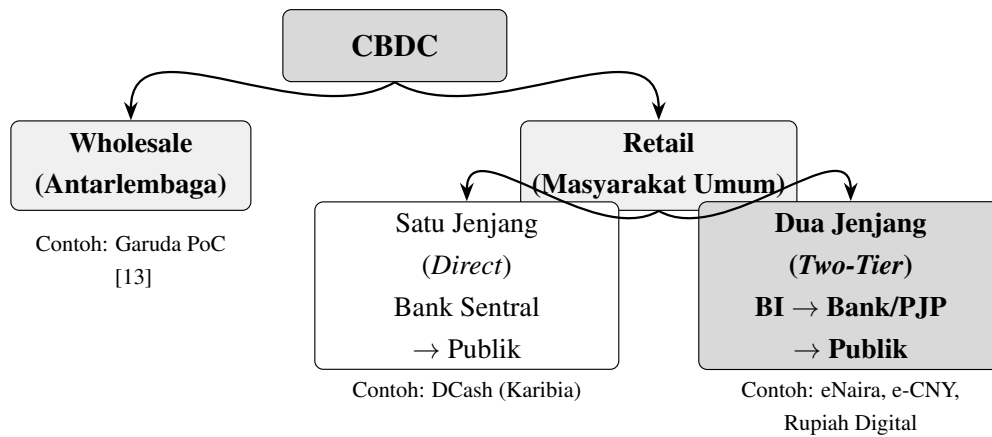
Penelitian	Platform	Fokus utama	Integrasi fitur ritel	Tolok ukur kinerja
Lovejoy dkk. (OpenCBDC) [32]	Account-based (non-DLT)	Kinerja tinggi	Sebagian	Ya (non-blockchain)
eNaira [30]	Hyper ledger Fabric	CBDC ritel terdeploy	Ya	Tidak publik
Hanif [1]	Hyper ledger Fabric	Token (MeetCoin)	Tidak	Caliper
Penelitian ini	Hyper ledger Fabric	CBDC ritel multipihak	Ya	Caliper

2.2 Dasar Teori

2.2.1 Central Bank Digital Currency

Central Bank Digital Currency (CBDC) adalah representasi digital dari mata uang suatu negara yang diterbitkan oleh dan menjadi kewajiban langsung bank sentral [12]. Sebagai kewajiban bank sentral, CBDC memiliki kualitas bebas risiko yang setara dengan uang tunai, berbeda dengan uang elektronik yang merupakan kewajiban penerbit swasta. Secara umum CBDC dibedakan menjadi dua jenis berdasarkan pengguna sarannya, yaitu CBDC *wholesale* yang ditujukan untuk transaksi antarlembaga keuangan, dan CBDC *retail* yang ditujukan untuk masyarakat luas [4, 45].

Dalam penerapannya, sebagian besar bank sentral memilih model dua jenjang (*two-tier*), yaitu bank sentral menerbitkan CBDC dan mengelola *ledger* pada tingkat *wholesale*, sementara distribusi kepada masyarakat dilakukan melalui perantara seperti bank dan penyedia jasa pembayaran. Model ini menjaga peran intermediasi lembaga keuangan dan mengurangi risiko *disintermediation* pada sistem perbankan [16]. Di Indonesia, Bank Indonesia mengembangkan CBDC dengan nama Rupiah Digital melalui Proyek Garuda, yang menempuh pendekatan bertahap mulai dari Rupiah Digital *wholesale* kemudian diperluas menuju Rupiah Digital *retail* [7]. Desain Rupiah Digital menekankan interoperabilitas dengan infrastruktur sistem pembayaran dan setelmen, serta penerapan model dua jenjang dengan pihak perantara [13, 14]. Taksonomi CBDC berdasarkan segmen pengguna dan model distribusi ditunjukkan pada Gambar 2.2.

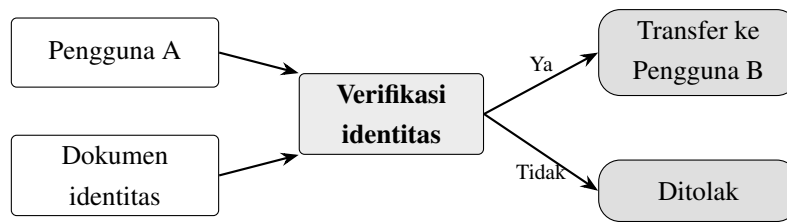


Gambar 2.2. Taksonomi CBDC berdasarkan segmen pengguna dan model distribusi; model dua jenjang yang diadopsi pada penelitian ini ditampilkan dengan latar abu-abu sesuai rancangan Rupiah Digital Bank Indonesia

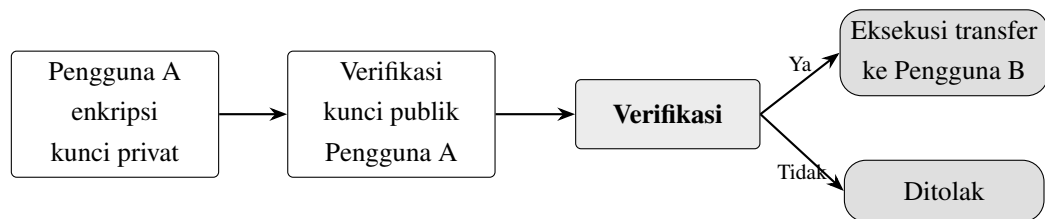
Peta jalan resmi Proyek Garuda menempatkan pengembangan Rupiah Digital dalam tiga fase, yaitu *immediate state* untuk w-Digital Rupiah berbasis *cash ledger*, *intermediate state* untuk w-Digital Rupiah dengan kas dan sekuritas digital, serta *end state* yang mengintegrasikan w-Digital Rupiah dengan r-Digital Rupiah. Pada fase *end state*, use case r-Digital Rupiah mencakup distribusi, penagihan, dan pembayaran P2P, baik melalui perantara maupun langsung dari bank sentral sebagaimana ditunjukkan pada Gambar 2.3 [7].

2.2.1.1 Model Pembayaran Berbasis Akun dan Token

Selain dibedakan berdasarkan segmen pengguna dan model distribusi, CBDC juga dapat diklasifikasikan berdasarkan mekanisme pembayaran berbasis akun (*account-based*) atau berbasis token (*token-based*). Pada model berbasis akun, akses pembayaran bergantung pada hubungan antara identitas pengguna, akun atau dompet, dan catatan saldo yang dikelola sistem. Pada model berbasis token, validitas pembayaran terutama bergantung pada verifikasi objek token atau instrumen nilai yang dipindahkan, sehingga penekanan desain berada pada keaslian token dan hak pemegangnya untuk membelanjakan token tersebut [6, 46]. Perbandingan alur verifikasi kedua model ditunjukkan pada Gambar 2.4.



(a) Berbasis akun (*account-based*)



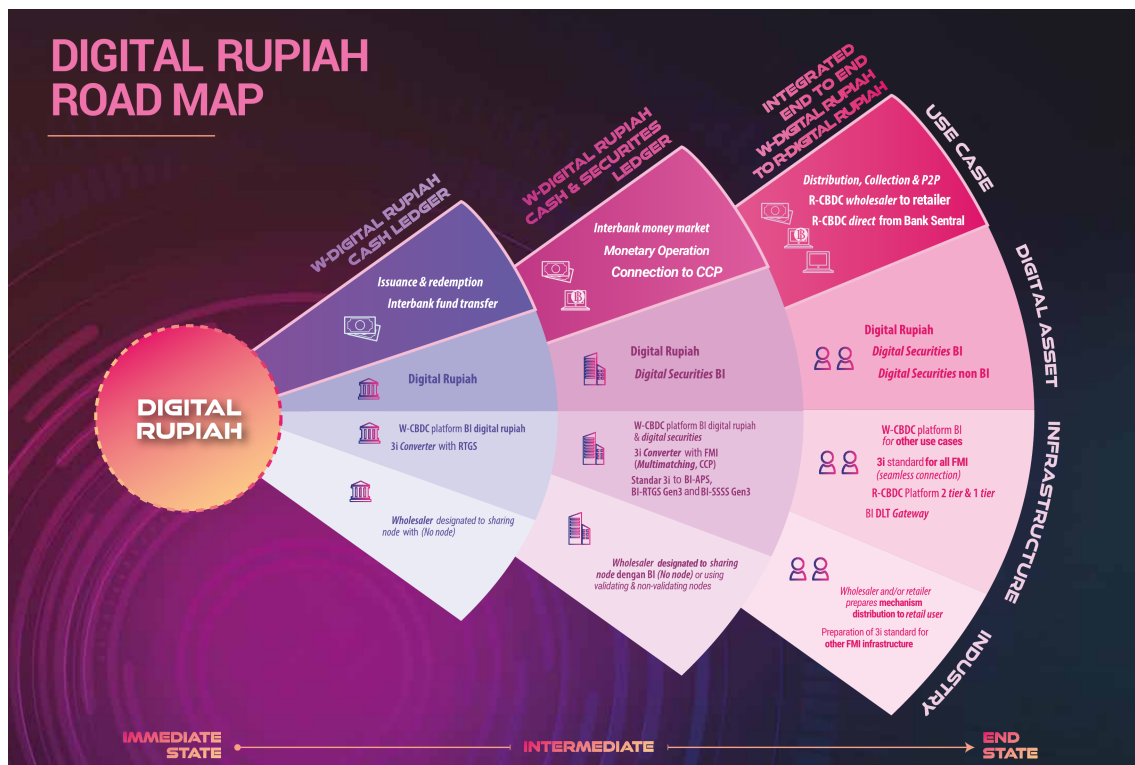
(b) Berbasis token (*token-based*)

Gambar 2.4. Perbandingan model pembayaran CBDC berbasis akun dan berbasis token: model akun memverifikasi identitas pemegang, sedangkan model token memverifikasi keaslian objek nilai. Diadaptasi dari Auer dan Böhme [6]

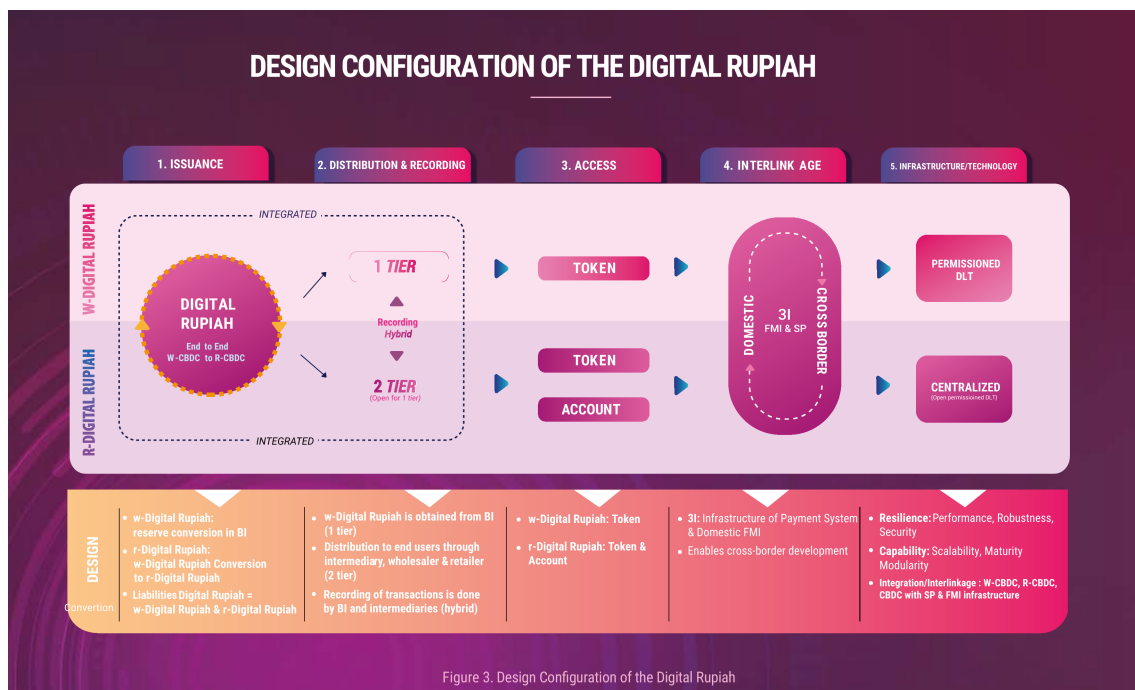
White Paper CBDC Bank Indonesia menempatkan Rupiah Digital dalam konfigurasi desain yang mencakup pemisahan w-Digital Rupiah dan r-Digital Rupiah, pilihan satu jenjang atau dua jenjang, akses berbasis token dan akun, serta infrastruktur DLT *permissioned* sebagaimana ditunjukkan pada Gambar 2.5. Konfigurasi tersebut menjadi dasar pembatasan ruang lingkup penelitian ini: purwarupa difokuskan pada model ritel dua jenjang berbasis dompet akun di atas jaringan *permissioned blockchain*, bukan implementasi penuh seluruh konfigurasi Rupiah Digital [7].

Pembedaan ini relevan karena memengaruhi desain KYC, AML/CFT, privasi, limit saldo, dan auditabilitas transaksi. Syarifuddin dan Bakhtiar menempatkan *account-based* dan *token-based payment system* sebagai salah satu dimensi desain CBDC yang perlu dipertimbangkan dalam strategi kebijakan moneter [4]. Dalam konteks Indonesia, dokumen konsultatif Rupiah Digital menyatakan bahwa akses pada *wholesale Digital Rupiah* menggunakan verifikasi berbasis token karena dianggap sesuai untuk transaksi pasar keuangan yang lebih kompleks [14]. Namun, dokumen tersebut menempatkan Rupiah Digital ritel sebagai bagian dari tahap integrasi *end-state*; karena itu, penelitian ini tidak mengklaim bahwa desain pembayaran Rupiah Digital ritel Bank Indonesia telah final sebagai berbasis akun ataupun berbasis token.

Beberapa implementasi CBDC ritel menggunakan pendekatan dompet bertingkat yang dekat dengan model akun atau hibrida. eNaira menerapkan dompet pengguna dengan tingkat KYC dan batas transaksi [30, 47], sedangkan e-CNY menggunakan



Gambar 2.3. Peta jalan pengembangan Rupiah Digital dalam Proyek Garuda. Sumber: Bank Indonesia [7].

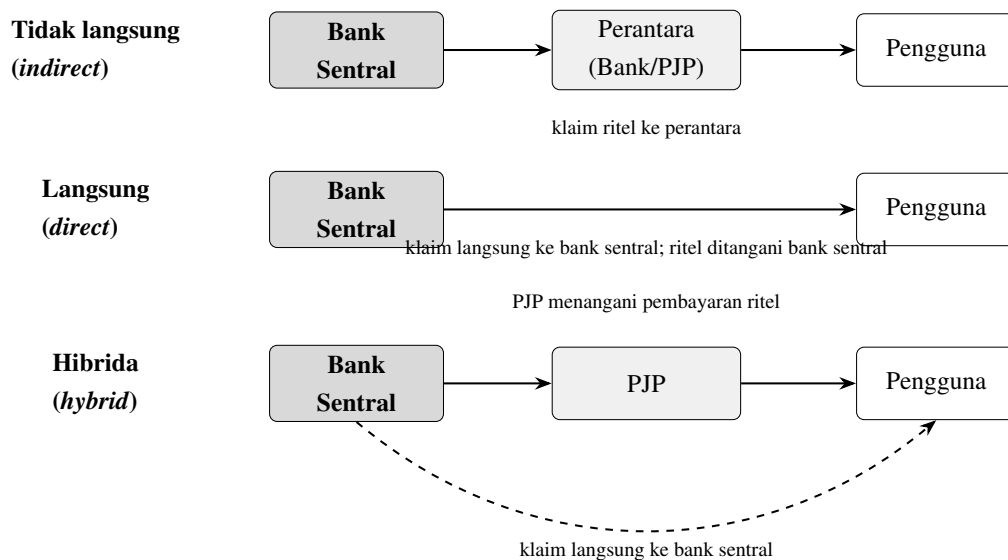


Gambar 2.5. Konfigurasi desain Rupiah Digital menurut White Paper CBDC Bank Indonesia. Sumber: Bank Indonesia [7].

kategori dompet bertingkat dengan kekuatan identifikasi dan batas saldo/transaksi yang berbeda [48]. Preseden tersebut menunjukkan bahwa model dompet berbasis identitas dan limit merupakan pendekatan yang umum digunakan untuk mengendalikan risiko kepatuhan pada CBDC ritel, meskipun arsitektur teknis setiap yurisdiksi tidak identik.

2.2.1.2 Arsitektur Penerbitan CBDC

Selain model distribusi satu atau dua jenjang, penerbitan CBDC dikategorikan menurut struktur klaim menjadi tiga arsitektur, yaitu langsung (*direct*), tidak langsung (*indirect*), dan hibrida (*hybrid*) [6]. Pada arsitektur langsung, bank sentral menangani layanan pembayaran ritel secara penuh; pada arsitektur tidak langsung, klaim ritel berada pada perantara yang menjamin CBDC; sedangkan pada arsitektur hibrida, CBDC tetap menjadi klaim langsung bank sentral tetapi perantara mengelola interaksi dengan pengguna. Syarifuddin dan Bakhtiar menempatkan pilihan arsitektur ini sebagai salah satu dimensi desain CBDC yang memengaruhi strategi kebijakan moneter [4]. Ketiga arsitektur tersebut dibandingkan pada Gambar 2.6.



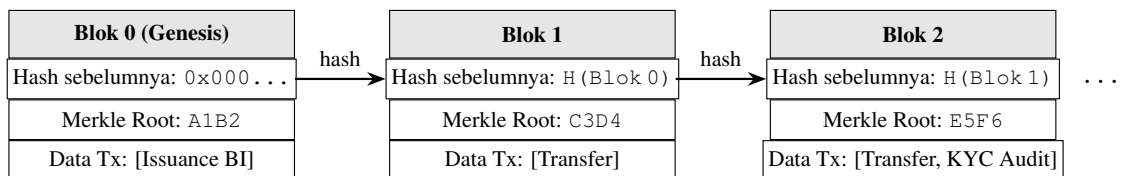
Gambar 2.6. Perbandingan arsitektur penerbitan CBDC: tidak langsung (*indirect*/dua jenjang), langsung (*direct*/satu jenjang), dan hibrida (*hybrid*). Diadaptasi dari Auer dan Böhme [6]

2.2.2 Blockchain dan Distributed Ledger Technology

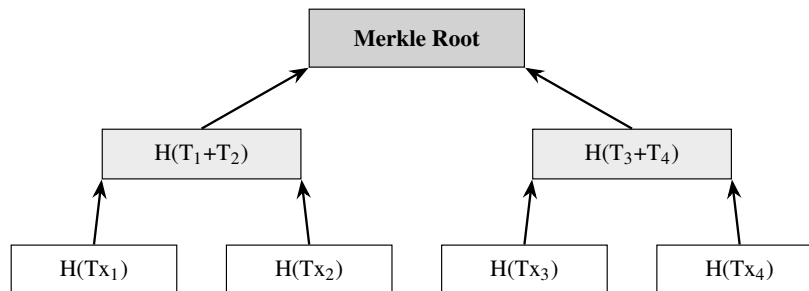
2.2.2.1 Struktur Blockchain

Distributed Ledger Technology (DLT) adalah teknologi pencatatan terdistribusi yang memungkinkan banyak pihak memelihara salinan basis data yang konsisten tanpa otoritas terpusat tunggal. *Blockchain* merupakan salah satu bentuk DLT yang menyusun transaksi ke dalam blok-blok yang saling terhubung secara kriptografis. Setiap blok memuat *hash* dari blok sebelumnya sehingga membentuk rantai yang tidak dapat

diubah; perubahan pada satu blok akan merusak keterkaitan *hash* pada seluruh blok berikutnya [15]. Sifat ini memberikan jaminan integritas dan ketertelusuran data. Struktur rantai blok diilustrasikan pada Gambar 2.7. Setiap blok juga memuat *Merkle root*, yaitu nilai hash yang dihasilkan dari pohon Merkle seluruh transaksi dalam blok tersebut, sehingga integritas setiap transaksi dapat diverifikasi secara efisien tanpa harus membaca semua data transaksi [15]. Struktur pohon Merkle ditunjukkan pada Gambar 2.8.



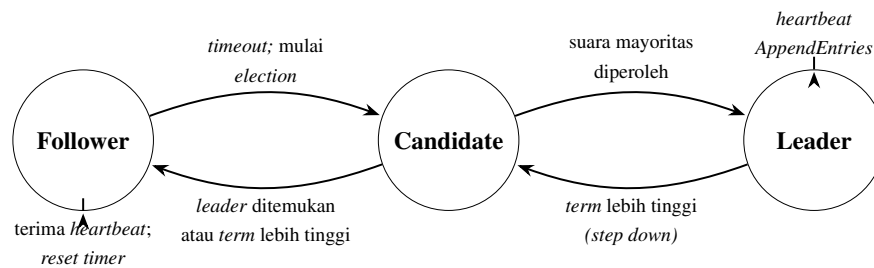
Gambar 2.7. Struktur rantai blok: setiap blok memuat *hash* kriptografis dari blok sebelumnya



Gambar 2.8. Pohon Merkle: *hash* transaksi digabungkan secara berjenjang membentuk Merkle Root yang disimpan dalam kepala blok

2.2.2.2 Algoritme Konsensus

Algoritme konsensus menjamin seluruh simpul pada jaringan menyepakati satu urutan transaksi yang sama. Pada *blockchain* publik digunakan mekanisme seperti *proof-of-work* (PoW) dan *proof-of-stake* (PoS) yang dirancang tahan terhadap perilaku jahat namun berbiaya komputasi tinggi. Sebaliknya, pada *blockchain permissioned* digunakan algoritme yang lebih efisien, baik varian *Byzantine Fault Tolerance* (BFT) maupun *Crash Fault Tolerance* (CFT) seperti Raft, karena seluruh peserta telah teridentifikasi sehingga risiko perilaku *Byzantine* dapat ditekan [18,49]. Algoritme Raft membagi peran simpul menjadi tiga *state*: *Leader*, *Candidate*, dan *Follower*. Diagram transisi *state* Raft ditunjukkan pada Gambar 2.9.



Gambar 2.9. Diagram transisi *state* algoritme konsensus Raft: setiap simpul berstatus *Follower*, *Candidate*, atau *Leader*

2.2.2.3 Tipe Blockchain

Berdasarkan model aksesnya, *blockchain* dapat dikelompokkan menjadi *public* (permissionless), *private*, dan *consortium* atau *permissioned* [17]. Pada *public blockchain*, siapa pun dapat bergabung dan berpartisipasi dalam konsensus, sehingga memerlukan mekanisme seperti *proof-of-work* yang mahal secara komputasi. Sebaliknya, pada *permissioned blockchain*, partisipasi dibatasi pada entitas yang teridentifikasi dan memperoleh izin, sehingga konsensus dapat menggunakan algoritme yang lebih efisien dan menghasilkan *throughput* lebih tinggi. Untuk kebutuhan CBDC, *permissioned blockchain* menjadi pilihan yang sesuai karena bank sentral memerlukan kendali tata kelola, identitas peserta yang jelas, dan kinerja yang memadai [16,43].

2.2.3 Hyperledger Fabric

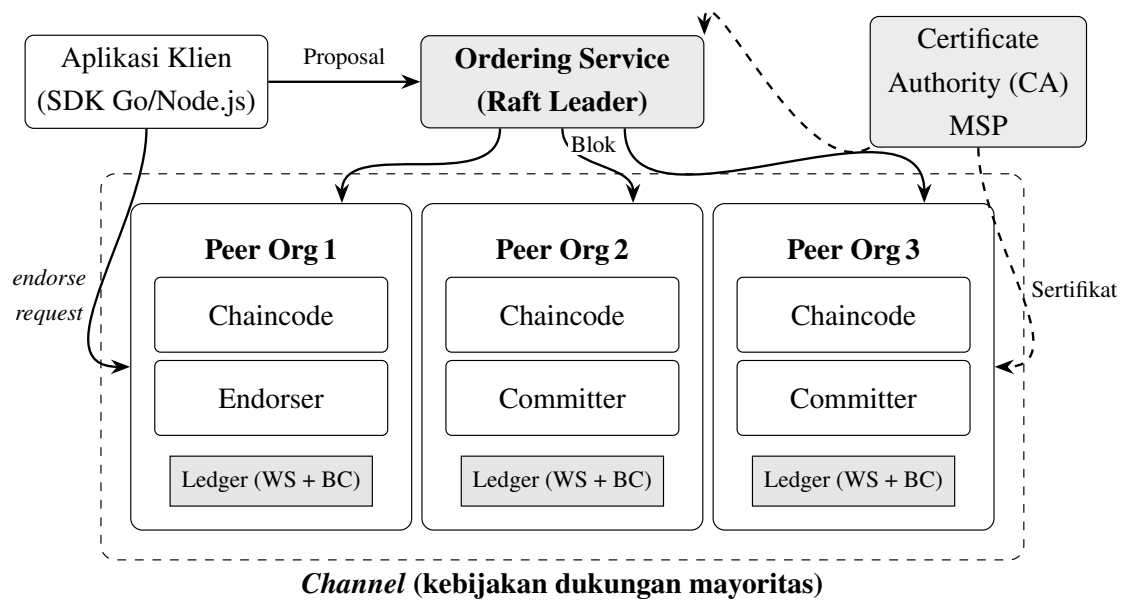
2.2.3.1 Arsitektur Hyperledger Fabric

Hyperledger Fabric adalah kerangka kerja *permissioned blockchain* sumber terbuka di bawah naungan *Hyperledger Foundation* yang dirancang untuk aplikasi perusahaan [18]. Fabric memiliki arsitektur modular dengan komponen utama sebagai berikut.

1. **Peer**, yaitu simpul yang menyimpan salinan *ledger* dan menjalankan *chaincode*. *Peer* berperan sebagai *endorser* yang menyimulasikan transaksi dan sebagai *committer* yang menulis blok ke *ledger*.
2. **Orderer**, yaitu layanan pengurut (*ordering service*) yang menyusun transaksi ke dalam blok dan mendistribusikannya ke *peer*. Fabric mendukung konsensus berbasis Raft (etcdraft) yang bersifat *crash fault tolerant*.
3. **Membership Service Provider (MSP)**, yaitu komponen yang mengelola identitas dan hak akses peserta berdasarkan sertifikat digital X.509 yang diterbitkan oleh *Certificate Authority (CA)*.
4. **Channel**, yaitu mekanisme pemisahan jaringan yang memungkinkan subkelompok peserta bertransaksi secara privat dengan *ledger* tersendiri.

5. **Chaincode**, yaitu program *smart contract* yang memuat logika bisnis dan aturan transaksi. Pada Fabric, *chaincode* dapat ditulis dalam bahasa Go, Java, atau Node.js.

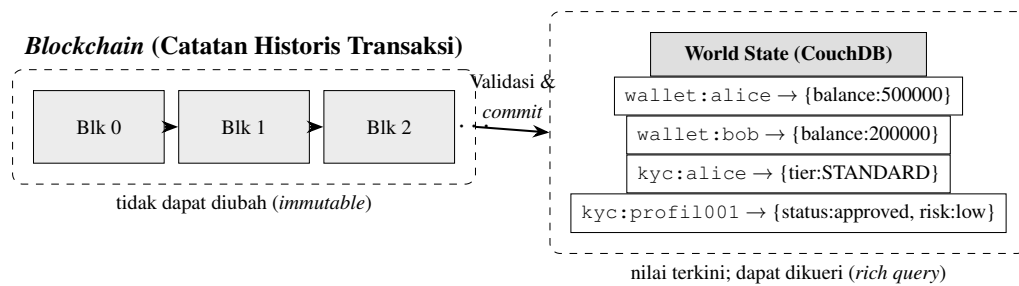
Arsitektur komponen Hyperledger Fabric secara keseluruhan, termasuk hubungan antara aplikasi klien, *endorsing peer*, *ordering service*, dan *channel*, ditunjukkan pada Gambar 2.10.



Gambar 2.10. Arsitektur komponen Hyperledger Fabric: klien berinteraksi dengan *peer endorser*, transaksi diurutkan oleh *orderer* berbasis Raft, lalu didistribusikan ke semua *peer* dalam *channel*

2.2.3.2 Hyperledger Fabric Ledger

Ledger pada Fabric terdiri atas dua bagian, yaitu *world state* yang menyimpan nilai terkini dari setiap kunci (umumnya menggunakan basis data LevelDB atau CouchDB), dan *blockchain* yang menyimpan riwayat transaksi secara berurutan dan tidak dapat diubah. Penggunaan CouchDB memungkinkan penyimpanan dokumen JSON serta kueri kaya (*rich query*) [50]. Dualitas kedua komponen *ledger* tersebut diilustrasikan pada Gambar 2.11: *blockchain* berfungsi sebagai catatan historis yang tidak dapat diubah, sedangkan *world state* menyimpan nilai terkini setiap kunci yang dapat dibaca secara langsung dan efisien oleh *chaincode*.



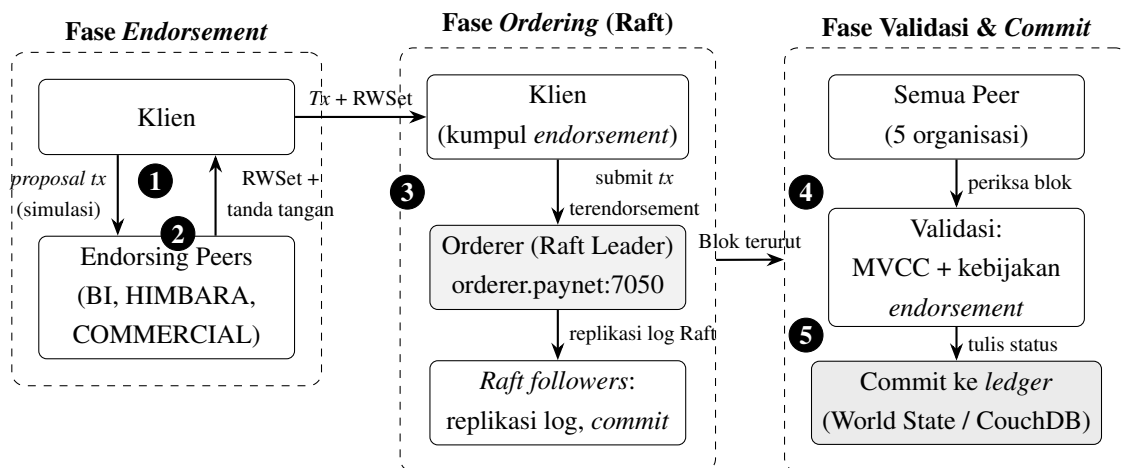
Gambar 2.11. Dualitas komponen *ledger* Hyperledger Fabric: *blockchain* sebagai catatan transaksi historis yang tidak dapat diubah, dan *world state* sebagai representasi status terkini pada CouchDB

2.2.3.3 Alur Transaksi Hyperledger Fabric

Karakteristik utama Fabric adalah model eksekusi *execute-order-validate* yang membedakannya dari model *order-execute* pada banyak *blockchain* lain. Alur transaksi Fabric berlangsung sebagai berikut: (1) aplikasi klien mengirim usulan transaksi (*transaction proposal*) ke sejumlah *peer endorser*; (2) *endorser* menyimulasikan transaksi terhadap *chaincode* dan mengembalikan tanda tangan dukungan (*endorsement*) beserta himpunan baca-tulis (*read-write set*); (3) klien mengumpulkan dukungan sesuai kebijakan dukungan (*endorsement policy*) dan mengirimkannya ke *ordering service*; (4) *orderer* mengurutkan transaksi ke dalam blok; (5) setiap *peer* memvalidasi transaksi (memeriksa kebijakan dukungan dan konflik versi) kemudian menuliskannya ke *ledger* [18]. Model ini meningkatkan skalabilitas karena eksekusi *chaincode* dilakukan secara paralel sebelum pengurutan.

Endorsement policy menentukan organisasi mana yang dukungannya diperlukan agar suatu transaksi dianggap sah. Pada jaringan multipihak, kebijakan seperti *majority* mensyaratkan dukungan dari mayoritas organisasi, sehingga tidak ada satu pihak pun yang dapat memutakhirkan *ledger* secara sepihak.

Gambar 2.12 mengilustrasikan siklus transaksi Hyperledger Fabric secara lengkap, mulai dari *proposal* ke *endorsing peer* hingga *commit* ke *ledger* melalui mekanisme konsensus Raft.



Gambar 2.12. Siklus transaksi Hyperledger Fabric dengan mekanisme konsensus Raft

2.2.4 Hyperledger Caliper

Hyperledger Caliper adalah kerangka kerja *benchmark* kinerja *blockchain* yang memungkinkan pengukuran kinerja terhadap sistem *blockchain* dengan beban kerja yang dapat dikustomisasi [2]. Caliper menghasilkan beban transaksi melalui sejumlah proses pekerja (*worker*) dan mengukur metrik kinerja utama berdasarkan metrik standar *Hyperledger Performance and Scale Working Group* [51], antara lain:

1. **Throughput**, yaitu jumlah transaksi yang berhasil diproses per satuan waktu, dinyatakan dalam *transactions per second* (TPS).
2. **Latency**, yaitu selang waktu antara pengiriman transaksi hingga transaksi tersebut tercatat dan terkonfirmasi pada *ledger*.
3. **Send rate**, yaitu laju pengiriman transaksi yang dikendalikan oleh *rate controller*.

Caliper juga dapat memantau penggunaan sumber daya *container* seperti penggunaan CPU dan memori selama pengujian, sehingga sesuai untuk mengevaluasi kelayakan kinerja sistem CBDC [33].

2.2.5 Kepatuhan: KYC, Tingkatan Dompot, dan Limit Transaksi

Know Your Customer (KYC) adalah proses verifikasi identitas nasabah yang menjadi syarat kepatuhan pada sistem keuangan untuk mencegah pencucian uang dan pendanaan terorisme. FATF menempatkan tingkat keandalan identitas digital dan risiko nasabah sebagai dasar penentuan intensitas uji tuntas, sedangkan risiko lebih tinggi memerlukan *enhanced due diligence* (EDD) [52]. Pendekatan berbasis risiko juga menjadi prinsip dalam ketentuan APU, PPT, dan PPPSPM Bank Indonesia [53].

Penerapan tingkatan dompet pada CBDC memiliki preseden pada e-CNY dan eNaira. Dokumen desain e-CNY menjelaskan dompet bertingkat dengan kekuatan

identifikasi, batas saldo, dan batas pembayaran yang berbeda [48]; rancangan eNaira juga membedakan batas berdasarkan tingkat verifikasi pengguna [47]. Literatur ekonomi menunjukkan bahwa batas kepemilikan dan penggunaan CBDC merupakan instrumen desain untuk menyeimbangkan kegunaan, intermediasi perbankan, dan risiko sistemik [54]. Bukti ini mendukung klasifikasi berbasis risiko, tetapi tidak menentukan angka limit yang universal untuk Rupiah Digital.

Purwarupa memisahkan tiga dimensi yang sering tercampur. *Subject type* membedakan pelanggan ritel, pedagang, dan peserta institusional. *Due diligence level* mencatat uji tuntas *simplified*, *standard*, atau *enhanced*. *Risk level* mencatat risiko rendah, menengah, tinggi, atau terlarang. Kombinasi ketiganya menghasilkan tingkatan BASIC, STANDARD, atau MERCHANT. Risiko tinggi tidak otomatis ditolak, tetapi hanya dapat disetujui setelah EDD dan persetujuan pejabat senior; transaksi selanjutnya menghasilkan peristiwa pengawasan tanpa data identitas mentah.

Angka limit penelitian merupakan parameter kebijakan purwarupa. Ketentuan uang elektronik Bank Indonesia digunakan sebagai proksi domestik, bukan sebagai dasar hukum limit CBDC [55]. Nilai dipilih konservatif dan eksplisit agar aturan dapat diuji: semakin ringan uji tuntas, semakin kecil eksposur saldo dan arus dana. Validasi mencakup saldo maksimum, batas per transaksi, pengeluaran harian, pengeluaran bulanan, serta penerimaan bulanan.

2.2.6 Transfer Ritel Langsung

Unit pembayaran pada purwarupa adalah fungsi *Transfer* yang sama untuk transfer antarpelanggan dan pelanggan-ke-pedagang. Penyatuan jalur ini menjaga aturan KYC, penghitung limit, pemeriksaan saldo penerima, dan pencatatan audit tetap deterministik. Penelitian tidak memodelkan standar instrumen pembayaran atau proses akseptasi eksternal; fokusnya adalah kebijakan buku besar CBDC ritel.

2.2.7 Application Programming Interface

Application Programming Interface (API) adalah antarmuka yang memungkinkan dua perangkat lunak saling berkomunikasi melalui kontrak yang terdefinisi. Sistem yang dibangun menggunakan *REST API* (*Representational State Transfer*) sebagai antarmuka komunikasi menuju lapisan *backend* melalui protokol HTTP. API ini menjembatani permintaan pengguna atau alat uji ke *chaincode* melalui *Fabric Gateway SDK* tanpa menjadikan antarmuka pengguna sebagai bagian kontribusi penelitian.

2.2.8 Bahasa Pemrograman Go

Go adalah bahasa pemrograman terkompilasi yang dikembangkan oleh Google dengan penekanan pada kesederhanaan dan dukungan konkurensi melalui *goroutine*. Pada penelitian ini Go digunakan untuk menulis *chaincode* dan *backend* karena dukungannya yang kuat terhadap Hyperledger Fabric melalui *Fabric Contract API* dan *Fabric Gateway SDK*, serta kinerja konkurensi yang baik untuk melayani banyak permintaan secara bersamaan.

2.3 Analisis Perbandingan Metode

Bagian ini menyajikan perbandingan sistematis terhadap empat keputusan desain utama yang mendasari penelitian ini: pilihan antara DLT dan sistem terpusat, pemilihan platform DLT, pemilihan algoritme konsensus, dan pemilihan model arsitektur CBDC. Perbandingan disajikan dalam bentuk tabel agar perbedaan antar-alternatif terlihat secara eksplisit dan dapat dipertanggungjawabkan secara ilmiah.

2.3.1 Perbandingan Hyperledger Fabric dengan Kerangka Kerja Blockchain Lain

DLT versus Sistem Terpusat. Tabel 2.2 membandingkan tiga pendekatan infrastruktur untuk sistem pembayaran CBDC: sistem terpusat tradisional (non-DLT), *permissionless blockchain* (publik), dan *permissioned blockchain* (DLT berizin). Sistem terpusat menawarkan *throughput* sangat tinggi tetapi bergantung pada otoritas tunggal dan tidak memiliki auditabilitas multipartit secara inheren. *Permissionless blockchain* menyediakan transparansi tetapi *throughput*-nya rendah akibat mekanisme konsensus berbasis *proof-of-work* [16, 43]. DLT *permissioned* menggabungkan auditabilitas multipartit, kendali tata kelola yang dapat dikonfigurasi, dan *throughput* tinggi tanpa konsumsi energi besar [18].

Tabel 2.2. Perbandingan sistem terpusat, *permissionless blockchain*, dan *permissioned blockchain* untuk CBDC

Kriteria	Sistem Terpusat (non-DLT)	Permissionless Blockchain	Permissioned Blockchain
Kekebalan data	Tidak (admin dapat ubah)	Ya	Ya
Auditabilitas multipartit	Terbatas (otoritas tunggal)	Ya (publik)	Ya (terkontrol)
Kendali tata kelola	Penuh terpusat	Tidak ada	Dapat dikonfigurasi
<i>Throughput</i>	>50.000 TPS	7–50 TPS	>3.500 TPS [18]
Konsumsi energi	Rendah	Sangat tinggi (PoW)	Rendah
<i>Smart contract</i>	Opsional	Ya	Ya
Privasi transaksi	Ya	Tidak (publik)	Ya (<i>channel</i> /data privat)
Kepatuhan regulasi	Mudah	Sulit	Mudah
Contoh penerapan CBDC	OpenCBDC [32]	Tidak direkomen dasikan	eNaira, Rupiah Digital [13, 30]

Simpulan: *Permissioned blockchain* dipilih karena menyeimbangkan auditabilitas multipartit yang tidak dapat diperoleh dari sistem terpusat, kendali tata kelola bank sentral yang tidak dimiliki *permissionless blockchain*, dan *throughput* tinggi yang dibutuhkan sistem pembayaran [16,43].

Perbandingan Platform DLT. Tabel 2.3 membandingkan empat platform DLT *permissioned* yang lazim digunakan dalam sistem keuangan dan CBDC: Hyperledger Fabric, R3 Corda, Quorum/Hyperledger Besu, dan Hyperledger Sawtooth [16, 18, 56].

Tabel 2.3. Perbandingan platform DLT *permissioned* untuk sistem CBDC

Kriteria	Hyperledger Fabric	R3 Corda	Quorum/ Besu	H. Sawtooth
Konsensus	Raft (CFT), <i>pluggable</i>	Notary (CFT)	IBFT/Clique	PoET, Raft
Model eksekusi	<i>Execute-order-validate</i>	<i>Flow-based</i>	<i>Order-execute</i>	<i>Order-execute</i>
Bahasa <i>smart contract</i>	Go, Java, Node.js	Kotlin, Java	Solidity (EVM)	Python, Go, Java
Manajemen identitas	MSP + X.509 CA	X.509 PKI	Berbasis akun	Berbasis peran
Privasi	<i>Channel, private data</i>	<i>Need-to-know</i>	TX privat	Terbatas
<i>Throughput</i>	>3.500 TPS [18]	~170 TPS [56]	~300 TPS	~1.000 TPS
Penerapan CBDC	eNaira, Proyek Garuda [13, 30]	Pilot BIS (sebagian)	Tidak publik	Tidak

Simpulan: Hyperledger Fabric dipilih karena: (1) MSP berbasis X.509 mendukung KYC berjenjang dan identitas terverifikasi; (2) *channel* memisahkan lapisan *wholesale* dan *retail* secara kriptografis; (3) *throughput* tertinggi di antara platform yang dibandingkan; dan (4) terdapat bukti penerapan CBDC nyata pada eNaira [30] serta Proyek Garuda Bank Indonesia [13].

Sebagai pembanding eksternal, Urbančok membandingkan kinerja transfer aset pada Ethereum, Hyperledger Fabric, dan Besu menggunakan skenario *send rate* 10, 20, 50, dan 100 TPS [8]. Hasil pada Gambar 2.13 menunjukkan bahwa Fabric kompetitif untuk beban transfer aset dan unggul pada beberapa titik uji, tetapi angka tersebut tidak digunakan sebagai hasil evaluasi purwarupa penelitian ini karena konfigurasi jaringan, versi perangkat lunak, dan beban kerja berbeda.

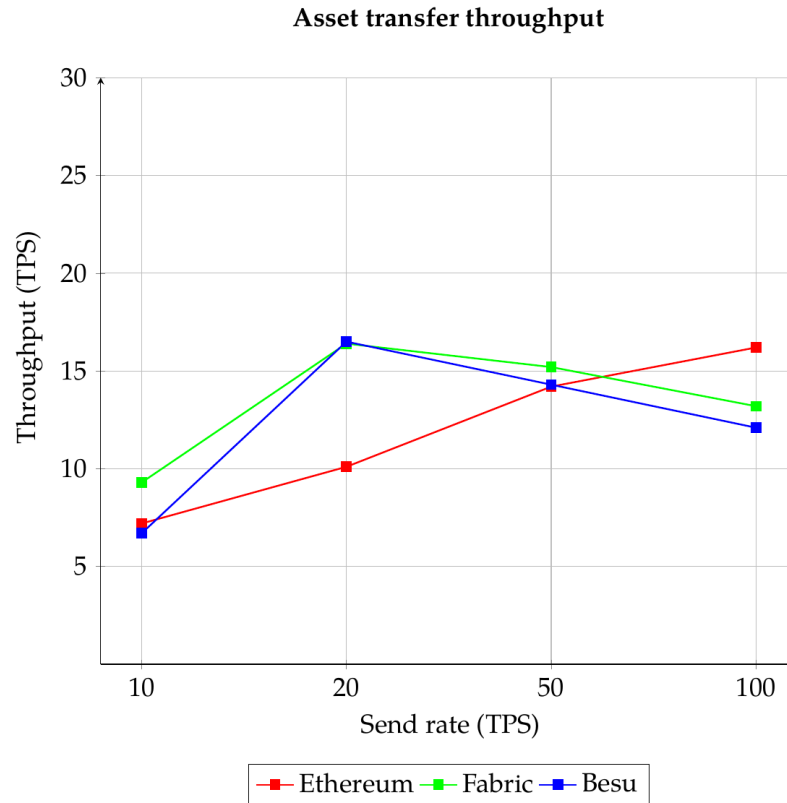


Figure 8.6: Throughput comparison of asset transfer

Gambar 2.13. Perbandingan *throughput* transfer aset pada Ethereum, Hyperledger Fabric, dan Besu. Sumber: [8].

2.3.2 Pemilihan Algoritme Konsensus Hyperledger Fabric

Tabel 2.4 membandingkan algoritme konsensus yang relevan untuk jaringan *blockchain permissioned*. Pada jaringan *permissioned*, peserta telah teridentifikasi sehingga risiko perilaku *Byzantine* lebih rendah dibandingkan jaringan publik; namun finalisasi deterministik tetap diperlukan karena sistem keuangan tidak dapat mentoleransi ketidakpastian *fork* [18,49].

Tabel 2.4. Perbandingan algoritme konsensus untuk *blockchain permissioned*

Algoritme	Toleransi kesalahan	Kompleksitas pesan	Throughput	Latensi	Finalisasi
Raft	CFT	$O(n)$	Tinggi	Rendah	Deterministik
PBFT	BFT	$O(n^2)$	Menengah	Menengah	Deterministik
HotStuff	BFT	$O(n)$	Menengah	Rendah	Deterministik
PoW	—	$O(n)$	Sangat rendah	Sangat tinggi	Probabilistik
PoS/PoA	BFT/CFT	$O(n)$	Menengah	Menengah	Semi-deterministik

Simpulan: Raft (*etcdraft*) dipilih karena: (1) pada jaringan *permissioned* dengan peserta teridentifikasi, toleransi *crash fault* (CFT) sudah mencukupi, karena risiko *Byzantine* diminimalkan oleh lapisan identitas MSP; (2) kompleksitas $O(n)$ menghasilkan *throughput* lebih tinggi dibandingkan PBFT yang terdegradasi akibat $O(n^2)$ [49]; (3) Raft didukung secara *native* pada Hyperledger Fabric melalui *etcdraft orderer* [18]; dan (4) finalisasi deterministik menghilangkan risiko *fork* pada sistem pembayaran.

2.3.3 Pemilihan Alat Pengujian Blockchain

Hyperledger Caliper dan BlockBench sama-sama dapat mengevaluasi jaringan *blockchain permissioned*, tetapi memiliki cakupan berbeda. Caliper menjalankan beban kerja yang dikonfigurasi terhadap kontrak aktual dan melaporkan metrik tingkat sistem, sedangkan BlockBench dirancang untuk membandingkan kinerja keseluruhan dan komponen internal beberapa platform [2, 3]. Perbandingan ringkas disajikan pada Tabel 2.5.

Tabel 2.5. Perbandingan Hyperledger Caliper dan BlockBench. Diadaptasi dari Hanif [1] dan sumber primer [2, 3].

Kriteria	Hyperledger Caliper	BlockBench
Fokus	Pengujian jaringan dengan beban kerja dan kontrak yang dapat dikonfigurasi	Perbandingan lintas platform pada tingkat makro dan mikro
Metrik utama	Tingkat keberhasilan, <i>throughput</i> , latensi, dan penggunaan sumber daya	<i>Throughput</i> , latensi, skalabilitas, dan toleransi kegagalan
Beban kerja	Modul beban kerja yang memanggil fungsi kontrak aktual	Kontrak pintar nyata dan sintetis melalui API benchmark
Kesesuaian	Integrasi langsung dengan jaringan Fabric dan <i>chaincode</i> penelitian	Tepat untuk analisis komponen dan perbandingan beberapa platform

Simpulan: Hyperledger Caliper dipilih karena tujuan penelitian adalah mengukur perilaku *chaincode* CBDC ritel aktual pada satu jaringan Fabric, bukan membandingkan implementasi internal beberapa platform.

2.3.4 Pemilihan Arsitektur CBDC Dua Jenjang

Istilah “dua jenjang” pada bagian ini menyatakan pembagian peran distribusi antara bank sentral dan perantara. Istilah tersebut berbeda dari “tingkatan dompet KYC”, yang menyatakan klasifikasi kebijakan pengguna ritel. Peserta institusional tidak menerima tingkatan BASIC, STANDARD, atau MERCHANT.

Tabel 2.6 membandingkan model arsitektur satu jenjang (*one-tier/direct*), di mana bank sentral mendistribusikan CBDC langsung kepada publik, dengan model dua jenjang (*two-tier*), di mana distribusi dilakukan melalui bank dan penyedia jasa pembayaran (PJP) [6, 16, 22].

Tabel 2.6. Perbandingan arsitektur CBDC satu jenjang dan dua jenjang

Kriteria	Satu Jenjang (<i>Direct</i>)	Dua Jenjang (<i>Two-Tier</i>)
Distribusi CBDC	Bank sentral langsung ke publik	Bank sentral → bank/PJP → publik
Risiko <i>disintermediation</i>	Tinggi	Rendah [16]
Beban infrastruktur bank sentral	Sangat tinggi	Dibagi ke perantara
KYC dan <i>onboarding</i>	Dikelola bank sentral	Dikelola bank/PJP
Privasi pengguna dari bank sentral	Rendah (semua transaksi <i>retail</i> terlihat)	Lebih baik (data <i>retail</i> di perantara)
Interoperabilitas ekosistem	Sulit	Mudah (bank/PJP sudah terintegrasi)
Contoh CBDC nyata	DCash (Karibia)	eNaira, e-CNY, Sand Dollar [30]
Keselarasan Proyek Garuda	Tidak	Ya [7, 13]

Simpulan: Model dua jenjang dipilih karena: (1) selaras dengan desain Rupiah Digital Bank Indonesia dalam Proyek Garuda [13]; (2) meminimalkan risiko *disintermediation* perbankan yang menjadi kekhawatiran utama bank sentral [16]; (3) KYC dan *onboarding* nasabah dikelola oleh bank dan PJP yang telah berpengalaman; serta (4) beban *ledger wholesale* tidak terbebani volume transaksi *retail* skala masif [6, 22].

2.3.5 Pemetaan Purwarupa terhadap Dimensi Desain CBDC Optimal

Syarifuddin dan Bakhtiar merumuskan desain CBDC optimal pendukung strategi kebijakan moneter ke dalam beberapa dimensi: cakupan ritel dan *wholesale*; remunerasi bunga (*wholesale*) dan tanpa bunga (ritel); model pembayaran berbasis akun dan token; derajat anonimitas yang terlacak; arsitektur hibrida; sistem buku besar berbasis DLT; serta cakupan domestik dan lintas batas [4, 29]. Karena penelitian ini berfokus pada perancangan, implementasi, dan evaluasi kinerja fungsional, dimensi yang bersifat makroekonomi dan kebijakan moneter berada di luar cakupan; Tabel 2.7 memetakan pemenuhan tiap dimensi pada purwarupa beserta justifikasinya.

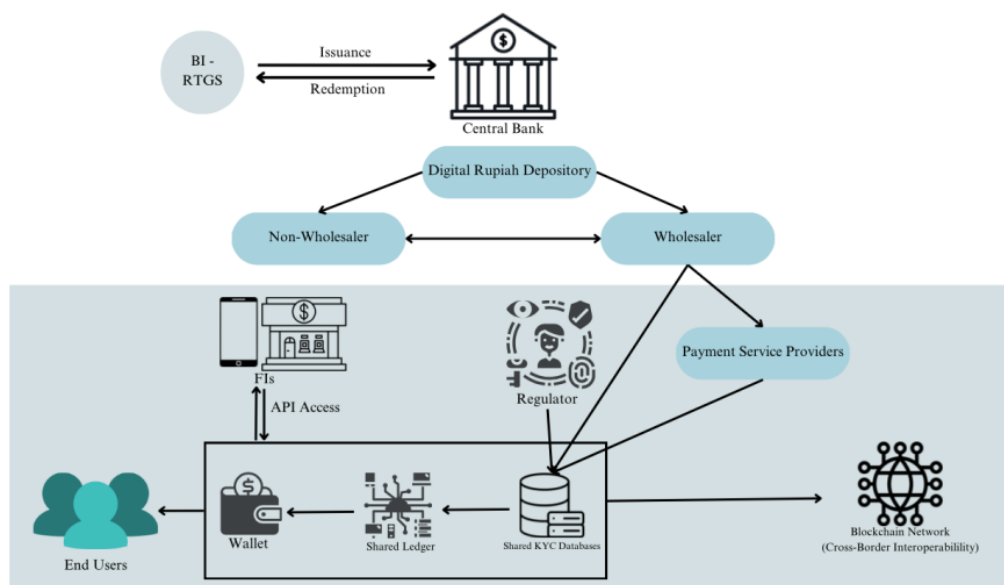
Tabel 2.7. Pemetaan purwarupa terhadap dimensi desain CBDC optimal [4]

Dimensi desain optimal	Status purwarupa	Keterangan
Sistem buku besar DLT	Terpenuhi	Hyperledger Fabric 2.5 sebagai buku besar terdistribusi <i>permissioned</i>
Arsitektur hibrida	Terpenuhi	Distribusi dua jenjang: bank sentral, bank/PJP perantara, lalu pengguna
Cakupan ritel dan <i>wholesale</i>	Terpenuhi	Fokus ritel; lapisan <i>wholesale</i> dimodelkan melalui dompet institusional dan distribusi antarpeserta
Model akun dan token	Sebagian	Implementasi berbasis akun (saldo); dimensi token dibahas namun tidak diklaim final (lihat Subbab Model Pembayaran Berbasis Akun dan Token)
Remunerasi ritel tanpa bunga	Konsisten	Rupiah Digital ritel purwarupa bersifat non-remunerasi, selaras dengan rekomendasi CBDC ritel optimal [4]
Remunerasi <i>wholesale</i> bunga / <i>CBDC rate</i>	Di luar cakupan	Instrumen kebijakan moneter; tidak dimodelkan pada purwarupa fungsional
Anonimitas terlacak	Di luar cakupan	Privasi transaksi tidak diimplementasikan; lihat batasan privasi pada Bab I
Cakupan domestik dan lintas batas	Sebagian	Domestik; interkoneksi CBDC lintas batas merupakan kesenjangan prinsip 3i

Simpulan: pada dimensi arsitektur (DLT, hibrida, dua jenjang, cakupan ritel–*wholesale*, serta remunerasi ritel tanpa bunga), purwarupa konsisten dengan desain CBDC

optimal [4, 29]. Dimensi *CBDC rate*, anonimitas terlacak, dan interkoneksi lintas batas dikeluarkan secara sengaja karena bersifat kebijakan moneter atau berada di luar lingkup evaluasi kinerja fungsional.

Aryani [9] mengusulkan arsitektur dua jenjang Digital Rupiah dengan lapisan tambahan berupa *Digital Rupiah Depository* sebagai entitas perantara antara Bank Indonesia dan peserta jaringan, pemisahan *wholesaler* dan *non-wholesaler* di jenjang atas, serta basis data KYC bersama (*shared KYC database*) yang terpisah dari *ledger* transaksi. Arsitektur tersebut juga mencakup koneksi ke BI-RTGS dan jaringan *blockchain* lintas batas untuk interoperabilitas. Arsitektur yang diusulkan Aryani ditunjukkan pada Gambar 2.14.



Gambar 2.14. Arsitektur Rupiah Digital yang diusulkan Aryani [9]

Penelitian ini mengadopsi prinsip inti arsitektur tersebut (distribusi berjenjang melalui bank validator dan PJP, serta KYC berbasis risiko) namun dengan tiga penyederhanaan yang disengaja sesuai cakupan purwarupa. Pertama, lapisan *Digital Rupiah Depository* dan pemisahan *wholesaler/non-wholesaler* diciutkan menjadi dua organisasi validator (HimbaraBankOrg dan CommercialBankOrg) yang langsung menerima penerbitan dari BankIndonesiaOrg. Kedua, data identitas KYC mentah disimpan pada PostgreSQL di luar buku besar, sedangkan *world state* hanya menyimpan status, risiko, tingkat uji tuntas, persetujuan senior, masa berlaku, dan hash dokumen. Pemisahan ini sejalan dengan pola arsitektur hibrid yang dijabarkan BIS, yakni bank sentral mempertahankan salinan jangkar kepatuhan sementara data sensitif dikelola di luar *ledger* oleh perantara yang menjalankan KYC [6]. Pendekatan serupa, yakni penyimpanan identitas KYC pada basis data terpisah dari buku besar, diadopsi pada proyek CBDC ritel Sand Dollar (Bahama) [57] dan uji coba e-Krona (Swedia) [58].

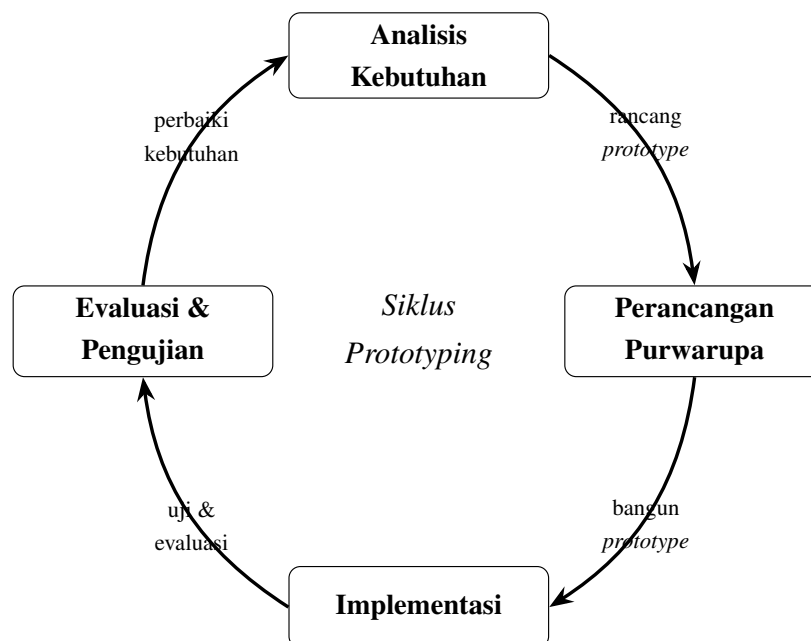
Bank Indonesia dan pengawas memperoleh akses baca terhadap jangkar kepatuhan dan peristiwa pengawasan, bukan kewenangan menjalankan KYC nasabah. Ketiga, integrasi BI-RTGS dan jaringan *blockchain* lintas batas tidak diimplementasikan dan dinyatakan secara eksplisit sebagai batasan cakupan penelitian (Bab I).

BAB III

METODE PENELITIAN

3.1 Metode yang Digunakan

Metode yang digunakan adalah *prototyping* dengan tahapan perancangan, implementasi, dan pengujian. Sistem dibangun sebagai purwarupa fungsional yang kemudian dievaluasi kinerjanya secara kuantitatif menggunakan Hyperledger Caliper. Siklus SDLC metode *prototyping* yang diterapkan ditunjukkan pada Gambar 3.15: keempat fase (analisis kebutuhan, perancangan, implementasi, dan evaluasi) diulang secara iteratif hingga purwarupa memenuhi spesifikasi.



Gambar 3.15. Siklus SDLC dengan metode *prototyping*: empat fase berulang hingga purwarupa memenuhi kebutuhan fungsional dan kinerja

3.1.1 Metode Perancangan

Perancangan sistem dimulai dari analisis kebutuhan berdasarkan model Rupiah Digital dua jenjang, kemudian dituangkan ke dalam activity diagram, arsitektur sistem, arsitektur jaringan, perancangan use case, dan model data *ledger*.

3.1.1.1 Analisis Kebutuhan Sistem

Analisis kebutuhan mengidentifikasi aktor sistem (Bank Indonesia, bank validator, PJP, pedagang, pelanggan ritel, supervisor, dan pengamat OJK) beserta kebutuhan fungsionalnya, yaitu penerbitan dan distribusi likuiditas berjenjang, pengelolaan KYC berbasis risiko, penurunan tingkatan dan limit dompet, transfer

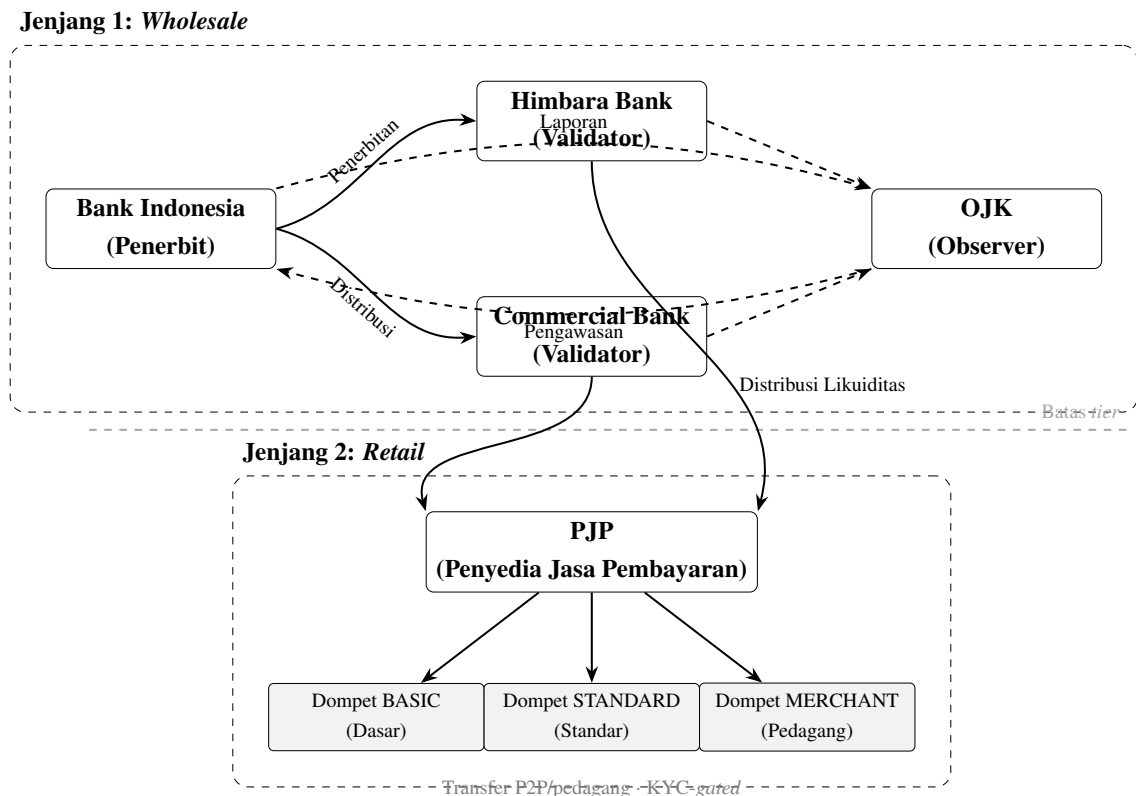
antarpelanggan serta pelanggan-ke-pedagang, pengawasan, dan pelaporan. Kebutuhan ini menjadi dasar perancangan activity diagram dan arsitektur pada subbab berikutnya.

3.1.1.2 Perancangan Activity Diagram

Alur setiap proses utama dimodelkan dengan activity diagram bergaya *swimlane* tiga jalur, yaitu jalur aktor, jalur *backend API*, dan jalur *blockchain (chaincode)*. Proses utama yaitu pendaftaran peserta (Gambar 3.16), penerbitan Digital Rupiah (Gambar 3.17), distribusi likuiditas ke PJP (Gambar 3.18), onboarding KYC pelanggan (Gambar 3.19), transfer saldo ritel (Gambar 3.20), pembekuan dan pencairan peserta (Gambar 3.21), serta laporan dan pengawasan (Gambar 3.22).

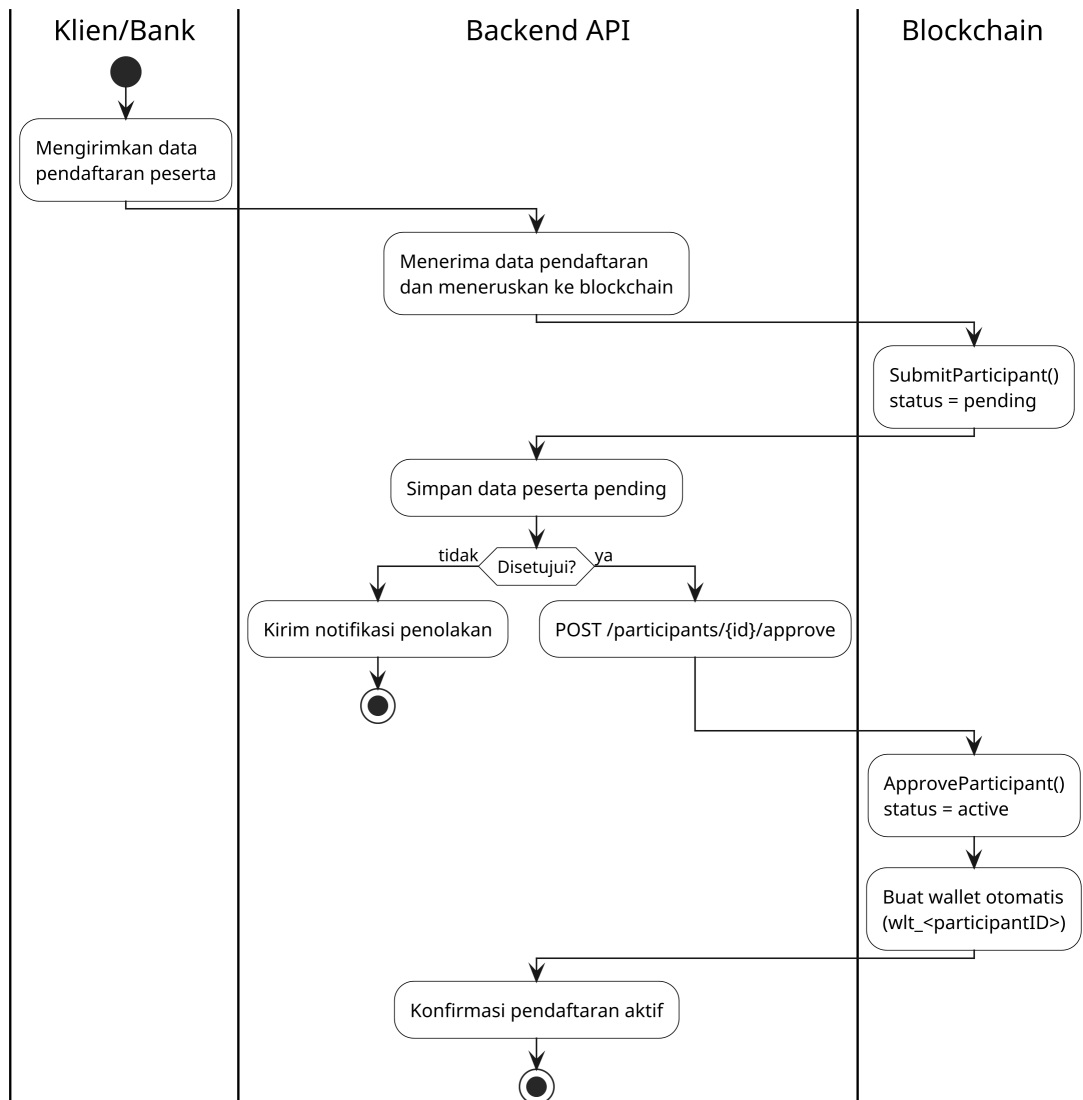
3.1.1.3 Perancangan Arsitektur Sistem Blockchain

Secara konseptual, sistem mengimplementasikan model Rupiah Digital dua jenjang sebagaimana ditunjukkan pada Gambar 3.23. Jenjang pertama (*wholesale*) menghubungkan Bank Indonesia selaku penerbit dengan bank validator (HimbaraBank dan CommercialBank) serta PJP melalui jaringan Hyperledger Fabric. Jenjang kedua (*retail*) menyediakan layanan pembayaran kepada pelanggan melalui PJP, termasuk transfer antarpelanggan, transfer pelanggan-ke-pedagang, dan kontrol kepatuhan berbasis KYC.

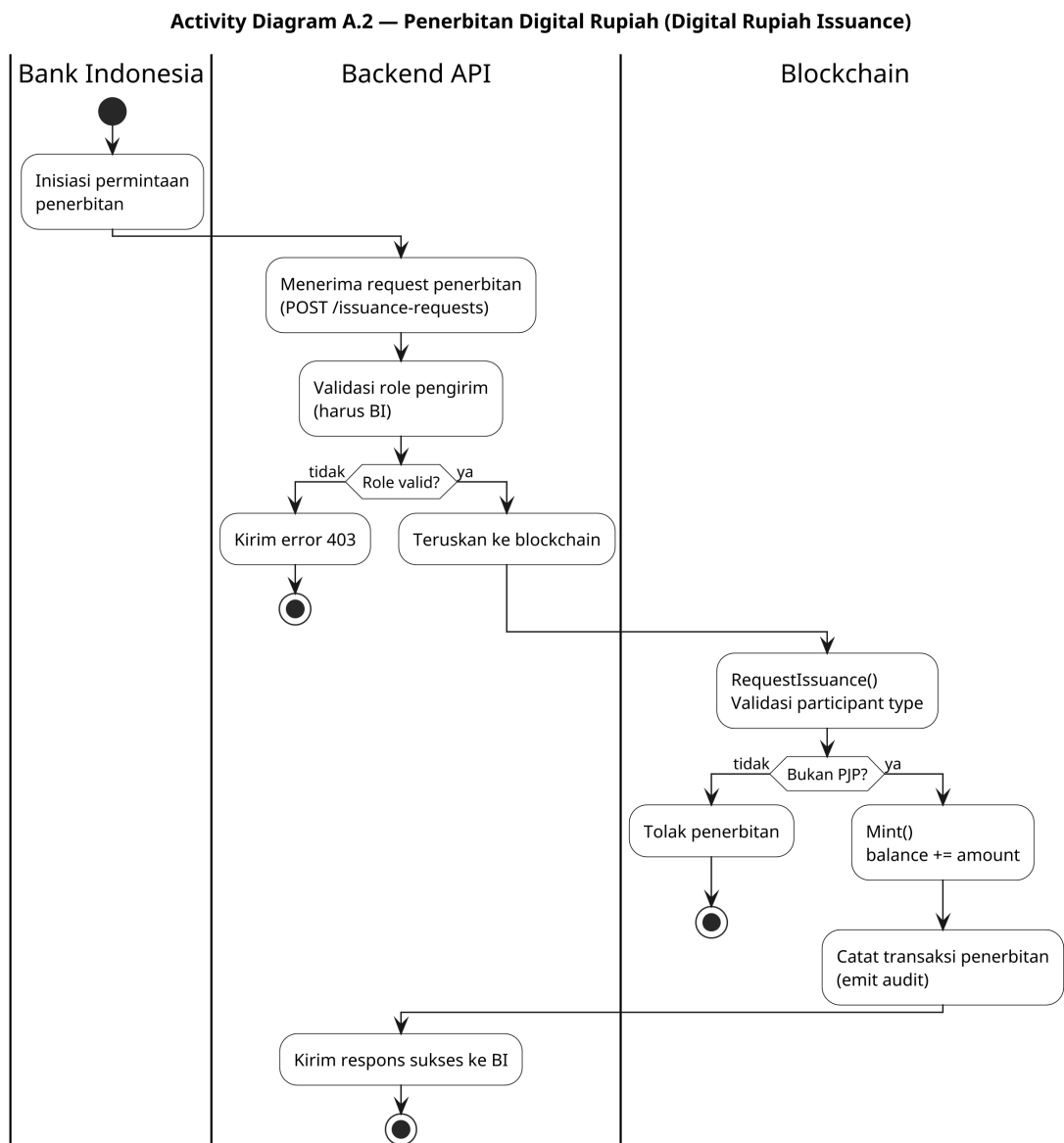


Gambar 3.23. Arsitektur dua jenjang sistem Rupiah Digital

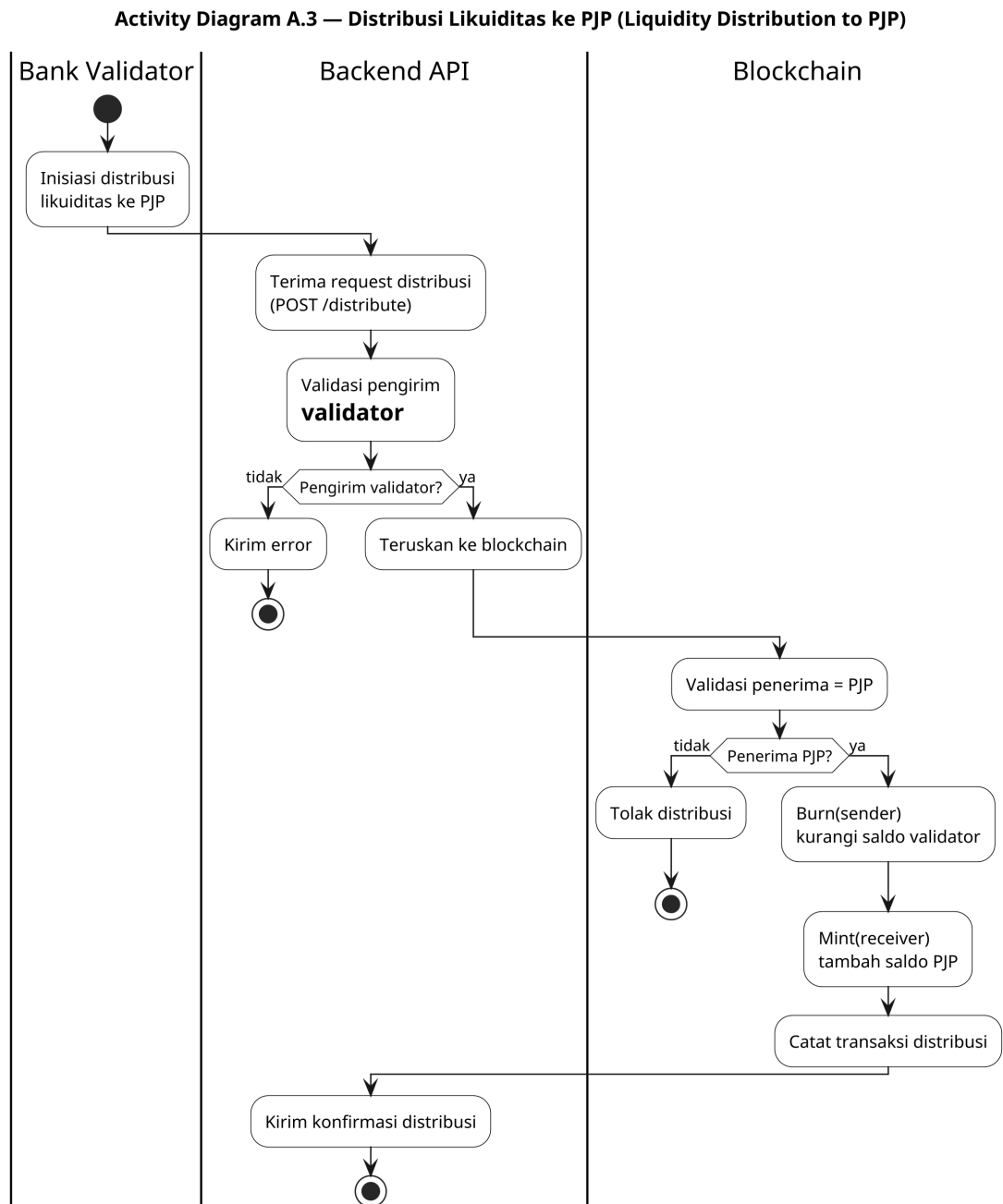
Activity Diagram A.1 — Pendaftaran Peserta (Participant Registration)



Gambar 3.16. Activity diagram pendaftaran peserta

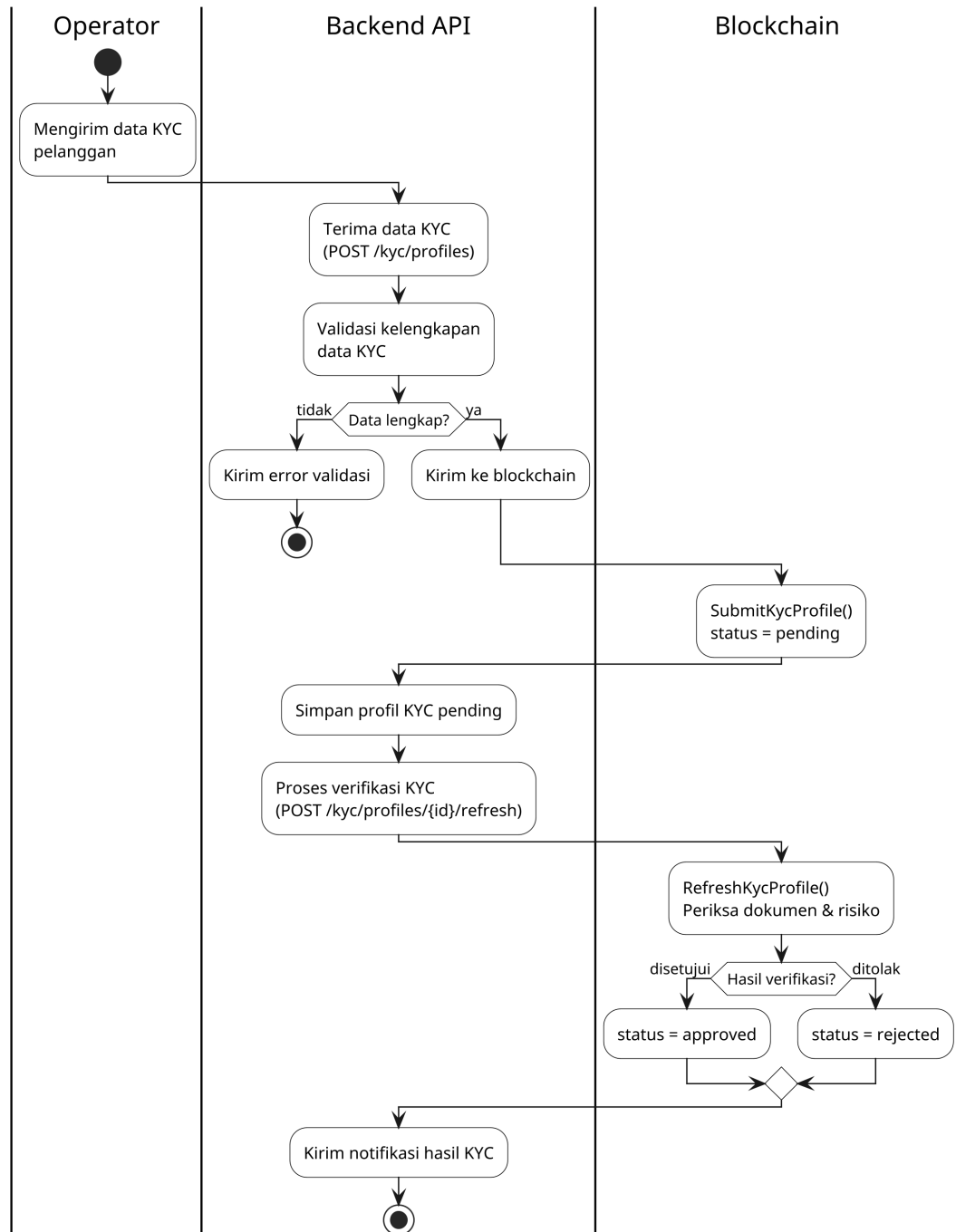


Gambar 3.17. Activity diagram penerbitan Digital Rupiah

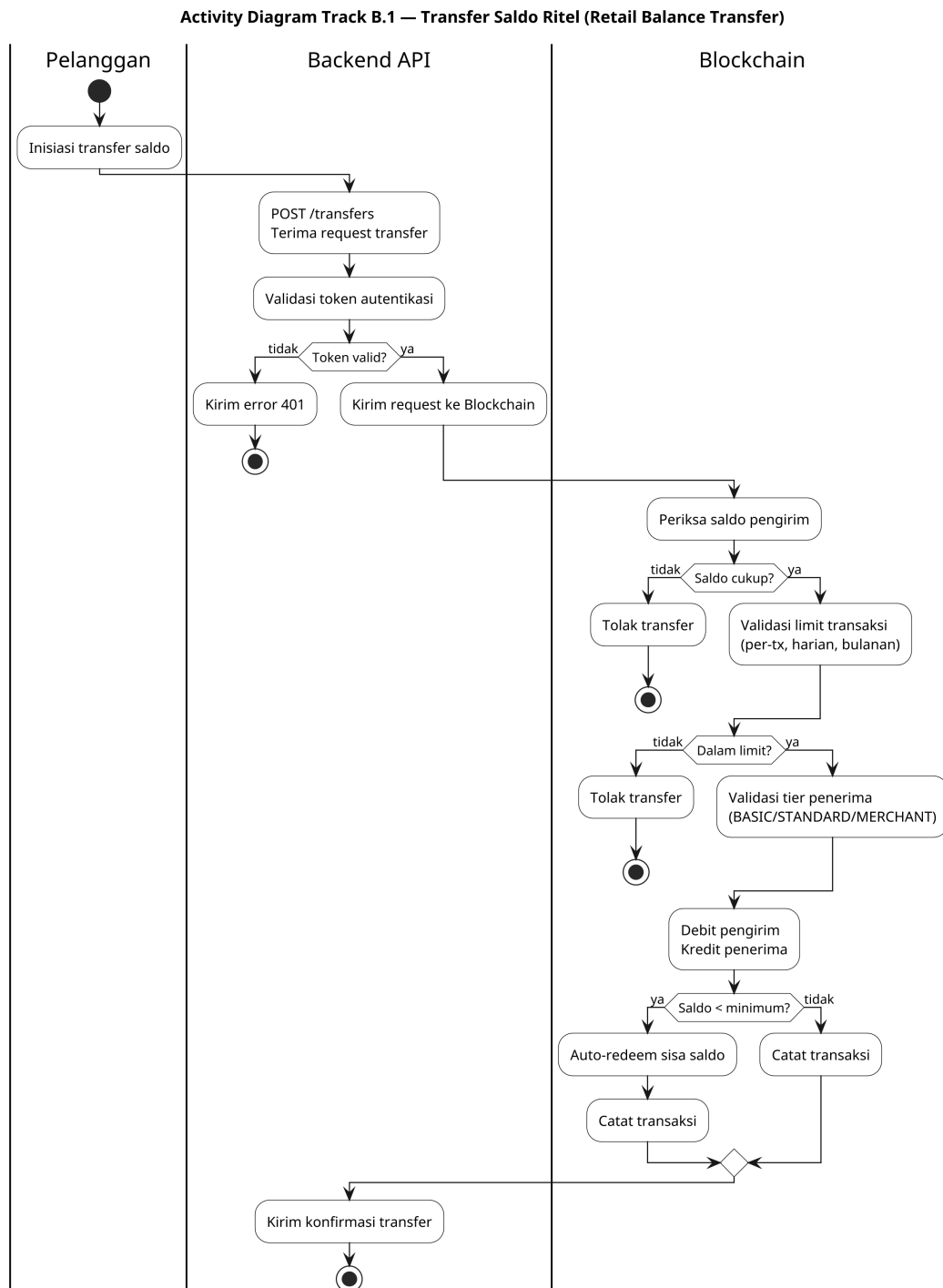


Gambar 3.18. Activity diagram distribusi likuiditas ke PJP

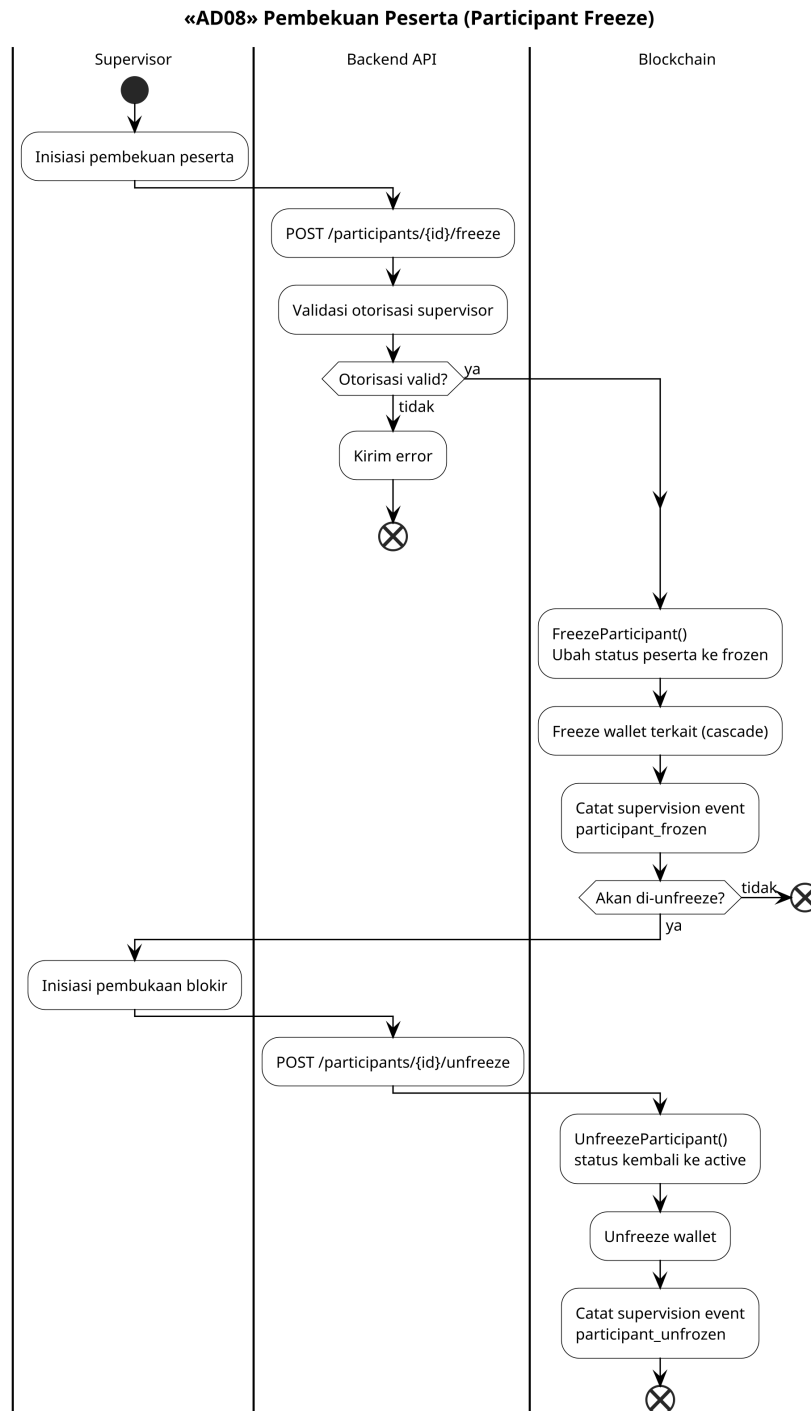
Activity Diagram A.4 — Onboarding KYC Pelanggan (Customer KYC Onboarding)



Gambar 3.19. Activity diagram onboarding KYC pelanggan



Gambar 3.20. Activity diagram transfer saldo ritel

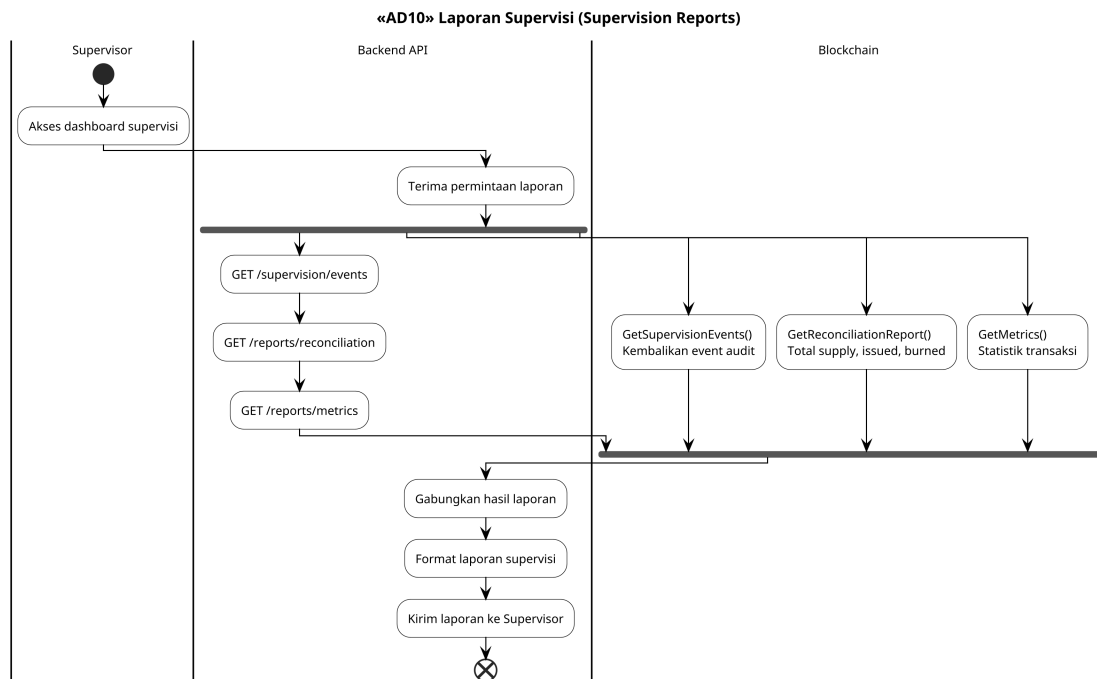


Gambar 3.21. Activity diagram pembekuan dan pencairan peserta

Arsitektur teknologi Rupiah Digital dalam White Paper CBDC Bank Indonesia terdiri atas lapisan platform teknologi, lapisan aset digital, dan lapisan use case. Komponen dasarnya mencakup *sandbox*, API, *smart contract*, layanan identitas, layanan regulasi, dan pengamanan kriptografi sebagaimana ditunjukkan pada Gambar 3.24 [7]. Purwarupa *bi-coin-fabric* memetakan gagasan tersebut secara terbatas melalui API backend, *chaincode*, identitas dan KYC, peran pengawasan, serta *ledger* Hyperledger Fabric. Dengan demikian, purwarupa ini digunakan sebagai model eksperimental yang selaras dengan prinsip arsitektur BI, tetapi tidak merepresentasikan arsitektur produksi Rupiah Digital secara penuh.

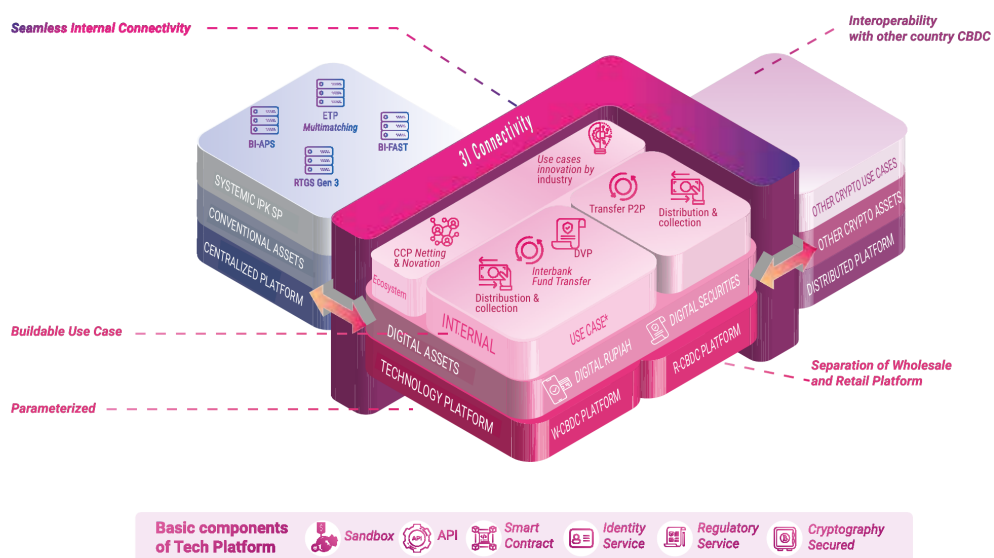
Sistem dirancang sebagai arsitektur layanan *backend API* dan jaringan *blockchain* sebagaimana ditunjukkan pada Gambar 3.25. Lapisan *backend API* bertindak sebagai *gateway* yang menerjemahkan permintaan HTTP menjadi pemanggilan *chaincode* sekaligus menegakkan kontrol akses berbasis peran, sedangkan lapisan jaringan *blockchain* menjalankan *chaincode* serta menyimpan *ledger*.

Berdasarkan klasifikasi model pembayaran pada Bab 2, purwarupa *bi-coin-fabric* dikategorikan sebagai model dompet ritel berbasis akun (*account-based wallet model*) yang diimplementasikan di atas *ledger* Hyperledger Fabric. Klasifikasi ini diturunkan dari rancangan data *on-chain*: objek *Wallet* menyimpan *OwnerID*, *Tier*, dan *Balance*; fungsi *Transfer* memeriksa status KYC, limit tingkatan, status beku, dan saldo, kemudian mendebit saldo pengirim serta mengkredit saldo penerima. Sistem tidak membentuk objek token pembawa (*bearer token*), *UTXO*, atau nomor seri token yang berpindah antarpemilik. Dengan demikian, mekanisme pembayaran purwarupa bukan model token murni, melainkan penyederhanaan dompet berbasis akun untuk menguji KYC bertingkat, limit saldo/transaksi, dan auditabilitas transfer ritel.

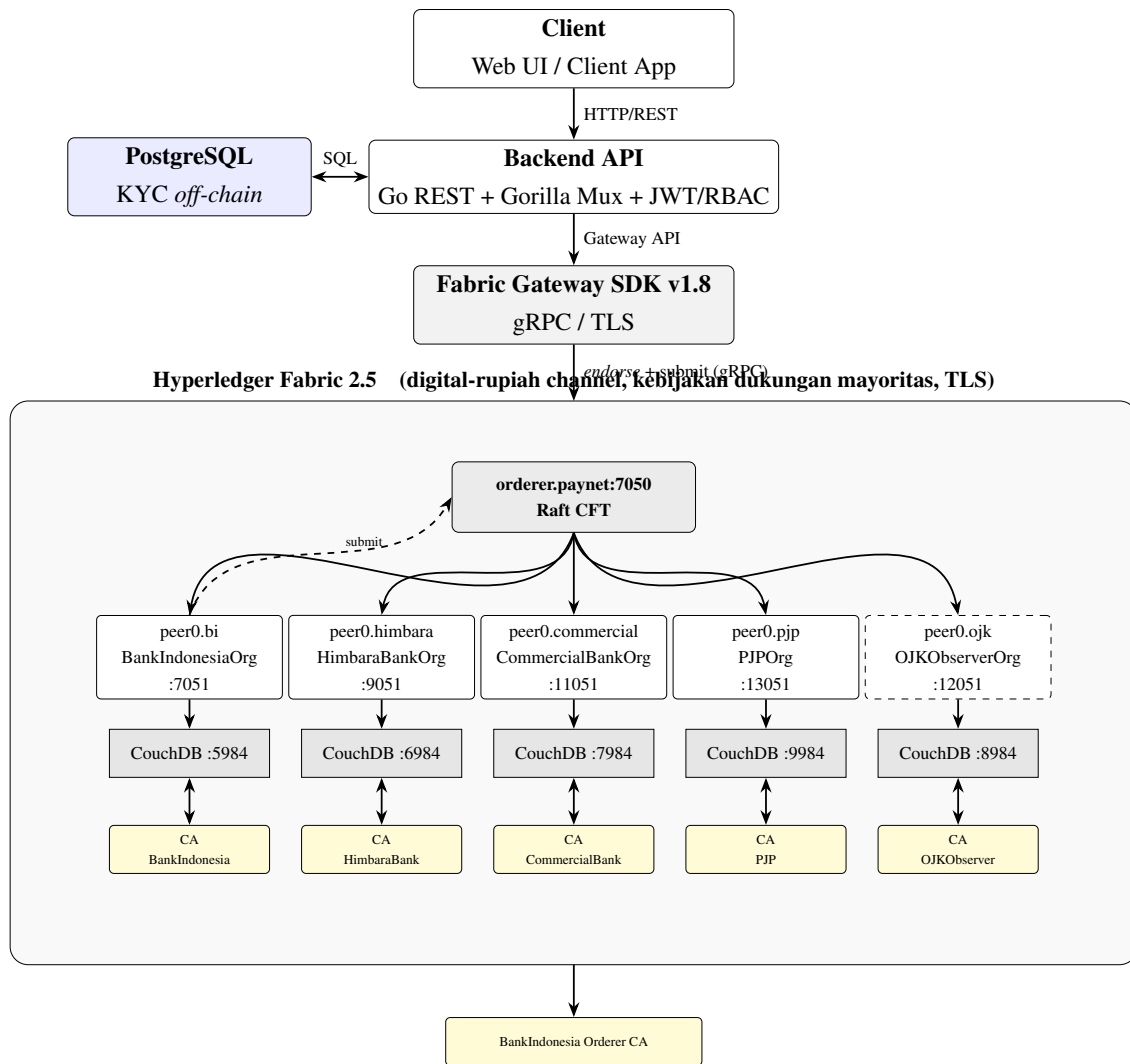


Gambar 3.22. Activity diagram laporan dan pengawasan supervisi

Figure 8. The Technology Architecture of Digital Rupiah



Gambar 3.24. Arsitektur teknologi Rupiah Digital menurut White Paper CBDC Bank Indonesia. Sumber: Bank Indonesia [7].



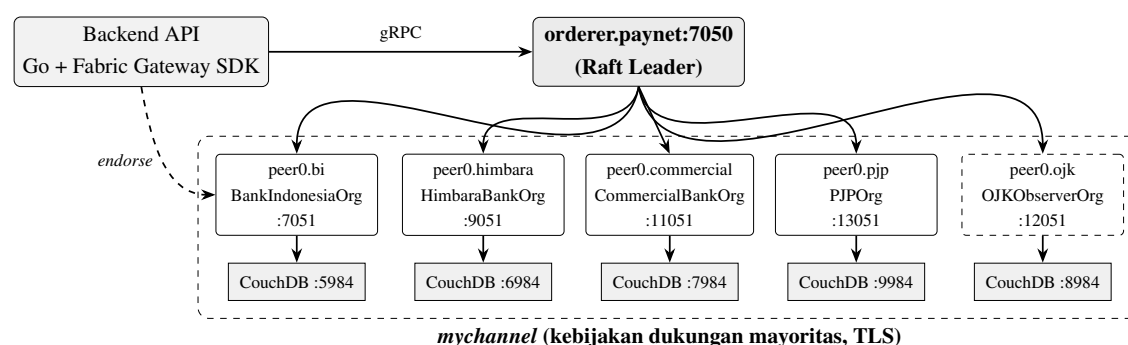
Gambar 3.25. Arsitektur sistem Rupiah Digital: lapisan *client*, *backend API* (Go REST + JWT/RBAC), basis data PostgreSQL luring, *Fabric Gateway SDK*, serta jaringan Hyperledger Fabric 2.5 dengan lima organisasi *peer*, CouchDB, CA per organisasi, dan *orderer* Raft

Jaringan *blockchain* dirancang dengan lima organisasi peserta dan satu organisasi *orderer*, merepresentasikan ekosistem Rupiah Digital multipihak. Setiap organisasi memiliki MSP dan *peer* tersendiri dengan basis data status CouchDB. Rincian organisasi disajikan pada Tabel 3.1.

Tabel 3.1. Organisasi pada jaringan Rupiah Digital

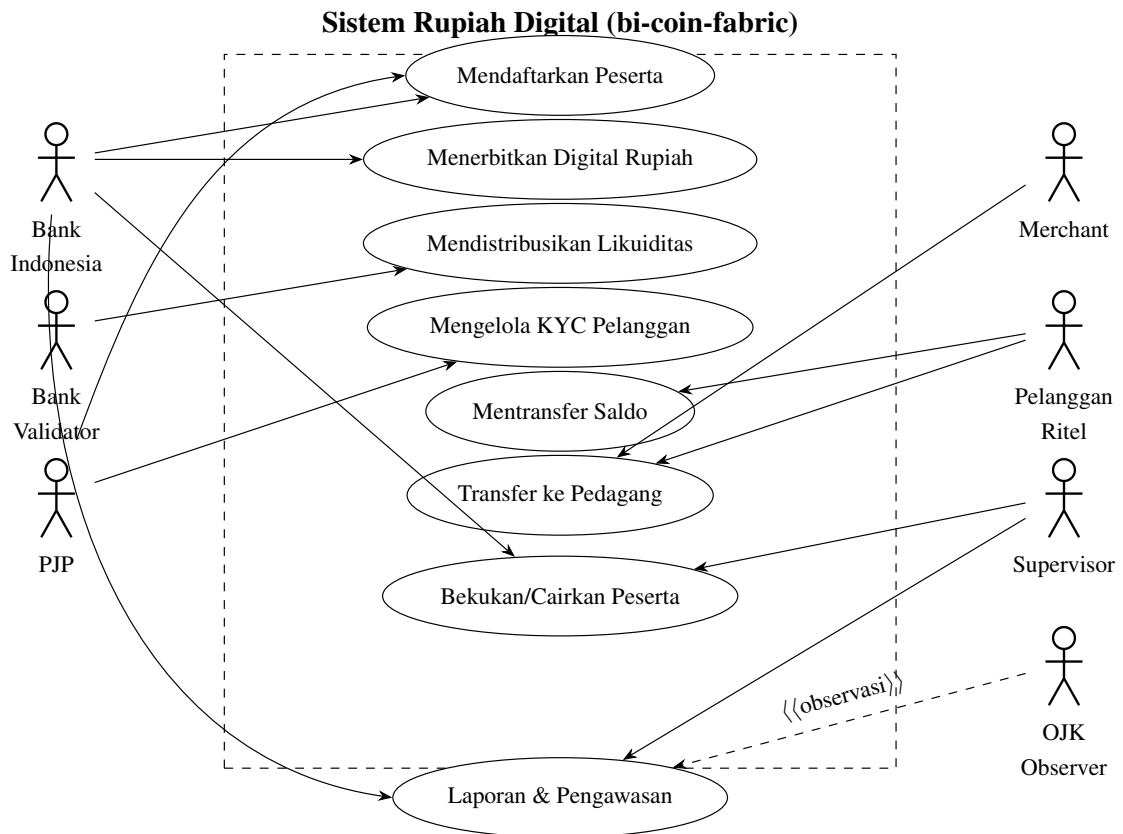
Organisasi	Peran	Keterangan
BankIndonesiaOrg	Penerbit	Penerbitan dan penarikan Digital Rupiah, pengawasan
HimbaraBankOrg	Validator	Bank distributor likuiditas ke PJP
CommercialBank Org	Validator	Bank distributor likuiditas ke PJP
PJP Org	PJP	Penyedia jasa pembayaran ke pelanggan ritel
OJKObserverOrg	Observer	Pengamatan/pengawasan (baca)
BankIndonesia Orderer	Orderer	Layanan pengurutan (Raft)

Seluruh peserta tergabung dalam satu kanal (*channel*) dengan kebijakan dukungan *majority*, sehingga pemutakhiran *ledger* memerlukan dukungan dari mayoritas organisasi. Komunikasi antarsimpul diamankan dengan TLS. Topologi jaringan secara lengkap ditunjukkan pada Gambar 3.26, yang memperlihatkan hubungan antara *orderer*, kelima *peer*, basis data CouchDB masing-masing organisasi, serta *backend API* yang terhubung ke jaringan melalui *Fabric Gateway SDK*.

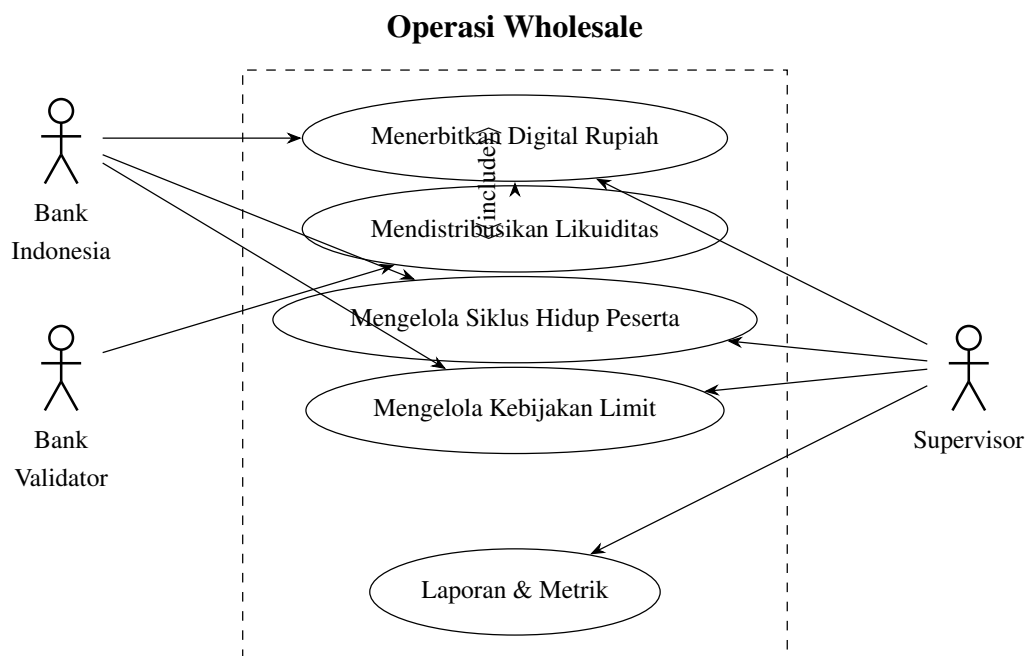


Gambar 3.26. Arsitektur jaringan Hyperledger Fabric sistem Rupiah Digital (5 organisasi)

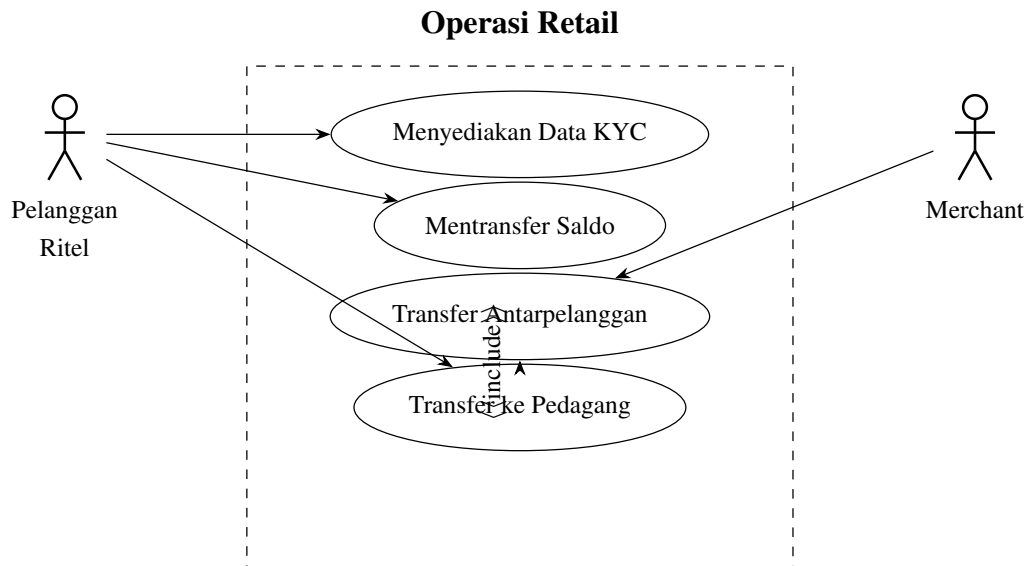
Kebutuhan fungsional sistem dimodelkan dengan diagram use case. Aktor sistem terdiri atas Bank Indonesia, bank validator, PJP, pedagang (*merchant*), pelanggan ritel, supervisor, dan pengamat OJK. Diagram use case keseluruhan ditunjukkan pada Gambar 3.27, sedangkan rincian operasi *wholesale* dan *retail* ditunjukkan pada Gambar 3.28 dan Gambar 3.29.



Gambar 3.27. Diagram use case keseluruhan sistem Rupiah Digital



Gambar 3.28. Diagram use case operasi wholesale



Gambar 3.29. Diagram use case operasi ritel

Pemetaan aktor pada diagram tersebut mengikuti kendali peran pada lapisan *backend*. Pendaftaran peserta diajukan oleh PJP atau Bank Indonesia (POST /participants), lalu disetujui oleh Bank Indonesia atau supervisor (/approve), sehingga pengajuan dan persetujuan terpisah (*four-eyes*). Pengamat OJK dan bank validator hadir sebagai organisasi pada jaringan Fabric (*peer* dengan akses baca pada *ledger*), bukan sebagai peran *backend* tersendiri; karena itu asosiasi OJK pada *use case* Laporan & Pengawasan digambarkan sebagai pengamatan tingkat *ledger*. Pada operasi ritel, data KYC disediakan oleh pelanggan, tetapi pengajuan profil KYC ke sistem (POST /kyc/profiles) dilakukan oleh operator PJP, bukan oleh pelanggan secara mandiri.

Status sistem disimpan pada *world state* dalam bentuk objek-objek yang dirangkum pada Tabel 3.2. Objek-objek ini menjadi dasar seluruh operasi *chaincode*.

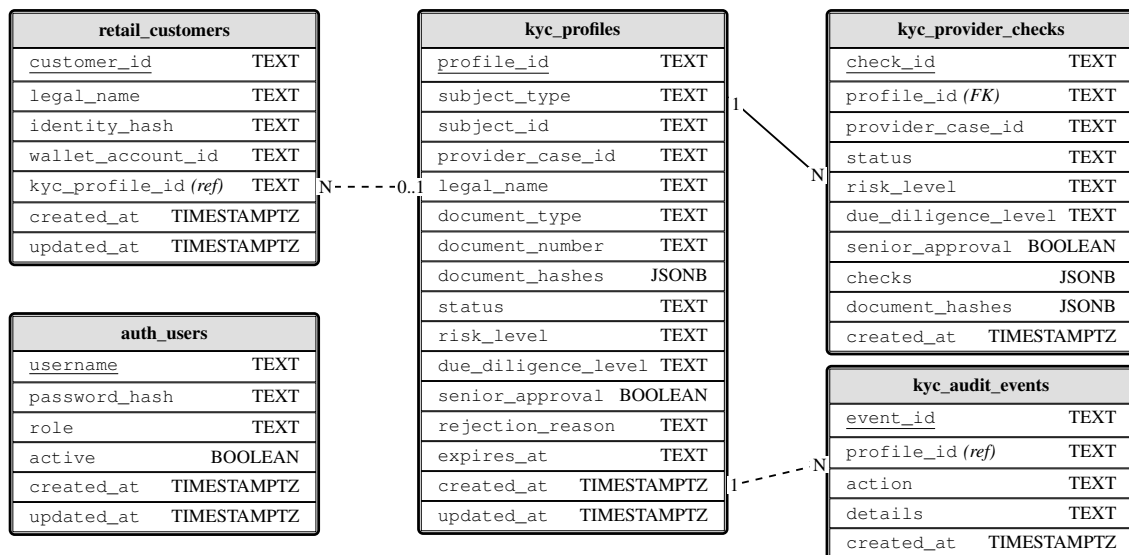
Tabel 3.2. Objek status utama pada ledger

Objek	Keterangan
Participant	Entitas peserta (validator/observer/PJP) beserta status siklus hidup
Wallet	Dompet dengan saldo, tingkatan ritel, status beku, serta penghitung keluar harian/bulanan dan masuk bulanan
TierLimit	Batas saldo, per transaksi, keluar, dan masuk per tingkatan

Objek	Keterangan
KycProfile	Jangkar KYC: jenis subjek, status, risiko, uji tuntas, persetujuan senior, masa berlaku, dan hash dokumen
TransactionRecord	Catatan transaksi untuk audit dan pelaporan
SystemLimit	Batas tingkat sistem (pasokan global, saldo per peserta, nilai per transaksi)

3.1.1.4 Perancangan Basis Data Luring

Selain status *on-chain* pada *world state* yang dirangkum pada Tabel 3.2, sistem memerlukan basis data relasional luring (*off-chain*) pada PostgreSQL untuk menyimpan data yang tidak layak atau tidak perlu ditempatkan pada *ledger*, yaitu kredensial autentikasi portal, pengayaan data KYC (nama legal, jenis dan nomor dokumen), serta riwayat pemeriksaan penyedia dan jejak audit yang bersifat *append-only*. Rancangan basis data luring tersebut ditunjukkan dengan diagram *Entity-Relationship* pada Gambar 3.30. Tabel `auth_users` berdiri sendiri untuk autentikasi portal, sedangkan `kyc_profiles` menjadi pusat relasi yang ditautkan oleh `kyc_provider_checks` melalui kendala *foreign key* berkebijakan *ON DELETE CASCADE*. Tautan lain, yakni `kyc_audit_events` dan `retail_customers`, menggunakan referensi lunak (tanpa kendala basis data) karena pengidentifikasinya seperti `subject_id` dan `wallet_account_id` mencerminkan pengidentifikasi *on-chain* yang otoritasnya berada pada *ledger*.



Keterangan: teks = *primary key* (PK); (FK) = *foreign key* dengan kendala basis data; (ref) = referensi lunak (tanpa kendala).
Garis utuh = relasi berkendala (ON DELETE CASCADE); garis putus-putus = referensi lunak.

Gambar 3.30. Diagram *Entity-Relationship* basis data luring (*off-chain*) pada PostgreSQL. Tabel `auth_users` berdiri sendiri untuk autentikasi portal, sedangkan `kyc_profiles` menjadi pusat relasi profil KYC, riwayat pemeriksaan penyedia, dan jejak audit.

3.1.2 Metode Implementasi

Pemilihan teknologi dalam implementasi didasarkan pada kesesuaian dengan karakteristik sistem Rupiah Digital yang bersifat *permissioned*, multitenjang, dan memerlukan kueri status yang kompleks.

3.1.2.1 Software Development Life Cycle

Metode *prototyping* dipilih sebagai SDLC penelitian ini. Sistem dibangun sebagai purwarupa fungsional yang mencakup seluruh komponen inti (*chaincode*, jaringan Fabric, dan *backend API*), kemudian dievaluasi secara kuantitatif. Pendekatan ini sesuai dengan karakteristik penelitian eksplorasi pada sistem *blockchain* yang belum memiliki spesifikasi teknis baku dari regulator [14].

3.1.2.2 Kerangka Kerja Blockchain

Hyperledger Fabric 2.5 dipilih sebagai kerangka kerja *blockchain* karena mendukung model keanggotaan berbasis MSP yang sesuai dengan struktur hierarkis Rupiah Digital dua jenjang [18]. Fabric memungkinkan pembatasan akses per peran melalui *endorsement policy*, pemisahan kanal, dan integrasi basis data status yang fleksibel.

3.1.2.3 Mekanisme Konsensus

Konsensus Raft (*etcdraft*) dipilih karena mendukung model *crash fault tolerance* (CFT) yang memadai untuk skenario jaringan berizin antarpeserta terpercaya [18]. Dibanding Kafka, Raft memiliki latensi *ordering* lebih rendah dan tidak bergantung pada infrastruktur tambahan.

3.1.2.4 State Database

CouchDB dipilih sebagai basis data status *world state* menggantikan LevelDB bawaan. CouchDB mendukung kueri kaya berbasis JSON (*rich query*) yang diperlukan untuk pelaporan portofolio dompet, pemfilteran KYC, dan pengawasan saldo. Kemampuan ini tidak dapat dicapai dengan efisien menggunakan LevelDB yang hanya mendukung kueri berbasis kunci. Perbandingan kedua pilihan tersebut dirangkum pada Tabel 3.3.

Tabel 3.3. Perbandingan basis data status Hyperledger Fabric. Diadaptasi dari Hanif [1] dan dokumentasi resmi Hyperledger Fabric [5].

Kriteria	LevelDB	CouchDB
Model data	Pasangan kunci–nilai yang tertanam pada proses <i>peer</i>	Dokumen JSON pada layanan basis data terpisah
Kueri	Pembacaan berdasarkan kunci dan rentang kunci	Kueri kaya berbasis bidang JSON dan selector
Indeks	Tidak menyediakan indeks JSON	Mendukung indeks JSON untuk kueri yang deterministik
Operasional	Konfigurasi dan kebutuhan sumber daya lebih sederhana	Memerlukan kontainer dan pengelolaan indeks tambahan
Kesesuaian	Status sederhana dengan pola akses berbasis kunci	Pelaporan dompet, pemfilteran KYC, dan pengawasan saldo

3.1.2.5 Basis Data Luring

Sementara CouchDB menampung *world state* Fabric, basis data luring (*off-chain*) yang menampung data sensitif memerlukan paradigma yang berbeda. Pemilihan basis data relasional PostgreSQL didasarkan pada lima kriteria yang dipaparkan berikut ini, kemudian dirangkum pada Tabel 3.4.

Relasional versus non-relasional. Data KYC dan kredensial autentikasi merupakan data terstruktur dengan skema kaku, hubungan antarentitas yang jelas (profil → pemeriksaan penyedia → jejak audit), serta persyaratan integritas transaksional yang ketat. Basis data relasional menjamin ketiga aspek tersebut melalui kepatuhan ACID penuh, kendala *foreign key*, dan transaksi DDL yang atomik. Sebaliknya, basis data dokumen NoSQL (mis. MongoDB) dirancang untuk data semiterstruktur dengan skema fleksibel serta beban baca dominan, dan secara tradisional mengorbankan konsistensi transaksional demi skalabilitas horizontal [59]. Untuk jejak audit yang bersifat *append-only* dan riwayat pemeriksaan KYC, penjaminan bahwa suatu peristiwa benar-benar tersimpan sebelum komit (MVCC tanpa *read skew*) merupakan kebutuhan mutlak, bukan opsional.

PostgreSQL versus MySQL dan SQL Server. Antara Alternatif relasional, PostgreSQL dipilih berdasarkan pertimbangan teknis berikut:

1. *JSON/JSONB dengan indeks GIN.* Dokumen pendukung KYC (luisan metadata, struktur respons penyedia) disimpan sebagai JSONB yang dapat diindeks secara native melalui GIN, memungkinkan kueri *rich* tanpa kehilangan kepatuhan ACID [60]. MySQL hanya mendukung tipe JSON dengan kapabilitas pengindeksan yang lebih terbatas.
2. *Row-Level Security (RLS).* PostgreSQL menyediakan RLS secara native, sehingga kebijakan isolasi data antarorganisasi peserta (mis. bank validator hanya membaca KYC nasabahnya sendiri) dapat ditekankan pada tingkat basis data, bukan sekadar lapisan aplikasi. Hal ini sejalan dengan prinsip regulasi UU Pelindungan Data Pribadi (UU PDP) yang mensyaratkan kontrol akses berbasis peran. MySQL tidak memiliki fitur ini dan SQL Server hanya menyediakannya pada edisi Enterprise [60].
3. *MVCC dan transaksi DDL.* PostgreSQL mengelola konkurensi via MVCC tanpa *lock* memblokir pembaca, dan membungkus perubahan skema (DDL) dalam transaksi yang dapat dieksekusi secara atomik — fitur yang tidak dimiliki MySQL karena DDL-nya bersifat *implicit commit* [61].
4. *Lisensi terbuka.* Lisensi PostgreSQL mirip BSD memungkinkan penggunaan, modifikasi, dan distribusi tanpa biaya royalti, menghindari *vendor lock-in*. SQL Server bersifat proprietari, sedangkan MySQL berlisensi ganda (GPL/komersial) yang membatasi integrasi pada produk berlisensi tidak kompatibel [61].
5. *Kesesuaian ekosistem Fabric.* Pola arsitektur Fabric merekomendasikan penyimpanan data sensitif di luar *ledger* melalui *private data collections* atau basis data eksternal, dengan hanya hash kriptografis yang dibukukan [62]. PostgreSQL merupakan *backend* relasional yang banyak diadopsi dalam implementasi Fabric

pendidikan dan industri, sehingga ketersediaan *driver* Go (pgx) dan pola integrasi yang teruji lebih matang dibandingkan alternatif lain.

Pola pemisahan on-chain/off-chain yang diadopsi penelitian ini sejalan dengan rekomendasi BIS untuk arsitektur CBDC ritel hibrid, yakni bank sentral mempertahankan jangkar kriptografis pada *ledger* sementara perantara menyimpan data identitas pada sistem terpisah yang menjalankan KYC [6]. Pendekatan serupa terlihat pada proyek Sand Dollar (Bahama) yang mengelola data KYC pada *layer* identitas digital terpisah dari buku besar [57], serta uji coba e-Krona (Swedia) yang memisahkan registri peserta dari *ledger* token [58].

Tabel 3.4. Perbandingan basis data kandidat untuk penyimpanan luring KYC.

Kriteria		PostgreSQL	MySQL	SQL Server
Kepatuhan penuh	ACID	Ya (DDL transaksional)	Sebagian (DDL non-atomik)	Ya
JSONB + GIN	indeks	Native	Terbatas	Terbatas
Row-Level Security		Native	Tidak ada	Edisi Enterprise saja
MVCC tanpa baca	<i>lock</i>	Ya	Ya (InnoDB)	Ya (snapshot)
Lisensi		BSD-like (bebas)	GPL / komersial	Proprietari
Adopsi Fabric	ekosistem	Umum	Jarang	Jarang

3.1.2.6 Bahasa Pemrograman

Chaincode dan *backend API* ditulis dalam bahasa Go. Go dipilih karena (1) SDK *Fabric Contract API* dan *Fabric Gateway SDK* tersedia secara resmi dan aktif dipelihara untuk ekosistem Fabric; (2) model konkurensi goroutine cocok untuk beban I/O tinggi pada *backend API*; dan (3) kompilasi statis mempermudah distribusi kontainer tanpa dependensi eksternal [18].

3.1.3 Metode Pengujian

Pengujian terdiri atas pengujian fungsional dan pengujian kinerja. Pengujian fungsional memverifikasi bahwa setiap alur proses berjalan sesuai rancangan, termasuk penegakan limit dan kontrol akses. Pengujian kinerja dilakukan menggunakan

Hyperledger Caliper untuk mengukur *throughput* dan latensi pada berbagai skenario beban.

Skenario pengujian kinerja dirancang untuk mengamati pengaruh laju pengiriman transaksi (*send rate*), jumlah *worker* Caliper, dan jenis fungsi rantai blok terhadap *throughput* dan latensi. Pengujian transfer ritel dijalankan dengan 1, 2, dan 4 *worker*. Pengujian fungsi lain dijalankan terpisah agar hasil transfer, KYC, operasi moneter, pembacaan pengawasan, dan kebijakan administratif tidak dicampur menjadi satu angka TPS umum. Konfigurasi ronde pengujian yang digunakan dirangkum pada Tabel 3.5.

Tabel 3.5. Skenario beban pengujian kinerja

Ronde	Jumlah tx	Laju (TPS)	Beban kerja
Pemanasan	40	10	Transfer ritel, 1/2/4 <i>worker</i>
Beban stabil	40	30	Transfer ritel, 1/2/4 <i>worker</i>
Beban puncak	40	60	Transfer ritel, 1/2/4 <i>worker</i>
Fungsional terpisah	40–80	5–30	KYC, moneter, pengawasan, administratif

Metrik yang diukur meliputi *throughput* (TPS), latensi rata-rata, latensi maksimum dan minimum, serta tingkat keberhasilan transaksi [51]. Mengikuti definisi metrik kinerja blockchain dan formulasi yang digunakan Hanif [1], tingkat keberhasilan transaksi dihitung dengan Persamaan 3-1.

$$R_s = \frac{N_{\text{valid}}}{N_{\text{submitted}}} \times 100\% \quad (3-1)$$

Throughput valid dihitung dengan Persamaan 3-2.

$$T = \frac{N_{\text{valid}}}{t_{\text{end}} - t_{\text{start}}} \quad (3-2)$$

Latensi transaksi ke-*i* dihitung dengan Persamaan 3-3.

$$L_i = t_{\text{confirm},i} - t_{\text{submit},i} \quad (3-3)$$

Pada persamaan tersebut, N_{valid} adalah jumlah transaksi valid, $N_{\text{submitted}}$ adalah jumlah transaksi yang dikirim, dan selang $t_{\text{end}} - t_{\text{start}}$ dinyatakan dalam detik. Kegagalan

transaksi akibat konflik MVCC dicatat dan dianalisis sebagai bagian dari karakterisasi sistem, bukan dihilangkan dari hasil pengukuran.

Justifikasi pemilihan parameter. Parameter pengujian dipilih berdasarkan dua pertimbangan: literatur dan rekayasa sistem.

Berdasarkan literatur. Variasi 1, 2, dan 4 *worker* digunakan untuk memisahkan pengaruh pembangkit beban Caliper dari kapasitas jaringan Fabric. Dengan kebijakan *endorsement majority* (3 dari 5 organisasi), penambahan *worker* tidak otomatis meningkatkan kapasitas jaringan karena bottleneck dapat berpindah ke eksekusi *peer*, validasi blok, atau pemrosesan *orderer* [2, 18, 33]. Kontrol laju *fixed-rate* dipilih karena menahan laju pengiriman sebagai variabel independen sehingga penurunan *throughput* dan peningkatan latensi dapat diamati sebagai fungsi dari laju tersebut; kontrol *fixed-load* menyesuaikan TPS secara otomatis berdasarkan latensi sehingga menyembunyikan perilaku saturasi [2]. Laju 10 TPS digunakan sebagai garis dasar, 30 TPS sebagai beban stabil, dan 60 TPS sebagai beban puncak pada purwarupa lokal. Nilai tersebut dipilih agar seluruh profil dapat dijalankan pada topologi Garuda lokal dengan target keberhasilan 100%, bukan untuk mengklaim kapasitas produksi nasional.

Berdasarkan pertimbangan teknis sistem. Setiap ronde transfer mengirim 40 permintaan terukur. Profil 1 *worker* menggunakan 40 slot transaksi per *worker*, 80 pelanggan, dan 10 pedagang; profil 2 *worker* menggunakan 20 slot, 80 pelanggan, dan 10 pedagang per *worker*; profil 4 *worker* menggunakan 10 slot, 40 pelanggan, dan 10 pedagang per *worker*. Populasi tersebut menyediakan pengirim, penerima STANDARD, dan pedagang yang berbeda untuk setiap transaksi yang memutasi *world state*. Jenis transaksi ditentukan secara deterministik berdasarkan slot global yang diskalakan terhadap jumlah slot sehingga komposisi gabungan tetap 70% transfer antarpelanggan, 25% transfer pelanggan-ke-pedagang, dan 5% pembacaan dompet pada setiap profil *worker*. Pemilihan kunci tanpa pengulangan digunakan untuk memperoleh garis dasar kapasitas jaringan tanpa mencampurkan saturasi pemrosesan dengan konflik MVCC akibat dompet yang sama. Skenario *hot-wallet contention* tidak digunakan karena penelitian tidak memiliki jejak transaksi empiris untuk menetapkan konsentrasi akses maupun kebijakan *retry/backoff* yang representatif. Saldo awal STANDARD ditetapkan Rp5 juta dari batas Rp20 juta dan saldo BASIC Rp500 ribu dari batas Rp2 juta. Populasi pelanggan terdiri atas 20% BASIC dan 80% STANDARD, sedangkan seluruh pedagang memperoleh MERCHANT dari profil KYC. Seluruh nilai ini merupakan pilihan rekayasa, bukan estimasi distribusi pengguna nasional.

Kesesuaian dengan skenario CBDC ritel. Beban kerja pengujian dirancang untuk merepresentasikan alur transaksi Rupiah Digital ritel secara struktural. Campuran

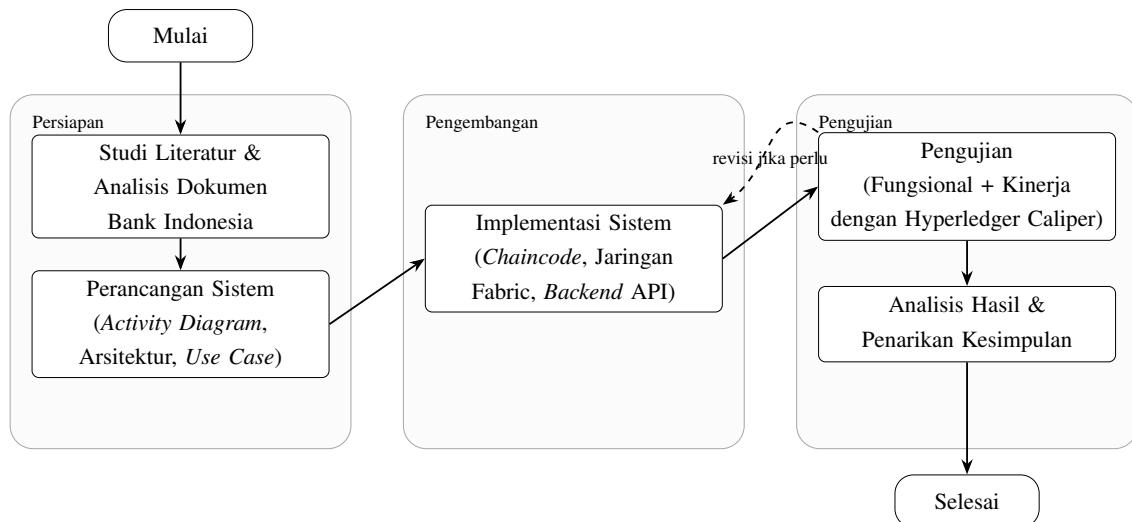
transaksi terdiri atas 70% transfer antarpelanggan, 25% transfer pelanggan-ke-pedagang, dan 5% pembacaan dompet. Komposisi ini dipilih karena CBDC ritel adalah instrumen pembayaran umum yang harus melayani pembayaran antarpengguna dan pembayaran ke pedagang, bukan hanya setelmen antarlembaga. Literatur teknologi CBDC ritel menempatkan desain dompet, privasi, dan arsitektur distribusi sebagai elemen inti sistem pembayaran publik [6], sedangkan e-CNY dan eNaira menunjukkan preseden operasional bahwa dompet ritel dapat dibedakan menurut tingkat identifikasi dan limit [47, 48]. Oleh karena itu, beban kerja yang hanya menguji transfer institusional tidak memadai untuk menjawab rumusan masalah purwarupa Rupiah Digital ritel.

Model dua jenjang penerbitan ($BI \rightarrow bank \rightarrow PJP \rightarrow pelanggan\ ritel$) sesuai dengan rancangan Rupiah Digital yang ditetapkan dalam dokumen konsultatif Bank Indonesia [13, 14]. Logika bisnis yang diuji bukan beban sintetik yang terlepas dari sistem, melainkan *chaincode* aktual yang mengimplementasikan profil KYC, penurunan tingkatan BASIC, STANDARD, dan MERCHANT, limit transaksi masuk dan keluar, serta pembacaan status dompet. Penggunaan Caliper dan metrik *throughput*/latensi mengikuti praktik pengukuran kinerja blockchain yang menekankan laju transaksi, latensi, dan karakterisasi kegagalan transaksi [2, 33, 51].

Perbedaan utama dengan sistem CBDC ritel produksi adalah skala (200 pelanggan per *worker* berbanding jutaan pengguna nyata), topologi jaringan (satu *peer* per organisasi dalam satu mesin lokal berbanding jaringan terdistribusi geografis), dan integrasi KYC nyata (disimulasikan secara terprogram). Keterbatasan ini berlaku umum pada studi prototipe CBDC berbasis *permissioned blockchain* [16] dan bukan kelemahan metodologis melainkan batasan cakupan yang dinyatakan secara eksplisit pada Batasan Penelitian (Bab I).

3.2 Alur Tugas Akhir

Tugas akhir ini dilaksanakan melalui tahapan berikut. Gambar 3.31 menyajikan alur pelaksanaan penelitian secara keseluruhan, mulai dari studi literatur hingga analisis hasil, dengan umpan balik iteratif antara implementasi dan pengujian sesuai siklus *prototyping*.



Gambar 3.31. Alur pelaksanaan tugas akhir: dari studi literatur hingga penarikan kesimpulan, dengan umpan balik iteratif antara implementasi dan pengujian

3.2.1 Mempelajari Proses Bisnis & Sistem Blockchain

Penelitian diawali dengan mempelajari konsep Rupiah Digital berdasarkan dokumen White Paper dan Consultative Paper Bank Indonesia [7, 13, 14], serta memahami mekanisme *blockchain* permissioned berbasis Hyperledger Fabric sebagai landasan implementasi.

3.2.2 Melakukan Studi Literatur

Mengkaji konsep CBDC, desain Rupiah Digital, Hyperledger Fabric, KYC berbasis risiko, dan limit dompet sebagai dasar perancangan sistem.

3.2.3 Menentukan Metode Penelitian

Menentukan aktor, kebutuhan fungsional, dan aturan bisnis sistem Rupiah Digital ritel berdasarkan model dua jenjang, serta menetapkan metode prototyping, pemilihan teknologi, dan parameter pengujian.

3.2.4 Pengembangan Sistem Blockchain bi-coin

Pengembangan sistem dilakukan secara iteratif meliputi instalasi infrastruktur Fabric, konfigurasi jaringan, *deploy chaincode*, pengembangan *gateway*, dan pengembangan *backend API*.

3.2.4.1 Instalasi Kerangka Kerja Hyperledger Fabric

Lingkungan pengembangan disiapkan dengan memasang Docker, Docker Compose, dan biner Fabric. Material kriptografi (sertifikat MSP dan TLS) dibangkitkan

menggunakan `cryptogen`. Blok genesis dan profil konfigurasi kanal dibuat dengan `configtxgen`.

3.2.4.2 Konfigurasi Jaringan Blockchain

Jaringan Fabric dibangun melalui skrip otomatisasi yang membangkitkan material kriptografi, blok genesis, dan konfigurasi kanal, kemudian menjalankan kontainer untuk lima *peer*, lima CouchDB, dan satu *orderer*. Chaincode dipaketkan, dipasang pada seluruh organisasi, disetujui sesuai kebijakan dukungan, lalu di-*commit* ke kanal.

3.2.4.3 Deploy Chaincode

Logika bisnis Rupiah Digital ditulis dalam bahasa Go menggunakan *Fabric Contract API* v2. Chaincode mengimplementasikan fungsi penerbitan (`Mint`, `RequestIssuance`), penarikan (`Burn`, `RequestRedemption`), distribusi berjenjang (`DistributeToParticipant`), transfer dengan penegakan limit (`Transfer`), KYC berbasis risiko, serta pelaporan dan pengawasan.

Gambar 3.32 menunjukkan implementasi fungsi `Mint` yang menambah saldo dompet setelah memverifikasi status beku dan batas tingkatan.

```

1 func (s *SmartContract) Mint(ctx contractapi.
    TransactionContextInterface ,
2     walletID string , amount int64) error {
3     if amount <= 0 {
4         return fmt.Errorf("amount must be positive")
5     }
6     wallet , err := s.getWallet(ctx , walletID)
7     if err != nil { return err }
8     if wallet.Frozen {
9         return fmt.Errorf("wallet %s is frozen", walletID)
10    }
11    limit , err := s.getLimit(ctx , wallet.Tier)
12    newBalance := wallet.Balance + amount
13    if newBalance > limit.MaxBalance {
14        return fmt.Errorf("mint would exceed max balance for
15        tier %s",
16            wallet.Tier)
17    }
18    wallet.Balance = newBalance
19    wallet.UpdatedAt = txNow(ctx).Format(time.RFC3339)
20    _ = s.emitAudit(ctx , "MINT" , walletID , "" , amount)
21    return s.recordTransaction(ctx , TxMint , walletID , "" , amount
    , TxSettled , "")
22 }

```

Gambar 3.32. Implementasi fungsi Mint pada *chaincode*: memverifikasi status beku dan batas saldo tingkatan sebelum menambah saldo dompet

Gambar 3.33 menunjukkan pemanggilan kebijakan transfer bersama. Fungsi kebijakan memeriksa batas per transaksi, pengeluaran harian dan bulanan, penerimaan bulanan, saldo pengirim, serta saldo maksimum penerima sebelum memperbarui penghitung secara deterministik.

```

1 func (s *SmartContract) Transfer(ctx contractapi.
    TransactionContextInterface ,
2     senderID string , receiverID string , amount int64) error {
3     // ... periksa sender & receiver tidak beku ...
4     senderLimit , _ := s.getLimit(ctx , sender.Tier)
5     receiverLimit , _ := s.getLimit(ctx , receiver.Tier)
6     if err := applyRetailTransferPolicy(&sender , &receiver ,
    amount ,
7         senderLimit , receiverLimit , txNow(ctx)); err != nil {
8         return err
9     }
10    sender.Balance -= amount
11    receiver.Balance += amount
12    // ... putWallet , recordTransaction ...
13 }

```

Gambar 3.33. Penegakan kebijakan transfer bersama untuk batas keluar, batas masuk, saldo maksimum, dan kecukupan saldo

Gambar 3.34 menunjukkan penurunan tingkatan dari bukti KYC. Peserta institusional ditolak dari fungsi ini karena memakai batas tingkat sistem, bukan tingkatan dompet ritel.

```

1 func deriveWalletTier(profile KycProfile, now time.Time) (
    WalletTier, error) {
2     if err := validateKycEligibility(profile, now); err != nil {
3         return "", err
4     }
5     if profile.SubjectType == SubjectParticipant {
6         return "", fmt.Errorf("participant wallets are outside
    retail tiering")
7     }
8     if profile.DueDiligenceLevel == DueDiligenceSimplified &&
9         profile.SubjectType == SubjectRetailCustomer &&
10        profile.RiskLevel == RiskLow {
11        return TierBasic, nil
12    }
13    if profile.SubjectType == SubjectMerchant {
14        return TierMerchant, nil
15    }
16    return TierStandard, nil
17 }

```

Gambar 3.34. Penurunan tingkatan dompet ritel dari jenis subjek, risiko, dan tingkat uji tuntas KYC

3.2.4.4 Pengembangan Fabric Gateway

Fabric Gateway SDK v1.8 digunakan untuk menghubungkan *backend API* ke jaringan Fabric melalui gRPC. *Gateway* mengelola siklus transaksi mulai dari pembuatan proposal, pengumpulan *endorsement*, pengiriman ke *orderer*, hingga konfirmasi blok, sehingga lapisan *backend* tidak perlu menangani protokol Fabric secara langsung.

3.2.4.5 Pengembangan Backend

Lapisan *backend* ditulis dalam bahasa Go menggunakan Gorilla mux dan *Fabric Gateway SDK*. Backend menyediakan antarmuka REST, menegakkan kontrol akses berbasis peran melalui *middleware*, serta menerjemahkan permintaan menjadi pemanggilan `SubmitTransaction` atau `EvaluateTransaction` pada *chaincode*. Gambar 3.35 menunjukkan pengelompokan rute REST berdasarkan peran akses menggunakan *middleware* `RequireRole`.

```

1 func (h *Handler) RegisterRoutes(r *mux.Router) {
2     // Publik: tidak memerlukan autentikasi
3     r.HandleFunc("/health", h.Health).Methods("GET")
4
5     // Pembacaan dompet oleh aktor terautentikasi
6     auth := r.NewRoute().Subrouter()
7     auth.Use(middleware.RequireRole(
8         middleware.RoleAuthenticated, middleware.RoleKycVerified
9         ,
10        middleware.RoleBankPjp, middleware.RoleBankIndonesia))
11    auth.HandleFunc("/wallets", h.ListWallets).Methods("GET")
12
13    // Bank/PJP melakukan onboarding dan keputusan KYC
14    kycMutation := r.NewRoute().Subrouter()
15    kycMutation.Use(middleware.RequireRole(middleware.
16        RoleBankPjp))
17    kycMutation.HandleFunc("/kyc/profiles", h.SubmitKycProfile).
18    Methods("POST")
19    kycMutation.HandleFunc("/kyc/profiles/{id}/refresh",
20        h.RefreshKycProfile).Methods("POST")
21    kycMutation.HandleFunc("/wallets", h.CreateWallet).Methods("
22    POST")
23
24    // Pelanggan/pedagang yang disetujui melakukan transfer
25    payments := r.NewRoute().Subrouter()
26    payments.Use(middleware.RequireRole(
27        middleware.RoleKycVerified, middleware.RoleMerchant))
28    payments.HandleFunc("/transfers", h.Transfer).Methods("POST")
29 }

```

Gambar 3.35. Pengelompokan rute REST *backend* berdasarkan peran akses: setiap subrouter menerapkan *middleware* `RequireRole` yang berbeda sesuai hierarki wewenang

3.2.5 Pengujian Sistem Blockchain bi-coin

Pengujian mencakup dua tahap: pengujian fungsional untuk memverifikasi kebenaran seluruh alur proses, dan pengujian kinerja menggunakan Hyperledger Caliper untuk mengukur *throughput* dan latensi.

3.2.5.1 Konfigurasi Hyperledger Caliper

Hyperledger Caliper dikonfigurasi dengan file jaringan (*network config*) yang mendeskripsikan topologi *peer*, *orderer*, kanal, dan identitas MSP. Modul beban kerja (*workload modules*) ditulis dalam JavaScript untuk mensimulasikan transfer antarpelanggan, transfer pelanggan-ke-pedagang, pembacaan dompet, KYC, operasi moneter, kueri pengawasan, dan kebijakan administratif sesuai Tabel 3.5.

3.2.5.2 Menjalankan Pengujian Hyperledger Caliper

Pengujian dijalankan dengan kontrol laju *fixed-rate*. Profil transfer ritel dijalankan untuk 1, 2, dan 4 *worker*, sedangkan profil fungsi lain dijalankan secara terpisah. Caliper mengumpulkan metrik *throughput* (TPS), latensi rata-rata, latensi maksimum, dan tingkat keberhasilan transaksi, yang kemudian dianalisis pada BAB IV.

BAB IV

HASIL DAN PEMBAHASAN

Bab ini menyajikan hasil rancang bangun sistem Rupiah Digital ritel berbasis Hyperledger Fabric beserta pembahasannya. Bagian pertama memaparkan hasil implementasi sistem yang menjawab tujuan pertama dan kedua, bagian kedua memaparkan hasil evaluasi kinerja yang menjawab tujuan ketiga, dan bagian ketiga membandingkan hasil penelitian dengan penelitian terdahulu.

4.1 Hasil Implementasi Sistem

4.1.1 Jaringan Hyperledger Fabric

Jaringan *blockchain* berhasil dibangun dengan lima organisasi peserta (BankIndonesiaOrg, HimbaraBankOrg, CommercialBankOrg, PJPOrg, dan OJKObserverOrg) serta satu organisasi *orderer*. Setiap organisasi peserta menjalankan satu *peer* dengan basis data status CouchDB, dan layanan pengurutan menggunakan konsensus Raft (etcdraft). Seluruh komponen dijalankan sebagai *container* melalui Docker Compose. Material kriptografi, blok genesis, dan konfigurasi kanal dibangkitkan secara otomatis melalui skrip, kemudian *chaincode* digital-rupiah dipaketkan, dipasang pada seluruh organisasi, disetujui sesuai kebijakan dukungan *majority*, dan di-*commit* ke kanal. Dengan kebijakan dukungan *majority*, setiap pemutakhiran *ledger* memerlukan dukungan mayoritas organisasi sehingga tidak ada satu pihak yang dapat mengubah status secara sepihak.

4.1.2 Chaincode Digital Rupiah

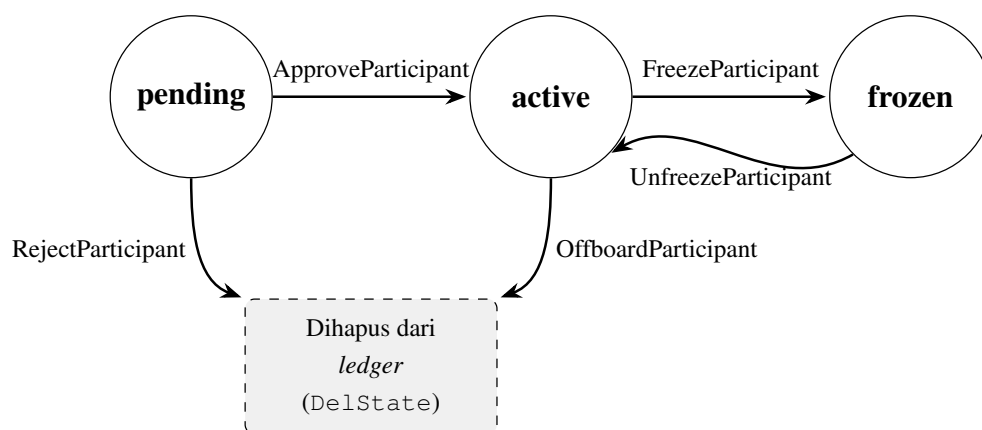
Chaincode digital-rupiah mengimplementasikan logika bisnis Rupiah Digital pada bahasa Go. Fungsi-fungsi *chaincode* dikelompokkan menurut domainnya sebagaimana dirangkum pada Tabel 4.1. Seluruh operasi yang mengubah status *ledger* mencatatkan jejak audit dan/atau *event* pengawasan sehingga dapat ditelusuri oleh pengawas.

Tabel 4.1. Kelompok fungsi chaincode digital-rupiah

Kelompok	Fungsi utama
Peserta	SubmitParticipant, ApproveParticipant, FreezeParticipant, UnfreezeParticipant, RejectParticipant, OffboardParticipant

Kelompok	Fungsi utama
Dompot	CreateWallet, CreateWholesaleWallet, Mint, Burn, Transfer, FreezeWallet, UnfreezeWallet
Likuiditas	RequestIssuance, RequestIssuanceRtgs, DistributeToParticipant, RequestRedemption
Limit	SetTierLimit, SetSystemLimit, SetAutoLimitPolicy
KYC	SubmitKycProfile, RefreshKycProfile, GetKycProfile
Pengawasan	GetReconciliationReport, GetMetrics, GetSupervisionEvents, GetTransactionHistory

Penegakan kepatuhan dilakukan langsung pada *chaincode*. Tingkatan dompet ritel diturunkan dari jenis subjek, tingkat uji tuntas, tingkat risiko, dan persetujuan senior pada profil KYC. Operator tidak dapat memilih tingkatan. Dompot institusional tidak menggunakan tingkatan ritel dan tunduk pada batas tingkat sistem. Tabel 4.2 menunjukkan parameter kebijakan purwarupa, sedangkan Gambar 4.36 menunjukkan siklus hidup peserta dari pendaftaran hingga pelepasan keanggotaan.



Gambar 4.36. Siklus hidup peserta pada *chaincode*: dari *pending* (menunggu persetujuan) menjadi *active*, kemudian dapat dibekukan (*frozen*) dan dicairkan kembali. Penolakan (*RejectParticipant*) maupun pelepasan keanggotaan (*OffboardParticipant*) menghapus rekaman peserta dari *ledger* (*DelState*), bukan menyimpannya sebagai status

Pada fungsi *Transfer*, sistem memeriksa status beku dompet, kelayakan KYC pengirim, batas per transaksi, pengeluaran harian, pengeluaran bulanan, penerimaan bulanan, kecukupan saldo pengirim, serta saldo maksimum penerima sebelum transaksi dieksekusi.

Tabel 4.2. Batas tingkatan dompet ritel (dalam Rupiah)

Tingkatan	Saldo maks.	Per transaksi	Harian keluar	Bulanan keluar	Bulanan masuk
BASIC	2.000.000	250.000	500.000	5.000.000	20.000.000
STANDARD	20.000.000	2.500.000	10.000.000	40.000.000	40.000.000
MERCHANT	200.000.000	10.000.000	50.000.000	500.000.000	500.000.000

Selain limit per tingkatan, sistem menetapkan batas tingkat sistem berupa pasokan global maksimum Rp10.000.000.000.000, saldo maksimum per peserta Rp1.000.000.000.000, dan nilai maksimum per transaksi Rp100.000.000.000.

4.1.3 Backend API

Lapisan *backend* menyediakan antarmuka REST yang menerjemahkan permintaan HTTP menjadi pemanggilan *chaincode* melalui *Fabric Gateway SDK*. Kontrol akses berbasis peran ditekankan melalui *middleware* dengan tujuh peran sebagaimana dirangkum pada Tabel 4.3. Pemisahan peran ini memastikan bahwa, misalnya, hanya peran `bank_indonesia` yang dapat melakukan penerbitan, sedangkan peran `supervisor` hanya memiliki akses baca terhadap laporan pengawasan.

Tabel 4.3. Peran akses pada backend API

Peran	Kewenangan utama
public	Pemeriksaan kesehatan dan topologi jaringan
authenticated	Pembacaan daftar dompet
kyc_verified	Transfer dan pembacaan saldo pelanggan yang telah disetujui
bank_pjp	Onboarding pelanggan, keputusan KYC, pembuatan dompet, dan operasi peserta
bank_indonesia	Penerbitan, kebijakan limit, dan pengawasan; akses baca KYC
merchant	Transfer dan pembacaan saldo pedagang yang telah disetujui
supervisor	Akses baca KYC, laporan, dan pengawasan

4.1.4 Hasil Pengujian Fungsional

Pengujian fungsional dilakukan untuk memverifikasi setiap alur proses yang dirancang pada BAB III. Hasil pengujian menunjukkan bahwa seluruh alur berjalan sesuai rancangan, yaitu:

1. **Penerbitan dan distribusi berjenjang.** Bank Indonesia menerbitkan Rupiah Digital ke peserta; bank validator mendistribusikan likuiditas ke PJP melalui `DistributeToParticipant` yang melakukan `Burn` pada pengirim dan `Mint` pada penerima; PJP kemudian menyalurkan saldo ke pelanggan ritel. Upaya penerbitan langsung kepada PJP ditolak sesuai aturan, sehingga PJP hanya memperoleh likuiditas melalui bank.
2. **Penegakan limit.** Transaksi yang melampaui batas per transaksi, keluar harian, keluar bulanan, atau masuk bulanan ditolak, demikian pula transfer yang menyebabkan saldo penerima melampaui batas.
3. **KYC dan pembekuan.** Bank/PJP mengajukan dan memperbarui profil KYC. BASIC diturunkan untuk pelanggan risiko rendah dengan uji tuntas sederhana; STANDARD untuk pelanggan dengan uji tuntas standar/lanjutan; MERCHANT untuk pedagang dengan uji tuntas standar/lanjutan. Risiko tinggi memerlukan EDD dan persetujuan senior. Dompet yang dibekukan tidak dapat bertransaksi.
4. **Transfer ritel.** Jalur `Transfer` yang sama berhasil digunakan untuk transaksi

antarpelanggan dan pelanggan-ke-pedagang.

5. **Pengawasan.** Laporan rekonsiliasi, metrik, dan *event* pengawasan dapat diambil oleh peran berwenang.

4.2 Hasil Evaluasi Kinerja

Evaluasi kinerja menggunakan Hyperledger Caliper 0.6 pada jaringan lokal lima organisasi dengan sapuan konfigurasi *worker* (1, 2, dan 4 *worker*) untuk mengidentifikasi batas pemrosesan sisi server. Setiap profil transfer menjalankan 40 transaksi pada laju kirim sekitar 10 TPS (pemanasan), 30 TPS (stabil), dan 60 TPS (puncak). Dua puluh persen pelanggan memperoleh BASIC melalui uji tuntas sederhana, sedangkan 80% memperoleh STANDARD melalui uji tuntas standar; pedagang memperoleh MERCHANT.

Beban terukur hanya memakai jalur yang masih berada dalam lingkup penelitian: 70% transfer antarpelanggan, 25% transfer pelanggan-ke-pedagang, dan 5% pembacaan dompet. Setiap subjek diberi jangkar KYC yang disetujui sebelum ronde dimulai. Dana awal STANDARD sebesar Rp5.000.000 dan BASIC sebesar Rp500.000 menyisakan ruang penerimaan di bawah saldo maksimum. Pengukuran melaporkan jumlah berhasil dan gagal, *throughput*, serta latensi minimum, rata-rata, dan maksimum. Nomor slot global menentukan jenis transaksi secara deterministik dan setiap kunci dompet yang dimutasi hanya digunakan satu kali dalam satu ronde terukur. Kegagalan kebijakan dan konflik MVCC tetap dihitung jika terjadi.

Hasil diperoleh dari laporan Caliper setelah *chaincode* versi 2.0 dipasang pada *ledger* bersih dan *InitLedger* dijalankan. Angka hasil tidak diwariskan dari beban kerja versi terdahulu karena semantik transaksi, batas dompet, dan strategi pemilihan kunci berbeda. Prosedur ini mencegah perbandingan dua beban kerja yang tidak setara dan menjaga keterlacakan hasil.

Konfigurasi pada Tabel 4.4 merupakan beban ritel yang menyerupai perilaku sistem CBDC ritel pada tingkat purwarupa, bukan klaim lalu lintas produksi nasional. Kesesuaiannya berasal dari tiga aspek. Pertama, transaksi yang diukur adalah transfer antarpelanggan, transfer pelanggan-ke-pedagang, pembacaan dompet, KYC, operasi moneter, kueri pengawasan, dan kebijakan administratif, sesuai fungsi pembayaran umum pada CBDC ritel [6]. Kedua, populasi pengujian memuat pelanggan BASIC dan STANDARD serta pedagang MERCHANT dengan KYC yang disetujui, mengikuti prinsip dompet bertingkat dan limit berbasis identifikasi yang juga muncul pada e-CNY dan eNaira [47, 48]. Ketiga, laju tetap 10, 30, dan 60 TPS pada profil transfer memungkinkan observasi hubungan antara *send rate*, *throughput*, latensi, dan kegagalan transaksi sebagaimana dianjurkan dalam metrik kinerja blockchain [2, 51]. Dengan batasan tersebut, hasil Caliper layak digunakan untuk menilai kelayakan desain dan

membangun garis dasar pengujian lanjutan, tetapi tidak untuk mengekstrapolasi kapasitas nasional tanpa data transaksi riil, topologi produksi, dan perangkat keras setara.

Tabel 4.4. Konfigurasi evaluasi transfer ritel

Profil	Worker	Tx	Laju	Komposisi
Transfer	1/2/4	40	10 TPS	70% P2P, 25% pelanggan–pedagang, 5% baca
Transfer	1/2/4	40	30 TPS	70% P2P, 25% pelanggan–pedagang, 5% baca
Transfer	1/2/4	40	60 TPS	70% P2P, 25% pelanggan–pedagang, 5% baca
Fungsional	2	40–80	5–30 TPS	KYC, moneter, pengawasan, administratif

Pengujian dijalankan pada jaringan Garuda yang diatur ulang, *chaincode* digital-rupiah versi 2.0 dideploy ulang dengan `NETWORK_MODE=garuda`, kemudian `InitLedger` dijalankan dan kontainer *chaincode* dipanaskan sebelum Caliper dieksekusi. Laporan pengujian menggunakan cap waktu 2026-06-30-113543. Seluruh 616 permintaan *chaincode* yang dilaporkan Caliper berhasil dengan 0 kegagalan. Log *peer* tidak menunjukkan konflik MVCC maupun penolakan kebijakan/limit.

Tabel 4.5. Hasil evaluasi transfer ritel Hyperledger Caliper

Ronde	Worker	Sukses	Gagal	Kirim (TPS)	Throughput (TPS)	Latensi (detik)		
						Min.	Rerata	Maks.
Pemanasan	1	40	0	10,3	7,8	0,01	0,78	2,14
Stabil	1	40	0	30,8	12,7	0,00	0,62	2,15
Puncak	1	40	0	57,6	14,5	0,07	0,66	2,32
Pemanasan	2	40	0	10,5	7,8	0,01	0,80	2,14
Stabil	2	40	0	31,6	12,3	0,01	0,64	2,25
Puncak	2	40	0	61,0	14,5	0,03	0,69	2,27
Pemanasan	4	40	0	11,1	8,0	0,01	0,81	2,18
Stabil	4	40	0	33,3	12,9	0,01	0,64	2,16
Puncak	4	40	0	65,9	14,5	0,01	0,70	2,30

Pada profil transfer, seluruh kombinasi 1, 2, dan 4 *worker* menghasilkan 40 permintaan sukses dan 0 kegagalan pada setiap ronde. Peningkatan laju kirim dari sekitar 10 TPS ke 60 TPS tidak menaikkan *throughput* secara linear. *Throughput* puncak berada pada sekitar 14,5 TPS untuk ketiga jumlah *worker*. Hal ini menunjukkan bahwa pada topologi lokal, batas pemrosesan lebih dipengaruhi oleh jaringan Fabric dan proses validasi dibanding jumlah *worker* Caliper. Keberhasilan 100% pada pengujian berdurasi

terbatas ini menunjukkan bahwa transaksi valid dapat diselesaikan tanpa konflik kunci pada konfigurasi yang dilaporkan, bukan bahwa sistem siap menangani beban nasional.

Tabel 4.6. Hasil evaluasi fungsi lain Hyperledger Caliper

Profil	Sukses	Gagal	Kirim (TPS)	Throughput (TPS)	Latensi (detik)		
					Min.	Rerata	Maks.
KYC/onboarding	56	0	4,4	3,7	0,13	0,88	2,29
Operasi moneter	80	0	20,5	19,9	0,10	0,32	0,53
Kueri pengawasan	80	0	30,8	17,0	0,01	1,10	3,24
Kebijakan administratif	40	0	5,3	5,2	0,09	0,91	1,72

Tabel 4.6 menunjukkan bahwa fungsi nontransfer juga dapat dieksekusi tanpa kegagalan pada beban terbatas. Profil KYC/onboarding berisi beberapa pemanggilan *chaincode* untuk satu alur bisnis, sehingga angka sukses pada tabel dibaca sebagai jumlah permintaan *chaincode* sukses, bukan jumlah lengkap pelanggan yang di-onboard. Kueri pengawasan memiliki *throughput* lebih rendah daripada laju kirim karena sebagian kueri membaca himpunan data buku besar, sedangkan operasi moneter memiliki latensi rata-rata paling rendah pada pengujian ini.

4.3 Perbandingan Hasil Penelitian dengan Hasil Terdahulu

Perbandingan dengan purwarupa token sederhana berbasis Hyperledger Fabric milik Hanif [1] dirangkum pada Tabel 4.7. Data penelitian terdahulu dipakai untuk menjelaskan perkembangan rancangan, bukan sebagai klaim kapasitas yang setara.

Penelitian ini menambahkan distribusi dua jenjang, pemisahan data KYC luring dan jangkar kepatuhan pada buku besar, penurunan tingkatan dompet berbasis risiko, limit masuk dan keluar, serta peristiwa pengawasan. Perbandingan angka kinerja langsung tidak dilakukan karena perangkat keras, topologi, kebijakan dukungan, dan semantik beban kerja berbeda.

Tabel 4.7. Perbandingan purwarupa MeetCoin dan purwarupa Rupiah Digital ritel. Sumber penelitian terdahulu: Hanif [1].

Dimensi	MeetCoin	Penelitian ini
Domain	Token dan dompet digital sederhana	Purwarupa CBDC ritel dua jenjang
Topologi	Dua <i>peer</i> , satu <i>orderer</i> , dan satu kanal	Lima organisasi peserta, satu organisasi <i>orderer</i> , dan satu kanal
Basis data status	LevelDB berbasis kunci–nilai	CouchDB berbasis dokumen JSON dan kueri kaya
Identitas dan kepatuhan	Fabric CA dan autentikasi JWT tanpa tingkatan KYC	KYC luring, jangkar kepatuhan, serta tingkatan BASIC, STANDARD, dan MERCHANT
Beban kerja	<i>Mint</i> , cek saldo, dan transfer; 5/10/15 <i>worker</i> ; 1.000/2.000/3.000 transaksi; 50/100/150 TPS	70% transfer antarpelanggan, 25% pelanggan–pedagang, dan 5% baca dompet dengan dua identitas organisasi
Metrik	Keberhasilan, <i>throughput</i> , latensi, CPU, dan memori	Keberhasilan, <i>throughput</i> , latensi, serta konflik MVCC

Perbandingan dengan PoC Proyek Garuda. Laporan PoC Proyek Garuda [13] mendokumentasikan uji coba *wholesale Rupiah Digital* (wRD) pada tahap *Immediate State*. Berbeda dengan penelitian ini, PoC Garuda bersifat grosir dan mengevaluasi dua platform non-Fabric: R3 Corda (UTXO, konsensus Notary) dan Kaleido Hyperledger Besu (akun, konsensus QBFT/PoA), yang dipilih dari 39 kandidat menjadi dua finalis. Pengujian dilakukan dengan 55 skenario bisnis dan teknis menggunakan Locust Dashboard serta AWS CloudWatch. Hasil *load test* melaporkan *throughput* 37 TPS (Corda) dan 39 TPS (Besu), keduanya memenuhi ambang batas lalu lintas grosir sekitar 30 TPS. Arsitektur yang diuji terdiri atas enam lapisan (*use case*, aset digital, eksekusi, data, konsensus, jaringan) dengan aspek keamanan menyeluruh, serta peran Master Node BI sebagai KDR (*Khazanah Digital Rupiah*), Regulator, Observer, Administrator, dan Provider. PoC Garuda tidak mengevaluasi dompet ritel berjenjang, KYC bertingkat, pembayaran luring, atau kanal pelanggan–pedagang karena lingkupnya terbatas pada penerbitan, penebusan, dan transfer dana antarlembaga. Penelitian ini mengisi celah ritel tersebut dengan Hyperledger Fabric, dompet BASIC, STANDARD, dan MERCHANT, serta pengujian Caliper pada beban transfer ritel. Perbedaan platform (Fabric vs Corda/Besu), alat ukur (Caliper vs Locust), dan lingkup (ritel vs grosir) membuat angka TPS kedua penelitian tidak dapat dibandingkan secara langsung, tetapi saling melengkapi gambaran kelayakan DLT untuk CBDC Rupiah Digital pada lapis grosir dan ritel. Distribusi dua jenjang yang diarahkan Bank Indonesia [7, 14] tetap dipertahankan dalam purwarupa ini.

Perbandingan dengan tinjauan literatur Aryani (2023). Aryani [9] melakukan tinjauan literatur kritis komparatif terhadap *White Paper Project Garuda* dan

implementasi CBDC di empat negara (eNaira, Sand Dollar, e-CNY, Project Orchid). Kajian tersebut merekomendasikan adopsi dompet berjenjang dan KYC bertingkat untuk r-Digital Rupiah, dengan mengutip eNaira (lima tingkat), Sand Dollar (dua tingkat), dan e-CNY (berjenjang berdasarkan kekuatan identifikasi). Penelitian ini mengimplementasikan rekomendasi tersebut sebagai kebijakan yang dapat diuji: tiga tingkatan dompet (BASIC, STANDARD, MERCHANT) diturunkan dari jenis subjek, tingkat uji tuntas, tingkat risiko, dan persetujuan senior pada profil KYC. Aryani juga mengutip metrik kuantitatif dari literatur global yang menunjukkan rata-rata DLT privat hanya mencapai sekitar 20 TPS, jauh di bawah jaringan terpusat seperti VISA (hingga 65.000 TPS), serta menyatakan keprihatinan bahwa skalabilitas DLT mungkin menghambat *settlement* ritel sehingga r-Digital Rupiah mungkin memerlukan infrastruktur terpusat. Penelitian ini memberikan bukti empiris pada titik tersebut: pengujian Caliper pada Hyperledger Fabric menghasilkan *throughput* transfer 7,8–14,5 TPS dan operasi fungsional 3,7–19,9 TPS, yang konsisten dengan rentang rata-rata DLT privat yang dikutip Aryani dan menggarisbawahi perlunya evaluasi topologi produksi sebelum klaim kapasitas nasional. Seluruh 616 permintaan berhasil tanpa konflik MVCC maupun penolakan kebijakan, meskipun dengan keterbatasan satu inah dan topologi purwarupa yang belum mewakili produksi nasional. Selain itu, Aryani mencatat bahwa BI Reg. 20/6/PBI/2018 membatasi uang elektronik tak terdaftar hingga Rp2.000.000 dan terdaftar hingga Rp10.000.000, yang dapat berfungsi sebagai proksi limit dompet berjenjang; parameter Rupiah pada purwarupa ini dinyatakan sebagai pilihan desain dengan ketentuan uang elektronik domestik hanya sebagai proksi dan bukan dasar hukum CBDC [55].

Perbandingan dengan e-CNY dan eNaira. e-CNY dan eNaira menunjukkan bahwa klasifikasi dompet dan limit dapat dikaitkan dengan kekuatan identifikasi pengguna [47, 48]. Penelitian ini mengadopsi prinsip tersebut sebagai kebijakan yang dapat diuji, tetapi tidak menyalin angka limit negara lain. Parameter Rupiah pada purwarupa dinyatakan sebagai pilihan desain, dengan ketentuan uang elektronik domestik hanya sebagai proksi dan bukan dasar hukum CBDC [55].

Tabel 4.8. Perbandingan penelitian ini dengan tinjauan literatur Aryani (2023) dan PoC Proyek Garuda (2024).

Dimensi	Aryani (2023)	PoC Garuda (2024)	Penelitian ini
Jenis CBDC	Ritel (analisis konseptual)	Grosir (PoC eksperimental)	Ritel (purwarupa)
Metodologi	Tinjauan literatur kritis komparatif	PoC dengan 55 skenario bisnis dan teknis	Implementasi purwarupa + Caliper
Platform DLT	Tidak dievaluasi (kutipan dari kasus negara)	R3 Corda + Hyperledger Besu	Hyperledger Fabric
Dompot berjenjang	Direkomendasikan (eNaira 5 tingkat, Sand Dollar 2, e-CNY berjenjang)	Tidak ada (grosir saja)	BASIC, STANDARD, MERCHANT (3 tingkat)
KYC	Direkomendasikan bertingkat	Tidak dibahas eksplisit	Luring, jangkar kepatuhan, penurunan berbasis risiko
Metrik kinerja	Dikutip dari literatur (DLT ~20 TPS, VISA ~65.000 TPS)	Diukur: Corda 37 TPS, Besu 39 TPS (Locust)	Diukur: transfer 7,8–14,5 TPS; fungsional 3,7–19,9 TPS (Caliper)
Lingkup pengujian	Tidak ada pengujian	Penerbitan, penebusan, transfer antarlembaga	Transfer antarpelanggan, pelanggan–pedagang, baca dompet
Pembayaran luring	Direkomendasikan untuk r-Digital Rupiah	Tidak dibahas	Tidak diuji dalam purwarupa ini
Tokenisasi	Didukung (konseptual)	UTXO (Corda) / ERC-20 + ERC-1400 (Besu)	Transfer berbasis kunci–nilai pada Fabric

Batas generalisasi. Purwarupa berjalan pada satu inah, memakai satu *peer* per organisasi, dan menyimulasikan keputusan penyedia KYC. Oleh karena itu, hasil fungsional membuktikan konsistensi implementasi terhadap rancangan, bukan kesiapan produksi. Angka Caliper yang diperoleh berlaku untuk konfigurasi yang dilaporkan dan tidak dapat langsung digeneralisasi menjadi kapasitas nasional. Keterbatasan ini sejalan dengan keprihatinan Aryani [9] bahwa skalabilitas DLT pada volume ritel nasional belum terbukti dan r-Digital Rupiah mungkin memerlukan infrastruktur terpusat jika ambang kapasitas DLT terlampaui. Sebaliknya, PoC Garuda [13] hanya menguji lapis grosir hingga ambang 30 TPS dan belum mengeksplorasi kemampuan maksimum, on-premise, atau multi-validator; celah ritel yang tertinggal menjadi ruang yang diisi oleh penelitian ini, tetapi dengan batas purwarupa yang sama seperti di atas.

BAB V

KESIMPULAN DAN SARAN

5.1 Kesimpulan

Berdasarkan hasil rancang bangun dan pengujian, kesimpulan penelitian adalah sebagai berikut.

1. Purwarupa Rupiah Digital ritel dibangun pada Hyperledger Fabric 2.5 dengan lima organisasi, *backend* Go, CouchDB, dan PostgreSQL. Model distribusi dua jenjang memisahkan Bank Indonesia, bank validator, PJP, dan pengguna ritel. Dompet institusional berada di luar tingkatan KYC ritel.
2. Kebijakan KYC diterapkan sebagai penurunan tingkatan yang deterministik. Pelanggan memberikan data identitas; bank/PJP melakukan uji tuntas dan keputusan KYC; Bank Indonesia serta pengawas membaca jangkar kepatuhan dan peristiwa pengawasan. Data identitas mentah tidak disimpan pada buku besar. Risiko tinggi memerlukan EDD dan persetujuan senior.
3. Tingkatan BASIC, STANDARD, dan MERCHANT menegakkan batas saldo maksimum, per transaksi, keluar harian, keluar bulanan, dan masuk bulanan. Jalur transfer yang sama menangani transaksi antarpelanggan dan pelanggan-ke-pedagang, sehingga pemeriksaan KYC dan limit tidak terduplikasi.
4. Pengujian otomatis memverifikasi pemetaan KYC, penolakan risiko terlarang, persyaratan risiko tinggi, penghitung limit deterministik, pemisahan data privat, dan batas peran. Klaim kinerja hanya boleh dibuat dari laporan Caliper versi 2.0 dengan beban transfer yang didokumentasikan; hasil beban kerja lama tidak digunakan kembali.

5.2 Saran

1. Mengulang protokol Caliper pada beberapa perangkat keras, topologi jaringan, dan jumlah *worker* yang lebih luas, lalu melaporkan konfigurasi, versi *chaincode*, jumlah sukses/gagal, latensi, *throughput*, konflik MVCC, dan pengulangan setiap skenario bersama hasilnya.
2. Memperluas pemisahan profil benchmark fungsional. Pengujian penelitian ini telah memisahkan transfer ritel, KYC/*onboarding*, operasi moneter, kueri pengawasan, dan kebijakan administratif; penelitian lanjutan perlu menambahkan variasi ukuran data, distribusi baca/tulis, dan pengulangan statistik agar setiap profil dapat dibandingkan secara lebih kuat.
3. Menambahkan skenario *hot-wallet contention* sebagai eksperimen terpisah dari

garis dasar tanpa perebutan kunci. Skenario ini tidak digunakan pada pengujian utama karena tingkat konsentrasi pedagang dan pola pengulangan transaksi belum didasarkan pada jejak transaksi empiris, sehingga dapat menambahkan asumsi yang belum tervalidasi. Pengujian lanjutan perlu menetapkan konsentrasi akses dari jejak transaksi yang dianonimkan atau distribusi sintetis yang parameternya dilaporkan eksplisit, lalu mengukur konflik MVCC awal, keberhasilan setelah *retry*, *throughput* akhir, serta latensi persentil ke-95 dan ke-99. Mekanisme *retry* harus terbatas, memakai *backoff*, dan menjaga idempotensi agar transaksi tidak diterapkan lebih dari satu kali.

4. Memvalidasi angka limit dan distribusi tingkatan bersama Bank Indonesia, bank/PJP, serta ahli APU/PPT. Nilai penelitian adalah parameter purwarupa dan tidak boleh diperlakukan sebagai ketentuan CBDC.
5. Mengkaji remunerasi CBDC (*CBDC rate*)—berbunga untuk *wholesale* dan tanpa bunga untuk ritel—beserta dampaknya terhadap kebijakan moneter dan risiko *disintermediation* sebagai studi makroekonomi terpisah, melengkapi purwarupa fungsional ini [4, 29].
6. Mengintegrasikan penyedia eKYC nyata dan menerapkan kontrol retensi, enkripsi, penghapusan, serta audit akses untuk data identitas di PostgreSQL.
7. Menguji *private data collection* atau teknik kriptografi penjaga privasi untuk mengurangi keterlihatan metadata transaksi tanpa menghilangkan kemampuan pengawasan [24, 36, 37].
8. Mengembangkan integrasi, interoperabilitas, dan interkoneksi dengan infrastruktur pasar keuangan pada penelitian terpisah setelah kebijakan buku besar ritel tervalidasi [7].
9. Mengembangkan pembayaran tanpa konektivitas sebagai lingkup terpisah yang mencakup perangkat aman, pencegahan *double-spending*, dan rekonsiliasi sesuai Project Polaris [34, 35].

DAFTAR PUSTAKA

- [1] L. I. Hanif, “Rancang bangun sistem blockchain aplikasi meetcoin menggunakan kerangka kerja hyperledger fabric,” Universitas Gadjah Mada, Yogyakarta, Skripsi Sarjana, Departemen Teknik Elektro dan Teknologi Informasi, 2024.
- [2] Hyperledger Foundation, “Hyperledger caliper: A blockchain performance benchmark framework,” 2024, accessed: Jun. 27, 2026. [Online]. Available: <https://hyperledger.github.io/caliper/>
- [3] T. T. A. Dinh, J. Wang, G. Chen, R. Liu, B. C. Ooi, and K.-L. Tan, “BLOCKBENCH: A framework for analyzing private blockchains,” in *Proceedings of the 2017 ACM International Conference on Management of Data*. ACM, 2017, pp. 1085–1100.
- [4] F. Syarifuddin and T. Bakhtiar, “Monetary policy strategy in the presence of central bank digital currency,” Bank Indonesia, Working Paper WP/09/2021, 2021.
- [5] Hyperledger Foundation, “Couchdb as the state database,” 2024, accessed: Jun. 28, 2026. [Online]. Available: https://hyperledger-fabric.readthedocs.io/en/latest/couchdb_as_state_database.html
- [6] R. A. Auer and R. Böhme, “The technology of retail central bank digital currency,” *BIS Quarterly Review*, pp. 85–100, Mar. 2020. [Online]. Available: https://www.bis.org/publ/qtrpdf/r_qt2003j.htm
- [7] Bank Indonesia, “Proyek garuda: Menavigasi arsitektur digital rupiah — white paper central bank digital currency,” Bank Indonesia, Jakarta, Tech. Rep., 2022.
- [8] D. Urbančok, “Blockchain open-source software comparison,” Master’s thesis, Masaryk University, Faculty of Informatics, Brno, 2019.
- [9] J. N. Aryani, “Designing central bank digital currency in indonesia: A critical literature review,” 2023, tugas Akhir Sarjana, Sekolah Bisnis dan Manajemen, Institut Teknologi Bandung.
- [10] Bank Indonesia, “Blueprint sistem pembayaran indonesia 2025,” Bank Indonesia, Jakarta, Tech. Rep., 2019.
- [11] R. A. Auer, J. Frost, L. Gambacorta, C. Monnet, T. Rice, and H. Shin, “Central bank digital currencies: motives, economic implications and the research frontier,” *SSRN Electronic Journal*, 2022.
- [12] L. Dionysopoulos, M. Marra, and A. Urquhart, “Central bank digital currencies: A critical review,” *International Review of Financial Analysis*, vol. 91, p. 103031, 2024.
- [13] Bank Indonesia, “Proyek garuda: Wholesale digital rupiah cash ledger — proof of concept report,” Bank Indonesia, Jakarta, Tech. Rep., 2024.
- [14] —, “Consultative paper pengembangan rupiah digital: Tahap immediate state,” Bank Indonesia, Jakarta, Tech. Rep., 2023.

- [15] T. Zhang and Z. Huang, "Blockchain and central bank digital currency," *ICT Express*, vol. 8, no. 2, pp. 264–270, 2022.
- [16] V. Sethaput and S. Innet, "Sok: Blockchain applications in central bank digital currencies (cbdcs)," in *2025 7th International Conference on Blockchain Computing and Applications (BCCA)*. IEEE, 2025, pp. 195–202.
- [17] Bhawana and S. Kumar, "Permission blockchain network based central bank digital currency," in *2021 IEEE 4th International Conference on Computing, Power and Communication Technologies (GUCON)*. IEEE, 2021, pp. 1–6.
- [18] E. Androulaki, A. Barger, V. Bortnikov, C. Cachin, K. Christidis, A. De Caro, D. Enyeart, C. Ferris, G. Laventman, Y. Manevich *et al.*, "Hyperledger fabric: A distributed operating system for permissioned blockchains," in *Proceedings of the Thirteenth EuroSys Conference (EuroSys '18)*. Porto, Portugal: ACM, 2018, pp. 1–15.
- [19] J. Barrdear and M. Kumhof, "The macroeconomics of central bank digital currencies," *Journal of Economic Dynamics and Control*, p. 104148, 2021.
- [20] J. Kiff, J. Alwazir, S. Davidovic, A. Farias, A. Khan, T. Khiaonarong, M. Malaika, H. Monroe, N. Sugimoto, H. Tourpe, and P. X. Zhou, "A survey of research on retail central bank digital currency," *IMF Working Papers*, 2020.
- [21] S. Allen, S. Čapkun, I. Eyal, G. Fanti, B. Ford, J. Grimmelmann, A. Juels, K. Kostianen, S. Meiklejohn, A. K. Miller, E. Prasad, K. Wüst, and F. Zhang, "Design choices for central bank digital currency: Policy and technical considerations," *SSRN Electronic Journal*, 2020.
- [22] A. Tsareva and M. Komarov, "Retail central bank digital currency design choices: Guide for policymakers," *IEEE Access*, vol. 12, pp. 66 129–66 146, 2024.
- [23] M. M. Islam and H. P. In, "A privacy-preserving transparent central bank digital currency system based on consortium blockchain and unspent transaction outputs," *IEEE Transactions on Services Computing*, vol. 16, no. 4, pp. 2372–2386, 2023.
- [24] N. Pocher and A. Veneris, "Privacy and transparency in cbdcs: A regulation-by-design aml/cft scheme," *IEEE Transactions on Network and Service Management*, vol. 19, pp. 1776–1788, 2022.
- [25] P. Michalopoulos, O. Olowookere, N. Pocher, J. Sedlmeir, A. G. Veneris, and P. Puri, "Privacy and compliance design options in offline central bank digital currencies," *IEEE Transactions on Network and Service Management*, vol. 22, pp. 3748–3763, 2025.
- [26] J. Hans, S. Khan, and D. Svetinovic, "Blockchain cbdc security threats using stride," in *2023 Fifth International Conference on Blockchain Computing and Applications (BCCA)*. IEEE, 2023, pp. 522–529.
- [27] H. Sun, H. Mao, X. Bai, Z. Chen, K. Hu, and W. Yu, "Multi-blockchain model for central bank digital currency," in *2017 18th International Conference on Parallel and Distributed Computing, Applications and Technologies (PDCAT)*. IEEE, 2017, pp. 360–367.

- [28] B. M. Zams, R. Indrastuti, A. G. Pangrsa, N. A. Hasniawati, F. A. Zahra, and I. A. Fauziah, “Designing central bank digital currency for indonesia: The delphi–analytic network process,” *Bulletin of Monetary Economics and Banking*, vol. 23, no. 3, pp. 413–440, 2020.
- [29] F. Syarifuddin, “Optimal central bank digital currency design for emerging economies,” *Journal of Central Banking Law and Institutions*, 2024.
- [30] J. Ree, “Nigeria’s enaira, one year after,” International Monetary Fund, IMF Working Paper WP/23/104, 2023.
- [31] N. Bhensook, T. Rattanakul, W. Synsatayakul, and P. Tohwisessuk, “Public use and distribution of retail cbdc: An evidence from thailand’s retail cbdc pilot program,” *Jahrbücher für Nationalökonomie und Statistik*, vol. 245, pp. 367 – 399, 2025.
- [32] J. Lovejoy, C. Fields, M. Virza, T. Frederick, D. Urness, K. Karwaski, A. Brownworth, and N. Narula, “A high performance payment processing system designed for central bank digital currencies,” *IACR Cryptology ePrint Archive*, vol. 2022, p. 163, 2022. [Online]. Available: <https://eprint.iacr.org/2022/163>
- [33] M. Kuzlu, M. Pipattanasomporn, L. Gurses, and S. Rahman, “Performance analysis of a hyperledger fabric blockchain framework: Throughput, latency and scalability,” in *2019 IEEE International Conference on Blockchain (Blockchain)*. IEEE, 2019, pp. 536–540.
- [34] Bank for International Settlements, “Project polaris part 2: A security and resilience framework for cbdc systems,” BIS Innovation Hub, Tech. Rep., 2023.
- [35] —, “Project polaris part 1: A handbook for offline payments with cbdc,” BIS Innovation Hub, Tech. Rep., 2023.
- [36] G. Soana and T. de Arruda, “Central bank digital currencies and financial integrity: finding a new trade-off between privacy and traceability within a changing financial architecture,” *Journal of Banking Regulation*, vol. 25, pp. 467 – 486, 2024.
- [37] J. E. Said and J. Lánský, “Designing retail central bank digital currencies: A systematic literature review of trade-offs between security, privacy, and financial stability,” *International Journal of Financial Studies*, 2026.
- [38] M. Kakebayashi, G. P. Presto, T. Yuyama, and S. Matsuo, “Policy design of retail central bank digital currencies: Embedding aml/cft compliance,” *SSRN Electronic Journal*, 2023.
- [39] Y.-R. Wang, C.-Q. Ma, and Y.-S. Ren, “A model for cbdc audits based on blockchain technology: Learning from the dcep,” *Research in International Business and Finance*, vol. 63, p. 101781, 2022.
- [40] O. Atangana, L. Khoukhi, M. Barbier, and A. Kokcam, “Block-pad: A blockchain-enabled framework for resilient and flexible cbdc transactions leveraging digital identity,” *Computer Networks*, vol. 275, p. 111805, 2026.

- [41] J. E. Abang, H. Takruri, R. Al-Zaidi, and M. Al-Khalidi, "Latency performance modelling in hyperledger fabric blockchain: Challenges and directions with an iot perspective," *Internet of Things*, vol. 26, p. 101217, 2024.
- [42] A. Lohachab, S. Garg, B. H. Kang, and M. B. Amin, "Performance evaluation of hyperledger fabric-enabled framework for pervasive peer-to-peer energy trading in smart cyber-physical systems," *Future Generation Computer Systems*, vol. 118, pp. 392–416, 2021.
- [43] M. Ajwalia and P. Shah, "Performance comparison of permissioned and permissionless blockchain by varying workload transaction," *BenchCouncil Transactions on Benchmarks, Standards and Evaluations*, vol. 5, no. 4, p. 100251, 2025.
- [44] C. Beer, S. Zingg, K. Kostianen, K. Wüst, V. Capkun, and S. Čapkun, "Payoff: A regulated central bank digital currency with private offline payments," 2024. [Online]. Available: <https://arxiv.org/abs/2408.06956>
- [45] H. Arslanian, "Retail central bank digital currencies," in *The Book of Crypto*. Springer International Publishing, 2022, pp. 203–232.
- [46] M. L. Bech and R. Garratt, "Central bank cryptocurrencies," *BIS Quarterly Review*, pp. 55–70, Sep. 2017. [Online]. Available: https://www.bis.org/publ/qtrpdf/r_qt1709f.htm
- [47] Central Bank of Nigeria, "Design paper for the enaira," Central Bank of Nigeria, Tech. Rep., 2021. [Online]. Available: https://enaira.gov.ng/wp-content/uploads/2023/06/Design-Paper-for-Nigerias-CBDC-02_Oct-2021.pdf
- [48] People's Bank of China, "Progress of research and development of e-cny in china," People's Bank of China, Tech. Rep., 2021. [Online]. Available: <https://www.pbc.gov.cn/en/3688110/3688172/4157443/4293696/2021072014364791207.pdf>
- [49] W. Xiong, X. Yao, and X. Deng, "Research on the performance analysis of mainstream consensus algorithms of consortium blockchains," in *2024 2nd International Conference on Mobile Internet, Cloud Computing and Information Security (MICCIS)*. IEEE, 2024.
- [50] S. Das, M. Mishra, and R. Priyadarshini, "Hyperledger fabric: The enterprise-grade blockchain for modern business applications," in *Fostering Machine Learning and IoT for Blockchain Technology*. Springer Nature Singapore, 2025, pp. 163–188.
- [51] Hyperledger Performance and Scale Working Group, "Hyperledger blockchain performance metrics," Hyperledger Foundation, Tech. Rep., 2018.
- [52] Financial Action Task Force, "Guidance on digital identity," FATF, Paris, France, Tech. Rep., 2020. [Online]. Available: <https://www.fatf-gafi.org/content/dam/fatf-gafi/guidance/Guidance-on-Digital-Identity.pdf>
- [53] Bank Indonesia, "Peraturan bank indonesia nomor 10 tahun 2024 tentang penerapan anti pencucian uang, pencegahan pendanaan terorisme, dan pencegahan pendanaan proliferasi senjata pemusnah massal bagi pihak yang diatur dan diawasi

- oleh bank indonesia,” 2024. [Online]. Available: https://www.bi.go.id/id/publikasi/peraturan/Pages/PBI_102024.aspx
- [54] I. Agur, A. Ari, and G. Dell’Ariccia, “Designing central bank digital currencies,” *Journal of Monetary Economics*, vol. 125, pp. 62–79, 2022. [Online]. Available: <https://doi.org/10.1016/j.jmoneco.2021.05.002>
- [55] Bank Indonesia, “Peraturan anggota dewan gubernur nomor 24/7/padg/2022,” 2022, ketentuan penyelenggaraan uang elektronik; digunakan sebagai proksi kebijakan untuk parameter purwarupa, bukan sebagai dasar hukum limit CBDC. [Online]. Available: https://www.bi.go.id/id/publikasi/peraturan/Pages/PADG_240722.aspx
- [56] C. Saraf and S. Sabadra, “Blockchain platforms: A compendium,” in *2018 IEEE International Conference on Innovative Research and Development (ICIRD)*. IEEE, 2018, pp. 1–6.
- [57] Central Bank of The Bahamas, “Project sand dollar: A digital fiat currency for the bahamas,” Central Bank of The Bahamas, Nassau, Tech. Rep., 2020. [Online]. Available: <https://www.centralbankbahamas.com/viewPDF/documents/2019-12-25-02-18-11-Project-Sanddollar.pdf>
- [58] Sveriges Riksbank, “E-krona pilot — phase 1 and phase 2 technical reports,” Sveriges Riksbank, Stockholm, Tech. Rep., 2021. [Online]. Available: <https://www.riksbank.se/en-gb/payments--cash/e-krona/technical-solution-for-the-e-krona-pilot/>
- [59] I. Systems. (2024) Nosql databases in financial services: Modernizing banking and insurance data management. [Online]. Available: <https://www.ispirer.com/blog/nosql-databases-financial-services>
- [60] PostgreSQL Global Development Group. (2025) Postgresql 18 documentation: Json types, row security policies, and mvcc concurrency control. PostgreSQL Global Development Group. [Online]. Available: <https://www.postgresql.org/docs/current/>
- [61] LogicMonitor. (2025) Postgresql vs. mysql: A comprehensive comparison. [Online]. Available: <https://www.logicmonitor.com/blog/postgresql-vs-mysql>
- [62] Hyperledger Foundation. (2025) Hyperledger fabric documentation: Private data collections. Linux Foundation. [Online]. Available: <https://hyperledger-fabric.readthedocs.io/en/latest/private-data/private-data.html>

LAMPIRAN

Lampiran ini memuat kode sumber inti sistem Rupiah Digital ritel yang dibangun pada penelitian ini, meliputi fungsi-fungsi utama *chaincode* *digital-rupiah* serta modul konfigurasi dan *workload* pengujian Hyperledger Caliper. Konfigurasi jaringan Hyperledger Fabric, skrip otomasi jaringan, dan lapisan *backend* REST API tidak ditampilkan karena merupakan *boilerplate* Fabric yang bersifat standar dan tidak menjadi kontribusi utama penelitian. Kode sumber lengkap tersedia pada repositori sistem.

L.1 Source Code Tipe Data dan Struktur *Chaincode*

Bagian ini memuat kode sumber definisi paket, impor, serta tipe data dan struktur utama (berkas `chaincode/digital_rupiah.go`).

```
1 package main
2
3 import (
4     "encoding/json"
5     "fmt"
6     "time"
7
8     "github.com/hyperledger/fabric-contract-api-go/v2/contractapi"
9 )
10
11 type SmartContract struct {
12     contractapi.Contract
13 }
14
15 // --- Enums -----
16
17 type WalletTier string
18
19 const (
20     TierBasic      WalletTier = "BASIC"
21     TierStandard   WalletTier = "STANDARD"
22     TierMerchant    WalletTier = "MERCHANT"
23 )
24
25 type WalletType string
26
27 const (
28     WalletHot   WalletType = "hot"
29     WalletCold  WalletType = "cold"
30 )
31
32 type ParticipantType string
33
34 const (
35     ParticipantValidator ParticipantType = "validator"
36     ParticipantObserver  ParticipantType = "observer"
37     ParticipantPJP        ParticipantType = "pjp"
38 )
39
40 type ParticipantStatus string
41
42 const (
43     ParticipantPending ParticipantStatus = "pending"
44     ParticipantActive   ParticipantStatus = "active"
45     ParticipantFrozen   ParticipantStatus = "frozen"
46 )
47
48 type KycStatus string
49
50 const (
51     KycPending    KycStatus = "pending"
52     KycInReview   KycStatus = "in_review"
53     KycApproved   KycStatus = "approved"
54     KycRejected   KycStatus = "rejected"
```

```

55     KycExpired    KycStatus = "expired"
56     KycSuspended KycStatus = "suspended"
57 )
58
59 type KycRiskLevel string
60
61 const (
62     RiskLow      KycRiskLevel = "low"
63     RiskMedium   KycRiskLevel = "medium"
64     RiskHigh      KycRiskLevel = "high"
65     RiskProhibited KycRiskLevel = "prohibited"
66 )
67
68 type KycSubjectType string
69
70 const (
71     SubjectParticipant    KycSubjectType = "participant"
72     SubjectRetailCustomer KycSubjectType = "retail_customer"
73     SubjectMerchant       KycSubjectType = "merchant"
74 )
75
76 type DueDiligenceLevel string
77
78 const (
79     DueDiligenceSimplified DueDiligenceLevel = "simplified"
80     DueDiligenceStandard   DueDiligenceLevel = "standard"
81     DueDiligenceEnhanced   DueDiligenceLevel = "enhanced"
82 )
83
84 type LimitScope string
85
86 const (
87     ScopeGlobalSupply      LimitScope = "global_supply"
88     ScopePerParticipantBalance LimitScope = "per_participant_balance"
89     ScopePerTxAmount        LimitScope = "per_tx_amount"
90     ScopeMinParticipantBalance LimitScope = "min_participant_balance"
91 )
92
93 type TransactionType string
94
95 const (
96     TxMint      TransactionType = "issuance"
97     TxBurn      TransactionType = "redemption"
98     TxTransfer  TransactionType = "transfer"
99 )
100
101 type TransactionStatus string
102
103 const (
104     TxSettled TransactionStatus = "settled"
105     TxRejected TransactionStatus = "rejected"
106 )
107
108 // --- State Objects -----
109
110 type Participant struct {
111     ParticipantID string    `json:"participant_id"`
112     Name           string    `json:"name"`
113     Domain         string    `json:"domain"`

```

```

114 AccountID          string          `json:"account_id"`
115 ParticipantType    ParticipantType `json:"participant_type"`
116 InitialReserveBalance string        `json:"initial_reserve_balance"`
117 ComplianceStatus   string        `json:"compliance_status"`
118 Status             ParticipantStatus `json:"status"`
119 CreatedAt          string        `json:"created_at"`
120 UpdatedAt          string        `json:"updated_at"`
121 }
122
123 type Wallet struct {
124     WalletID          string          `json:"wallet_id"`
125     OwnerID           string          `json:"owner_id"`
126     ParticipantID     string          `json:"participant_id"`
127     Tier              WalletTier      `json:"tier"`
128     WalletType        WalletType      `json:"wallet_type"`
129     Balance           int64           `json:"balance"`
130     Frozen            bool            `json:"frozen"`
131     DailySpent        int64           `json:"daily_spent"`
132     MonthlySpent      int64           `json:"monthly_spent"`
133     MonthlyReceived   int64           `json:"monthly_received"`
134     LastResetDay      string          `json:"last_reset_day"`
135     LastResetMonth    string          `json:"last_reset_month"`
136     CreatedAt         string          `json:"created_at"`
137     UpdatedAt         string          `json:"updated_at"`
138 }
139
140 type OldWallet struct {
141     OwnerID           string          `json:"owner_id"`
142     Tier              WalletTier      `json:"tier"`
143     Balance           int64           `json:"balance"`
144     Frozen            bool            `json:"frozen"`
145     DailySpent        int64           `json:"daily_spent"`
146     MonthlySpent      int64           `json:"monthly_spent"`
147     LastResetDay      string          `json:"last_reset_day"`
148     CreatedAt         string          `json:"created_at"`
149     UpdatedAt         string          `json:"updated_at"`
150 }
151
152 type TierLimit struct {
153     Tier              WalletTier      `json:"tier"`
154     MaxBalance        int64           `json:"max_balance"`
155     MinBalance        int64           `json:"min_balance"`
156     DailyTxLimit      int64           `json:"daily_tx_limit"`
157     MonthlyTxLimit    int64           `json:"monthly_tx_limit"`
158     MonthlyIncomingLimit int64           `json:"monthly_incoming_limit"`
159     PerTxLimit        int64           `json:"per_tx_limit"`
160 }
161
162 type KycProfile struct {
163     ProfileID         string          `json:"profile_id"`
164     SubjectType       KycSubjectType `json:"subject_type"`
165     SubjectID         string          `json:"subject_id"`
166     DocumentHashes    []string        `json:"document_hashes"`
167     Status            KycStatus       `json:"status"`
168     RiskLevel         KycRiskLevel    `json:"risk_level"`
169     DueDiligenceLevel DueDiligenceLevel `json:"due_diligence_level"`
170     SeniorApproval    bool            `json:"senior_approval"`
171     ExpiresAt         string          `json:"expires_at"`
172     CreatedAt         string          `json:"created_at"`

```

```

173 UpdatedAt      string          `json:"updated_at"`
174 }
175
176 type KycProviderCheck struct {
177     CheckID      string          `json:"check_id"`
178     ProfileID    string          `json:"profile_id"`
179     Status       KycStatus      `json:"status"`
180     RiskLevel    KycRiskLevel   `json:"risk_level"`
181     Checks       map[string]string `json:"checks"`
182     DocumentHashes []string        `json:"document_hashes"`
183     ExpiresAt    string          `json:"expires_at"`
184     Timestamp    string          `json:"timestamp"`
185 }
186
187 type KycAuditEvent struct {
188     EventID      string `json:"event_id"`
189     ProfileID    string `json:"profile_id"`
190     EventType    string `json:"event_type"`
191     OldStatus    string `json:"old_status"`
192     NewStatus    string `json:"new_status"`
193     Description  string `json:"description"`
194     Timestamp    string `json:"timestamp"`
195 }
196
197 type RetailCustomer struct {
198     CustomerID    string `json:"customer_id"`
199     IdentityHash  string `json:"identity_hash"`
200     WalletAccountID string `json:"wallet_account_id"`
201     KycProfileID  string `json:"kyc_profile_id"`
202     CreatedAt     string `json:"created_at"`
203     UpdatedAt     string `json:"updated_at"`
204 }
205
206 type SystemLimit struct {
207     Scope LimitScope `json:"scope"`
208     Value int64          `json:"value"`
209 }
210
211 type AutoLimitPolicy struct {
212     ParticipantID string `json:"participant_id"`
213     AutoRedemption bool  `json:"auto_redemption"`
214     MinBalance     int64 `json:"min_balance"`
215     UpdatedAt      string `json:"updated_at"`
216 }
217
218 type TransactionRecord struct {
219     TxID          string          `json:"tx_id"`
220     TransactionType TransactionType `json:"transaction_type"`
221     SenderID      string          `json:"sender_id"`
222     ReceiverID    string          `json:"receiver_id"`
223     Amount        int64          `json:"amount"`
224     Status        TransactionStatus `json:"status"`
225     RelatedIntentID string        `json:"related_intent_id"`
226     Timestamp     string          `json:"timestamp"`
227 }
228
229 type SupervisionEvent struct {
230     EventID      string `json:"event_id"`
231     EventType    string `json:"event_type"`

```

```

232 EntityType string `json:"entity_type"`
233 EntityID   string `json:"entity_id"`
234 Data       string `json:"data"`
235 Timestamp  string `json:"timestamp"`
236 }
237
238 type ReconciliationReport struct {
239     ReportID      string `json:"report_id"`
240     TotalSupply   int64  `json:"total_supply"`
241     TotalIssued   int64  `json:"total_issued"`
242     TotalBurned   int64  `json:"total_burned"`
243     Wallets       int    `json:"wallets"`
244     Participants  int    `json:"participants"`
245     GeneratedAt   string `json:"generated_at"`
246 }
247
248 type Metrics struct {
249     TotalTransactions int64 `json:"total_transactions"`
250     SettledCount      int64 `json:"settled_count"`
251     RejectedCount     int64 `json:"rejected_count"`
252     ActiveWallets     int64 `json:"active_wallets"`
253     ActiveParticipants int64 `json:"active_participants"`
254     TotalSupply       int64 `json:"total_supply"`
255 }
256
257 // --- InitLedger -----
258
259 // txNow returns the transaction's timestamp as a UTC time. Using the proposal
260 // timestamp (identical across all endorsing peers) instead of time.Now() keeps the
261 // chaincode deterministic, so multi-org endorsement read/write sets always match.
262 func txNow(ctx contractapi.TransactionContextInterface) time.Time {
263     ts, err := ctx.GetStub().GetTxTimestamp()
264     if err != nil || ts == nil {
265         return time.Unix(0, 0).UTC()
266     }
267     return ts.AsTime().UTC()
268 }

```

L.2 Source Code Inisialisasi *Ledger*

Bagian ini memuat kode sumber fungsi `InitLedger` yang menanamkan batas tingkatan dan batas sistem (berkas `chaincode/digital_rupiah.go`).

```

1 func (s *SmartContract) InitLedger(ctx contractapi.TransactionContextInterface) error {
2     limits := []TierLimit{
3         {Tier: TierBasic, MaxBalance: 1_000_000, MinBalance: 0, DailyTxLimit: 500_000,
4           MonthlyTxLimit: 5_000_000, PerTxLimit: 250_000},
5         {Tier: TierStandard, MaxBalance: 50_000_000, MinBalance: 100_000, DailyTxLimit: 10
6           _000_000, MonthlyTxLimit: 100_000_000, PerTxLimit: 2_500_000},
7         {Tier: TierMerchant, MaxBalance: 200_000_000, MinBalance: 500_000, DailyTxLimit: 50
8           _000_000, MonthlyTxLimit: 500_000_000, PerTxLimit: 10_000_000},
9     }
10    for _, limit := range limits {
11        if err := s.putLimit(ctx, limit); err != nil {
12            return err
13        }
14    }
15 }

```



```

13  now := txNow(ctx)
14
15  biWallet := Wallet{
16      OwnerID:      "bank_indonesia",
17      ParticipantID: "bank_indonesia",
18      Tier:         TierMerchant,
19      WalletType:    WalletHot,
20      Balance:       0,
21      LastResetDay:  now.Format("2006-01-02"),
22      CreatedAt:     now.Format(time.RFC3339),
23      UpdatedAt:     now.Format(time.RFC3339),
24  }
25  if err := s.putWallet(ctx, "bi_treasury", biWallet); err != nil {
26      return err
27  }
28
29  sysLimits := []SystemLimit{
30      {Scope: ScopeGlobalSupply, Value: 10_000_000_000_000},
31      {Scope: ScopePerParticipantBalance, Value: 1_000_000_000_000},
32      {Scope: ScopePerTxAmount, Value: 100_000_000_000},
33  }
34  for _, sl := range sysLimits {
35      if err := s.putSystemLimit(ctx, sl); err != nil {
36          return err
37      }
38  }
39
40  return nil
41 }

```

L.3 Source Code Penerbitan dan Distribusi Likuiditas

Bagian ini memuat kode sumber fungsi Mint, Burn, RequestIssuance, dan DistributeToParticipant (berkas chaincode/digital_rupiah.go).

```

1  func (s *SmartContract) Mint(ctx contractapi.TransactionContextInterface, walletID
    string, amount int64) error {
2      if amount <= 0 {
3          return fmt.Errorf("amount must be positive")
4      }
5      wallet, err := s.getWallet(ctx, walletID)
6      if err != nil {
7          return err
8      }
9      if wallet.Frozen {
10         return fmt.Errorf("wallet %s is frozen", walletID)
11     }
12     limit, err := s.getLimit(ctx, wallet.Tier)
13     if err != nil {
14         return err
15     }
16     newBalance := wallet.Balance + amount
17     if newBalance > limit.MaxBalance {
18         return fmt.Errorf("mint would exceed max balance %d for tier %s", limit.MaxBalance,
            wallet.Tier)
19     }
20     wallet.Balance = newBalance
21     wallet.UpdatedAt = txNow(ctx).Format(time.RFC3339)

```

```

22     if err := s.putWallet(ctx, walletID, wallet); err != nil {
23         return err
24     }
25     _ = s.emitAudit(ctx, "MINT", walletID, "", amount)
26     return s.recordTransaction(ctx, TxMint, walletID, "", amount, TxSettled, "")
27 }
28
29 func (s *SmartContract) Burn(ctx contractapi.TransactionContextInterface, walletID
    string, amount int64) error {
30     if amount <= 0 {
31         return fmt.Errorf("amount must be positive")
32     }
33     wallet, err := s.getWallet(ctx, walletID)
34     if err != nil {
35         return err
36     }
37     if wallet.Frozen {
38         return fmt.Errorf("wallet %s is frozen", walletID)
39     }
40     if wallet.Balance < amount {
41         return fmt.Errorf("insufficient balance: have %d, need %d", wallet.Balance, amount)
42     }
43     wallet.Balance -= amount
44     wallet.UpdatedAt = txNow(ctx).Format(time.RFC3339)
45     if err := s.putWallet(ctx, walletID, wallet); err != nil {
46         return err
47     }
48     _ = s.emitAudit(ctx, "BURN", walletID, "", amount)
49     return s.recordTransaction(ctx, TxBurn, walletID, "", amount, TxSettled, "")
50 }
51
52 func (s *SmartContract) RequestIssuance(ctx contractapi.TransactionContextInterface,
    participantID string, amount int64) (interface{}, error) {
53     p, err := s.getParticipant(ctx, participantID)
54     if err != nil {
55         return nil, err
56     }
57     if p.ParticipantType == ParticipantPJP {
58         return nil, fmt.Errorf("PJP participants cannot request direct issuance; receive
    liquidity via DistributeToParticipant from a bank validator")
59     }
60     if p.Status != "active" {
61         return nil, fmt.Errorf("participant %s is not active", participantID)
62     }
63     walletID := fmt.Sprintf("wlt_%s", participantID)
64     return nil, s.Mint(ctx, walletID, amount)
65 }
66
67 func (s *SmartContract) DistributeToParticipant(ctx contractapi.
    TransactionContextInterface, senderParticipantID string, receiverParticipantID
    string, amount int64) error {
68     if amount <= 0 {
69         return fmt.Errorf("amount must be positive")
70     }
71     sender, err := s.getParticipant(ctx, senderParticipantID)
72     if err != nil {
73         return fmt.Errorf("sender participant: %w", err)
74     }
75     if sender.ParticipantType != ParticipantValidator {

```

```

76     return fmt.Errorf("sender %s must be a validator (bank); got %s",
77         senderParticipantID, sender.ParticipantType)
78 }
79 if sender.Status != ParticipantActive {
80     return fmt.Errorf("sender participant %s is not active", senderParticipantID)
81 }
82 receiver, err := s.getParticipant(ctx, receiverParticipantID)
83 if err != nil {
84     return fmt.Errorf("receiver participant: %w", err)
85 }
86 if receiver.ParticipantType != ParticipantPJP {
87     return fmt.Errorf("receiver %s must be a PJP; got %s", receiverParticipantID,
88         receiver.ParticipantType)
89 }
90 if receiver.Status != ParticipantActive {
91     return fmt.Errorf("receiver participant %s is not active", receiverParticipantID)
92 }
93 senderWalletID := fmt.Sprintf("wlt_%s", senderParticipantID)
94 receiverWalletID := fmt.Sprintf("wlt_%s", receiverParticipantID)
95 if err := s.Burn(ctx, senderWalletID, amount); err != nil {
96     return fmt.Errorf("burn sender: %w", err)
97 }
98 if err := s.Mint(ctx, receiverWalletID, amount); err != nil {
99     return fmt.Errorf("mint receiver: %w", err)
100 }
101 _ = s.emitSupervisionEvent(ctx, "pjp_distribution", "participant",
102     receiverParticipantID, map[string]interface{}{
103         "sender": senderParticipantID,
104         "receiver": receiverParticipantID,
105         "amount": amount,
106     })
107 return nil
108 }

```

L.4 Source Code Transfer dan Penegakan Limit

Bagian ini memuat kode sumber fungsi Transfer dengan penegakan limit per transaksi, harian, dan bulanan (berkas chaincode/digital_rupiah.go).

```

1 func (s *SmartContract) Transfer(ctx contractapi.TransactionContextInterface, senderID
2     string, receiverID string, amount int64) error {
3     if amount <= 0 {
4         return fmt.Errorf("amount must be positive")
5     }
6     if senderID == receiverID {
7         return fmt.Errorf("sender and receiver must differ")
8     }
9     sender, err := s.getWallet(ctx, senderID)
10    if err != nil {
11        return err
12    }
13    if sender.Frozen {
14        return fmt.Errorf("sender wallet %s is frozen", senderID)
15    }
16    receiver, err := s.getWallet(ctx, receiverID)
17    if err != nil {
18        return err
19    }

```

```

19     if receiver.Frozen {
20         return fmt.Errorf("receiver wallet %s is frozen", receiverID)
21     }
22     senderLimit, err := s.getLimit(ctx, sender.Tier)
23     if err != nil {
24         return err
25     }
26     if amount > senderLimit.PerTxLimit {
27         return fmt.Errorf("per-transaction limit exceeded: %d for tier %s", senderLimit.
            PerTxLimit, sender.Tier)
28     }
29     today := txNow(ctx).Format("2006-01-02")
30     s.resetSpendingIfNewDay(&sender, today)
31     newDaily := sender.DailySpent + amount
32     if newDaily > senderLimit.DailyTxLimit {
33         return fmt.Errorf("daily transaction limit exceeded: %d for tier %s", senderLimit.
            DailyTxLimit, sender.Tier)
34     }
35     newMonthly := sender.MonthlySpent + amount
36     if newMonthly > senderLimit.MonthlyTxLimit {
37         return fmt.Errorf("monthly transaction limit exceeded: %d for tier %s", senderLimit.
            MonthlyTxLimit, sender.Tier)
38     }
39     if sender.Balance < amount {
40         return fmt.Errorf("insufficient balance: have %d, need %d", sender.Balance, amount)
41     }
42     receiverLimit, err := s.getLimit(ctx, receiver.Tier)
43     if err != nil {
44         return err
45     }
46     if receiver.Balance+amount > receiverLimit.MaxBalance {
47         return fmt.Errorf("transfer would exceed receiver max balance %d for tier %s",
            receiverLimit.MaxBalance, receiver.Tier)
48     }
49     sender.Balance -= amount
50     sender.DailySpent = newDaily
51     sender.MonthlySpent = newMonthly
52     sender.UpdatedAt = txNow(ctx).Format(time.RFC3339)
53     receiver.Balance += amount
54     receiver.UpdatedAt = txNow(ctx).Format(time.RFC3339)
55     if err := s.putWallet(ctx, senderID, sender); err != nil {
56         return err
57     }
58     if err := s.putWallet(ctx, receiverID, receiver); err != nil {
59         return err
60     }
61     if senderLimit.MinBalance > 0 && sender.Balance < senderLimit.MinBalance {
62         policy, err := s.getAutoLimitPolicy(ctx, sender.ParticipantID)
63         if err == nil && policy.AutoRedemption && sender.Balance > 0 {
64             if arErr := s.autoRedeem(ctx, senderID, sender.Balance); arErr != nil {
65                 _ = s.emitAudit(ctx, "AUTO_REDEMPTION_FAILED", senderID, "", sender.Balance)
66             }
67         }
68     }
69     _ = s.emitAudit(ctx, "TRANSFER", senderID, receiverID, amount)
70     return s.recordTransaction(ctx, TxTransfer, senderID, receiverID, amount, TxSettled, "
    ")
71 }

```

L.5 Source Code Kepatuhan KYC

Bagian ini memuat kode sumber fungsi SubmitKycProfile dan RefreshKycProfile (berkas chaincode/digital_rupiah.go).

```
1 func (s *SmartContract) SubmitKycProfile(ctx contractapi.TransactionContextInterface,
   profileID string, subjectType string, subjectID string, providerCaseID string,
   documentHashesJSON string) error {
2   exists, err := s.kycProfileExists(ctx, profileID)
3   if err != nil {
4       return err
5   }
6   if exists {
7       return fmt.Errorf("KYC profile %s already exists", profileID)
8   }
9   var hashes []string
10  if err := json.Unmarshal([]byte(documentHashesJSON), &hashes); err != nil {
11      return fmt.Errorf("invalid document_hashes: %w", err)
12  }
13  now := txNow(ctx)
14  profile := KycProfile{
15      ProfileID:      profileID,
16      SubjectType:    KycSubjectType(subjectType),
17      SubjectID:      subjectID,
18      ProviderCaseID: providerCaseID,
19      DocumentHashes: hashes,
20      Status:         KycPending,
21      RiskLevel:      RiskLow,
22      KycChecks:      map[string]string{},
23      CreatedAt:      now.Format(time.RFC3339),
24      UpdatedAt:      now.Format(time.RFC3339),
25  }
26  if err := s.putKycProfile(ctx, profile); err != nil {
27      return err
28  }
29  return s.emitKycAudit(ctx, profileID, "profile_created", "", "pending", "KYC profile
   submitted")
30 }
31
32 func (s *SmartContract) RefreshKycProfile(ctx contractapi.TransactionContextInterface,
   profileID string, status string, riskLevel string, checksJSON string,
   documentHashesJSON string, rejectionReason string, expiresAt string) error {
33  profile, err := s.getKycProfile(ctx, profileID)
34  if err != nil {
35      return err
36  }
37  oldStatus := string(profile.Status)
38  var checks map[string]string
39  if checksJSON != "" {
40      if err := json.Unmarshal([]byte(checksJSON), &checks); err != nil {
41          return fmt.Errorf("invalid checks: %w", err)
42      }
43  }
44  var hashes []string
45  if documentHashesJSON != "" {
46      if err := json.Unmarshal([]byte(documentHashesJSON), &hashes); err != nil {
47          return fmt.Errorf("invalid document_hashes: %w", err)
48      }
49  }
```

```

50 now := txNow(ctx)
51 profile.Status = KycStatus(status)
52 profile.RiskLevel = KycRiskLevel(riskLevel)
53 if checks != nil {
54     profile.KycChecks = checks
55 }
56 if len(hashes) > 0 {
57     profile.DocumentHashes = hashes
58 }
59 profile.RejectionReason = rejectionReason
60 profile.ExpiresAt = expiresAt
61 profile.UpdatedAt = now.Format(time.RFC3339)
62 if err := s.putKycProfile(ctx, profile); err != nil {
63     return err
64 }
65 checkID := fmt.Sprintf("chk_%s_%s", profileID, now.Format("20060102150405"))
66 pc := KycProviderCheck{
67     CheckID:      checkID,
68     ProfileID:    profileID,
69     Status:       KycStatus(status),
70     RiskLevel:    KycRiskLevel(riskLevel),
71     Checks:       checks,
72     DocumentHashes: hashes,
73     RejectionReason: rejectionReason,
74     ExpiresAt:    expiresAt,
75     Timestamp:    now.Format(time.RFC3339),
76 }
77 if err := s.putKycProviderCheck(ctx, pc); err != nil {
78     return err
79 }
80 return s.emitKycAudit(ctx, profileID, "profile_refreshed", oldStatus, status, fmt.
    Sprintf("Provider decision: %s risk=%s", status, riskLevel))
81 }

```

L.6 Source Code Pengawasan dan Rekonsiliasi

Bagian ini memuat kode sumber fungsi laporan rekonsiliasi, metrik, dan emisi *event* pengawasan (berkas `chaincode/digital_rupiah.go`).

```

1 func (s *SmartContract) GetReconciliationReport(ctx contractapi.
    TransactionContextInterface) (*ReconciliationReport, error) {
2     wallets, err := s.GetAllWallets(ctx)
3     if err != nil {
4         return nil, err
5     }
6     participants, err := s.getAllParticipants(ctx)
7     if err != nil {
8         participants = nil
9     }
10    var totalSupply int64
11    for _, w := range wallets {
12        totalSupply += w.Balance
13    }
14    report := ReconciliationReport{
15        ReportID:      fmt.Sprintf("rec_%s", txNow(ctx).Format("20060102150405")),
16        TotalSupply:    totalSupply,
17        TotalIssued:    0,
18        TotalBurned:    0,

```

```

19     Wallets:      len(wallets),
20     Participants: len(participants),
21     GeneratedAt:  txNow(ctx).Format(time.RFC3339),
22 }
23 txIter, err := ctx.GetStub().GetStateByPartialCompositeKey("tx", []string{})
24 if err == nil {
25     defer txIter.Close()
26     for txIter.HasNext() {
27         kv, err := txIter.Next()
28         if err != nil {
29             continue
30         }
31         var tr TransactionRecord
32         if err := json.Unmarshal(kv.Value, &tr); err != nil {
33             continue
34         }
35         switch tr.TransactionType {
36         case TxMint:
37             report.TotalIssued += tr.Amount
38         case TxBurn:
39             report.TotalBurned += tr.Amount
40         }
41     }
42 }
43 return &report, nil
44 }
45
46 func (s *SmartContract) GetMetrics(ctx contractapi.TransactionContextInterface) (*
47     Metrics, error) {
48     wallets, err := s.GetAllWallets(ctx)
49     if err != nil {
50         return nil, err
51     }
52     participants, err := s.getAllParticipants(ctx)
53     if err != nil {
54         participants = nil
55     }
56     var totalSupply int64
57     activeWallets := int64(0)
58     for _, w := range wallets {
59         totalSupply += w.Balance
60         if !w.Frozen {
61             activeWallets++
62         }
63     }
64     activeParticipants := int64(0)
65     for _, p := range participants {
66         if p.Status == ParticipantActive {
67             activeParticipants++
68         }
69     }
70     m := Metrics{
71         ActiveWallets:      activeWallets,
72         ActiveParticipants: activeParticipants,
73         TotalSupply:         totalSupply,
74     }
75     txIter, err := ctx.GetStub().GetStateByPartialCompositeKey("tx", []string{})
76     if err == nil {
77         defer txIter.Close()

```

```

77     for txIter.HasNext() {
78         kv, err := txIter.Next()
79         if err != nil {
80             continue
81         }
82         var tr TransactionRecord
83         if err := json.Unmarshal(kv.Value, &tr); err != nil {
84             continue
85         }
86         m.TotalTransactions++
87         switch tr.Status {
88             case TxSettled:
89                 m.SettledCount++
90             case TxRejected:
91                 m.RejectedCount++
92         }
93     }
94 }
95 return &m, nil
96 }
97
98 func (s *SmartContract) emitSupervisionEvent(ctx contractapi.TransactionContextInterface
99     , eventType string, entityType string, entityID string, data interface{}) error {
100     now := txNow(ctx)
101     eventID := fmt.Sprintf("sup_%s_%s", entityID, now.Format("20060102150405"))
102     var dataStr string
103     if data != nil {
104         b, err := json.Marshal(data)
105         if err != nil {
106             return err
107         }
108         dataStr = string(b)
109     }
110     ev := SupervisionEvent{
111         EventID:    eventID,
112         EventType:  eventType,
113         EntityType: entityType,
114         EntityID:   entityID,
115         Data:       dataStr,
116         Timestamp:  now.Format(time.RFC3339),
117     }
118     key, err := ctx.GetStub().CreateCompositeKey("supervision", []string{entityID, eventID})
119     if err != nil {
120         return err
121     }
122     evData, err := json.Marshal(ev)
123     if err != nil {
124         return err
125     }
126     return ctx.GetStub().PutState(key, evData)
127 }

```

L.7 Source Code Konfigurasi Jaringan Caliper

Bagian ini memuat kode sumber konfigurasi koneksi Caliper ke jaringan Fabric (berkas benchmark/networkconfig.yaml).


```

1 ---
2 name: PayNet Garuda Network
3 version: "2.0.0"
4 caliper:
5   blockchain: fabric
6
7 channels:
8   - channelName: mychannel
9     contracts:
10      - id: digital-rupiah
11
12 organizations:
13   - mspid: BankIndonesiaOrgMSP
14     identities:
15       certificates:
16         - name: User1
17           clientSignedCert:
18             path: network/organizations/peerOrganizations/bi.paynet/users/User1@bi.
19               paynet/msp/signcerts/cert.pem
20             clientPrivateKey:
21               path: network/organizations/peerOrganizations/bi.paynet/users/User1@bi.
22               paynet/msp/keystore/key.pem
23             connectionProfile:
24               path: network/organizations/peerOrganizations/bi.paynet/connection-bi.json
25               discover: true
26
27   - mspid: HimbaraBankOrgMSP
28     identities:
29       certificates:
30         - name: User1
31           clientSignedCert:
32             path: network/organizations/peerOrganizations/himbara.paynet/users/
33               User1@himbara.paynet/msp/signcerts/cert.pem
34             clientPrivateKey:
35               path: network/organizations/peerOrganizations/himbara.paynet/users/
36               User1@himbara.paynet/msp/keystore/key.pem
37             connectionProfile:
38               path: network/organizations/peerOrganizations/himbara.paynet/connection-himbara.
39               json
40               discover: true
41
42   - mspid: CommercialBankOrgMSP
43     identities:
44       certificates:
45         - name: User1
46           clientSignedCert:
47             path: network/organizations/peerOrganizations/commercial.paynet/users/
48               User1@commercial.paynet/msp/signcerts/cert.pem
49             clientPrivateKey:
50               path: network/organizations/peerOrganizations/commercial.paynet/users/
51               User1@commercial.paynet/msp/keystore/key.pem
52             connectionProfile:
53               path: network/organizations/peerOrganizations/commercial.paynet/connection-
54               commercial.json
55               discover: true

```

L.8 Source Code Konfigurasi *Benchmark Caliper*

Bagian ini memuat kode sumber definisi ronde pengujian, laju, dan durasi (berkas `benchmark/benchconfig.yaml`).

```
1 test:
2   name: Digital Rupiah Retail CBDC Benchmark
3   description: Performance evaluation of direct retail Digital Rupiah transfers on
4     Hyperledger Fabric
5   workers:
6     type: local
7     number: 2
8   rounds:
9     - label: retail-warmup
10      txNumber: 40
11      rateControl:
12        type: fixed-rate
13        opts:
14          tps: 10
15      workload:
16        module: benchmark/workloads/retail-transfer.js
17        arguments:
18          scenario: transfer-default
19          customers: 80
20          merchants: 10
21          transactionSlots: 20
22          fundedRatio: 1
23    - label: retail-sustained
24      txNumber: 40
25      rateControl:
26        type: fixed-rate
27        opts:
28          tps: 30
29      workload:
30        module: benchmark/workloads/retail-transfer.js
31        arguments:
32          scenario: transfer-default
33          customers: 80
34          merchants: 10
35          transactionSlots: 20
36          fundedRatio: 1
37    - label: retail-peak
38      txNumber: 40
39      rateControl:
40        type: fixed-rate
41        opts:
42          tps: 60
43      workload:
44        module: benchmark/workloads/retail-transfer.js
45        arguments:
46          scenario: transfer-default
47          customers: 80
48          merchants: 10
49          transactionSlots: 20
50          fundedRatio: 1
51  observer:
```

```

54 type: local
55 interval: 1
56
57 monitor:
58   type: docker
59   docker:
60     name:
61       - peer0.bi.paynet
62       - peer0.himbara.paynet
63       - peer0.commercial.paynet
64       - peer0.pjp.paynet
65       - orderer.paynet
66   interval: 1

```

L.9 Source Code Skrip Pembangkit Aset Jaringan Caliper

Bagian ini memuat kode sumber skrip penyiapan aset jaringan untuk Caliper (berkas benchmark/gen-network-assets.sh).

```

1  #!/bin/bash
2  # Generates Caliper connection profiles + stable identity files for the garuda network.
3  # Idempotent; safe to re-run after the network is up. Run from repo root.
4  set -euo pipefail
5
6  SCRIPT_DIR="$(cd "$(dirname "$0")" && pwd)"
7  PROJECT_DIR="$(dirname "$SCRIPT_DIR)"
8  ORG_ROOT="$PROJECT_DIR/network/organizations/peerOrganizations"
9
10 # org | OrgName | MSPID | peerPort
11 ORGS=(
12   "bi|BankIndonesiaOrg|BankIndonesiaOrgMSP|7051"
13   "himbara|HimbaraBankOrg|HimbaraBankOrgMSP|9051"
14   "commercial|CommercialBankOrg|CommercialBankOrgMSP|11051"
15 )
16
17 for entry in "${ORGS[@]}; do
18   IFS='|' read -r ORG ORGNAME MSPID PORT <<< "$entry"
19   DOMAIN="$ORG.paynet"
20   ORG_DIR="$ORG_ROOT/$DOMAIN"
21   PEER="peer0.$DOMAIN"
22   TLS_CA="$ORG_DIR/peers/$PEER/tls/ca.crt"
23   USER_MSP="$ORG_DIR/users/User1@$DOMAIN/msp"
24
25   # Stable identity filenames (cryptogen emits hashed keystore names).
26   cp "$USER_MSP/signcerts/User1@$DOMAIN-cert.pem" "$USER_MSP/signcerts/cert.pem"
27   cp "$USER_MSP/keystore/*_sk" "$USER_MSP/keystore/key.pem"
28
29   cat > "$ORG_DIR/connection-$ORG.json" <<EOF
30 {
31   "name": "$ORG-paynet",
32   "version": "1.0.0",
33   "client": { "organization": "$ORGNAME" },
34   "channels": {
35     "mychannel": { "peers": { "$PEER": {} } }
36   },
37   "organizations": {
38     "$ORGNAME": { "mspid": "$MSPID", "peers": ["$PEER"] }
39   },

```

```

40  "peers": {
41    "$PEER": {
42      "url": "grpc://localhost:$PORT",
43      "tlsCACerts": { "path": "$TLS_CA" },
44      "grpcOptions": {
45        "ssl-target-name-override": "$PEER",
46        "hostnameOverride": "$PEER"
47      }
48    }
49  }
50 }
51 EOF
52 echo "Generated connection-$ORG.json + stable identity for $MSPID"
53 done

```

L.10 Source Code *Workload* Transfer Ritel

Bagian ini memuat kode sumber modul beban transfer antarpelanggan, transfer pelanggan-ke-pedagang, dan pembacaan dompet (berkas `benchmark/workloads/retail-transfer.js`).

```

1  'use strict';
2
3  const { RetailWorkloadBase } = require('./retail-base');
4
5  /**
6   * Transfer-only retail CBDC workload:
7   *   70% customer-to-customer transfer
8   *   25% customer-to-merchant transfer
9   *   5% wallet read
10  *
11  * Customer population is 20% BASIC and 80% STANDARD. Merchant wallets are
12  * policy-derived MERCHANT wallets. All subjects receive approved KYC anchors
13  * during deterministic seeding before measured traffic starts.
14  */
15  class RetailTransferWorkload extends RetailWorkloadBase {
16    async initializeWorkloadModule(workerIndex, totalWorkers, roundIndex, roundArguments
17      , sutAdapter, sutContext) {
18      await super.initializeWorkloadModule(workerIndex, totalWorkers, roundIndex,
19        roundArguments, sutAdapter, sutContext);
20      this.workerIndex = workerIndex;
21      await this.seedPopulation({
22        numCustomers: this.arg('customers', 200),
23        numMerchants: this.arg('merchants', 20),
24        fundedRatio: this.arg('fundedRatio', 1),
25        fundStandard: this.arg('fundStandard', 5000000),
26        fundBasic: this.arg('fundBasic', 500000),
27      });
28      this.configureMeasuredTraffic(this.arg('transactionSlots', 0), totalWorkers);
29    }
30
31    operationKind(slot) {
32      const totalSlots = this.transactionSlots * this.totalWorkers;
33      const globalSlot = (slot * this.totalWorkers) + this.workerIndex;
34      const phase = Math.floor((globalSlot * 100) / totalSlots);
35      if (phase < 70) return 'customer';
36      if (phase < 95) return 'merchant';

```

```

35         return 'read';
36     }
37
38     configureMeasuredTraffic(transactionSlots, totalWorkers) {
39         this.transactionSlots = transactionSlots;
40         this.totalWorkers = totalWorkers;
41
42         const senders = this.customers.slice(0, transactionSlots);
43         const receivers = this.customers
44             .slice(transactionSlots)
45             .filter(customer => customer.tier === 'STANDARD');
46         const required = Array.from({ length: transactionSlots }, (_, slot) => this.
operationKind(slot));
47         const requiredReceivers = required.filter(kind => kind === 'customer').length;
48         const requiredMerchants = required.filter(kind => kind === 'merchant').length;
49
50         if (senders.length < transactionSlots || receivers.length < requiredReceivers ||
this.merchants.length < requiredMerchants) {
51             throw new Error(
52                 'insufficient contention-free population: need ${transactionSlots}
53                 senders, ` +
54                 `${requiredReceivers} STANDARD receivers, and ${requiredMerchants}
55                 merchants`,
56                 );
57
58             let receiverIndex = 0;
59             let merchantIndex = 0;
60             this.transactionPlans = required.map((kind, slot) => {
61                 if (kind === 'customer') {
62                     return { kind, sender: senders[slot], receiver: receivers[receiverIndex
63                     ++] };
64                 }
65                 if (kind === 'merchant') {
66                     return { kind, sender: senders[slot], receiver: this.merchants[
67                     merchantIndex++] };
68                 }
69                 return { kind, customer: senders[slot] };
70             });
71
72             planTransaction(slot) {
73                 const plan = this.transactionPlans && this.transactionPlans[slot];
74                 if (!plan) throw new Error(`transaction slot ${slot} exceeds configured
75                 contention-free population`);
76                 return plan;
77             }
78
79             async submitTransaction() {
80                 const plan = this.planTransaction(this.txIndex++);
81                 if (plan.kind === 'read') {
82                     return this.submit('GetWallet', [plan.customer.walletId, true]);
83                 }
84                 const amount = this.retailAmount(plan.sender.perTxCap);
85                 return this.submit('Transfer', [plan.sender.walletId, plan.receiver.walletId,
amount]);
86             }
87         }
88     }
89 }
90

```

```

86 function createWorkloadModule() {
87     return new RetailTransferWorkload();
88 }
89
90 module.exports.createWorkloadModule = createWorkloadModule;

```

L.11 Source Code *Workload* Beban Dasar Ritel

Bagian ini memuat kode sumber modul inisialisasi populasi dan beban dasar (berkas benchmark/workloads/retail-base.js).

```

1  'use strict';
2
3  const { WorkloadModuleBase } = require('@hyperledger/caliper-core');
4
5  const CONTRACT = { contractId: 'digital-rupiah', contractVersion: '2.0' };
6
7  // Tier per-transaction caps (must match chaincode InitLedger).
8  const PER_TX_BASIC = 250000;
9  const PER_TX_STANDARD = 2500000;
10
11 /**
12  * Shared base for the retail CBDC workloads.
13  *
14  * Seeds a worker- and round-partitioned population (customers and merchants) so
15  * that load spreads across many distinct keys instead of a single hot key-pair.
16  * Provides realistic Indonesian-retail amount sampling and selection helpers.
17  *
18  * IDs are namespaced per worker (and per round unless a fixed 'scenario' is given) to
19  * avoid "already exists" collisions and cross-worker MVCC contention on the same key.
20  */
21 class RetailWorkloadBase extends WorkloadModuleBase {
22     constructor() {
23         super();
24         this.customers = []; // { id, walletId, tier, perTxCap, funded }
25         this.standardCustomers = []; // subset of customers on the STANDARD tier
26         this.merchants = []; // { id, walletId }
27         this.txIndex = 0;
28     }
29
30     arg(name, def) {
31         const v = this.roundArguments ? this.roundArguments[name] : undefined;
32         return v === undefined ? def : v;
33     }
34
35     // Namespace: per-worker+round by default; per-worker only when a fixed scenario is
36     // set, so a follow-up round can address the same population.
37     ns() {
38         const scenario = this.arg('scenario', null);
39         return scenario ? `${scenario}_w${this.workerIndex}` : `w${this.workerIndex}_r${this.roundIndex}`;
40     }
41
42     customerId(i) { return `c_${this.ns()}_${i}`; }
43     merchantId(i) { return `m_${this.ns()}_${i}`; }
44     walletId(ownerId) { return `wlt_${ownerId}`; }
45     isBasic(i) { return i % 5 === 0; } // ~20% BASIC, rest STANDARD
46

```

```

47     randint(min, max) { return Math.floor(Math.random() * (max - min + 1)) + min; }
48     pick(arr) { return arr[Math.floor(Math.random() * arr.length)]; }
49
50     // Run async tasks with bounded concurrency. Seeding is otherwise sequential and,
51     // with
52     // a 2s orderer BatchTimeout, far too slow; concurrency lets blocks cut on
53     // MaxMessageCount.
54     async runPool(tasks, concurrency = 8) {
55         let next = 0;
56         const workers = Array.from({ length: Math.min(concurrency, tasks.length) },
57         async () => {
58             while (next < tasks.length) {
59                 const i = next++;
60                 await tasks[i]();
61             }
62         });
63         await Promise.all(workers);
64     }
65
66     async submit(fn, args, readOnly = false) {
67         await this.sutAdapter.sendRequests({
68             ...CONTRACT,
69             contractFunction: fn,
70             contractArguments: args.map(String),
71             readOnly,
72         });
73     }
74
75     // Small-value-heavy distribution typical of retail payments, clamped to cap.
76     retailAmount(cap) {
77         const r = Math.random();
78         let a;
79         if (r < 0.70) a = this.randint(5000, 100000);
80         else if (r < 0.92) a = this.randint(100000, 500000);
81         else if (r < 0.99) a = this.randint(500000, 2000000);
82         else a = this.randint(2000000, 2500000);
83         return Math.min(a, cap);
84     }
85
86     pickTwoDistinct() {
87         if (this.customers.length < 2) return null;
88         const a = this.randint(0, this.customers.length - 1);
89         let b = this.randint(0, this.customers.length - 1);
90         if (b === a) b = (b + 1) % this.customers.length;
91         return [this.customers[a], this.customers[b]];
92     }
93
94     // Pick a STANDARD-tier receiver distinct from senderId. Routing most P2P receipts
95     // to
96     // STANDARD wallets avoids expected BASIC balance-cap rejections dominating results.
97     pickStandardReceiver(senderId) {
98         if (this.standardCustomers.length === 0) return null;
99         for (let attempt = 0; attempt < 5; attempt++) {
100             const c = this.pick(this.standardCustomers);
101             if (c.id !== senderId) return c;
102         }
103         return null;
104     }
105 }

```

```

102  /**
103   * Seed merchants and customers (tier mix + funding).
104   * 'fundedRatio' of customers are funded normally; the remainder are underfunded.
105   */
106  async seedPopulation({ numCustomers, numMerchants, fundedRatio, fundStandard,
107    fundBasic }) {
108    const merchantTasks = [];
109    for (let i = 0; i < numMerchants; i++) {
110      const id = this.merchantId(i);
111      const walletId = this.walletId(id);
112      this.merchants.push({ id, walletId });
113      merchantTasks.push(async () => {
114        const profileId = `kyc_${id}`;
115        const hashes = JSON.stringify(['sha256:${id}']);
116        await this.submit('SubmitKycProfile', [profileId, 'merchant', id, hashes
117        ]);
118        await this.submit('RefreshKycProfile', [profileId, 'approved', 'low', '
119        standard', false, hashes, '2099-12-31T23:59:59Z']);
120        await this.submit('CreateWallet', [walletId, id, 'MERCHANT']);
121      });
122    }
123    await this.runPool(merchantTasks);
124
125    const customerTasks = [];
126    for (let i = 0; i < numCustomers; i++) {
127      const id = this.customerId(i);
128      const walletId = this.walletId(id);
129      const basic = this.isBasic(i);
130      const tier = basic ? 'BASIC' : 'STANDARD';
131      const perTxCap = basic ? PER_TX_BASIC : PER_TX_STANDARD;
132      const funded = (i / numCustomers) < fundedRatio;
133      const fund = funded ? (basic ? fundBasic : fundStandard) : this.randInt
134      (1000, 20000);
135      const cust = { id, walletId, tier, perTxCap, funded };
136      this.customers.push(cust);
137      if (!basic) this.standardCustomers.push(cust);
138      // Per-customer steps are ordered; different customers run concurrently.
139      customerTasks.push(async () => {
140        const profileId = `kyc_${id}`;
141        const hashes = JSON.stringify(['sha256:${id}']);
142        const diligence = basic ? 'simplified' : 'standard';
143        await this.submit('CreateRetailCustomer', [id, `sha256:identity:${id}`,
144        walletId, profileId]);
145        await this.submit('SubmitKycProfile', [profileId, 'retail_customer', id,
146        hashes]);
147        await this.submit('RefreshKycProfile', [profileId, 'approved', 'low',
148        diligence, false, hashes, '2099-12-31T23:59:59Z']);
149        await this.submit('CreateWallet', [walletId, id, tier]);
150        if (fund > 0) {
151          await this.submit('Mint', [walletId, fund]);
152        }
153      });
154    }
155    await this.runPool(customerTasks);
156  }
157 }
158
159 module.exports = { RetailWorkloadBase, CONTRACT, PER_TX_BASIC, PER_TX_STANDARD };

```